

PROJECT BUILD MANUAL

My Web App V2

Full-Stack Work Tracker

Technical Build Manual & Source Code - Stages 1-5

From the first browser page to a persistent, authenticated application with Admin management, a personal User workspace, and Stage 5 regression coverage.

Prepared 30 September 2026. Repository checkpoint: **main / 8dea970**.

This expanded edition combines the build history with implementation walkthroughs, workflow diagrams, full numbered source listings, and an extractable source archive. It serves as a study guide, a rebuild reference, and evidence of the completed application scope.

Stage 6 is a future step. It is not documented as completed.

Backend tests*	Frontend tests*	Live API checks*
56 passing	20 passing	89 requests passed

*Historical Stage 5 results recorded in docs/stage-5-testing.md; these counts are not a new live certification.

Reading guide & evidence

How to use this manual

Read chapters 1-4 for the architecture and startup sequence. Chapters 5-11 explain the staged build. Chapters 12-15 connect Git history, testing, defects, and verification. Chapters 16-18 provide troubleshooting, the Stage 6 boundary, and a practical API reference.

The stage headings follow the requested learning sequence. Actual commits interleave the work: the backend and PostgreSQL foundations landed on September 11, before the larger frontend implementation on September 14. Stage numbers are a learning map, not a claim that commits followed an uninterrupted numerical order.

What the evidence can establish

The current README, six project guides, database schema, selected implementation files, package manifests, and Git history are the primary sources. Early work is reconstructed from commits, not a complete transcript of earlier conversations. Unrecorded commands, decisions, or test sessions are not invented.

The Stage 4.6A, Stage 4.6B, and Stage 5 verification sections below retain historical results. Their references to work being uncommitted describe the time of those checks. Current Git history supersedes those status notes: Admin management is 1295fd4, User experience is 2198120, and stabilization is 8dea970 on main.

This documentation task reads source and history and validates the generated PDF. It does not rerun the 89-request live verifier, repeat browser testing, re-audit dependencies, or modify production/application data. Recorded zero-vulnerability results describe the earlier audit, not a permanent security guarantee.

Private environment files and historical secret values are excluded. Examples use placeholders or fictional accounts. The historical credential issue mentioned by the Stage 5 record is not reproduced or remediated by this document.

Contents

Full-Stack Work Tracker	1
Reading guide & evidence	2
1. Project overview	7
2. Technology stack & architecture	8
3. Project structure & data model	9
4. Setup & reproducible local startup	10
5. Stage 1 - Frontend foundations	12
6. Stage 2 - Backend & REST API	13
7. Stage 3 - PostgreSQL integration	14
8. Stage 4 - Full-stack integration	15
9. Stage 4.5 - Authentication & RBAC	16
10. Stage 4.6A - Admin management	18
11. Stage 4.6B - User experience	21
12. Git workflow & project chronology	24
13. Stage 5 - Testing & stabilization	25
14. The four Stage 5 defects	29
15. Final verification & capability matrix	30
16. Troubleshooting & lessons learned	31
17. Current state & Stage 6 boundary	32
18. API reference & request examples	33
Source register	37
19. Technical edition: scope & navigation	38
20. Runtime architecture & trust boundaries	39

21. React component tree & state transitions	41
22. HTTP pipeline & task workflow	43
23. Authentication & revocation sequences	45
24. Database design, locks & migrations	47
25. Validation, errors & security contracts	49
26. Test architecture & engineering workflow	50
27. Complete source-code appendix	52
backend/config/database.js	53
backend/config/environment.js	54
backend/config/management.js	55
backend/config/passwords.js	56
backend/config/session.js	57
backend/server.js	58
backend/middleware/auth.js	60
backend/middleware/errors.js	61
backend/middleware/jsonBody.js	62
backend/routes/auth.js	63
backend/routes/profile.js	65
backend/routes/tasks.js	67
backend/routes/users.js	71
backend/database/accounts.sql	74
backend/database/admin-management.sql	75
backend/database/schema.sql	76
backend/scripts/activate-legacy-user.js	77
backend/scripts/bootstrap-admin.js	78
backend/scripts/check-database.js	79
backend/scripts/init-database.js	80

backend/scripts/migrate-accounts.js	81
backend/scripts/migrate-management.js	82
backend/scripts/verify-stage5.cjs	83
frontend/src/App.jsx	86
frontend/src/components/ConfirmDelete.jsx	90
frontend/src/components/DataState.jsx	91
frontend/src/components/PriorityBadge.jsx	92
frontend/src/components/StatusBadge.jsx	93
frontend/src/components/TaskForm.jsx	94
frontend/src/components/UserForm.jsx	97
frontend/src/data/taskOptions.js	99
frontend/src/index.css	100
frontend/src/main.jsx	106
frontend/src/pages/AdminTasks.jsx	107
frontend/src/pages/Dashboard.jsx	109
frontend/src/pages/Login.jsx	111
frontend/src/pages/Profile.jsx	112
frontend/src/pages/Tasks.jsx	114
frontend/src/pages/Users.jsx	116
frontend/src/services/api.js	118
frontend/src/services/management.js	121
frontend/src/services/personal.js	122
backend/test/api.test.js	123
backend/test/auth.test.js	128
backend/test/database-outage.test.js	132
backend/test/environment.test.js	133
backend/test/management.test.js	134
backend/test/profile.test.js	138
backend/test/startup.test.js	142

frontend/test/api.test.js	143
frontend/test/management.test.js	146
frontend/test/personal.test.js	147
.gitignore	149
backend/.env.example	150
backend/package.json	151
frontend/.env.example	152
frontend/index.html	153
frontend/package.json	154
frontend/vite.config.js	155

1. Project overview

Evidence: README.md; Git history

The Work Tracker is a focused full-stack application for assigning work and tracking progress. Admins manage accounts and tasks. Regular Users see their own workload, update task status, and maintain their own profile and password.

The project is a structured rebuild of the earlier my-web-app project. The README explicitly keeps that earlier project unchanged as a learning reference. The new repository separates frontend, backend, database, and documentation so each layer can be understood and verified independently.

Learning objectives

- Understand how HTML, CSS, JavaScript, and React produce an interactive browser experience.
- Trace an HTTP request from a React form through Express validation and authorization to PostgreSQL.
- Distinguish temporary component state from persisted database records.
- Enforce identity, roles, ownership, and business rules on the server.
- Use Git checkpoints, isolated tests, live database checks, and browser verification together.

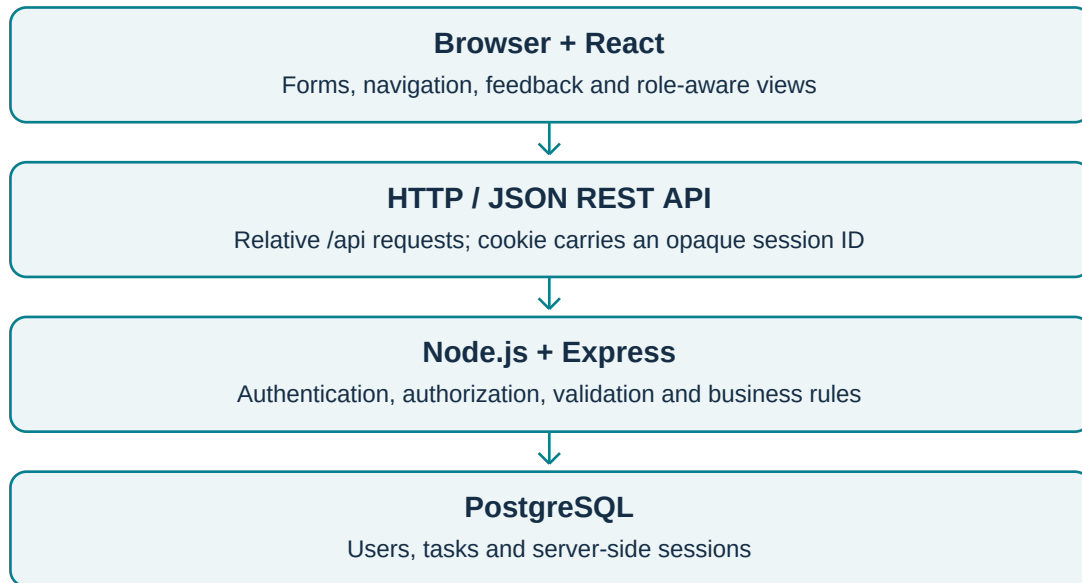
Deliberately focused scope

The application uses simple page selection and shared React state, inline details and confirmations, and local filtering. It does not add a routing framework, charting package, real-time transport, public registration, email delivery, or password-reset service. The documented implementation keeps each new technology tied to a concrete requirement.

At Stage 5 the result is a verified local application and a useful future DevOps workload. Hosting, deployment automation, monitoring, backup operations, and scaling are not established by the completed application work.

2. Technology stack & architecture

Evidence: package manifests; backend/server.js; frontend/vite.config.js



Local ports and responsibilities

Component	Local port	Role
React / Vite development	5173	Serves the frontend and proxies /api
Express API	3000	Handles requests and accesses PostgreSQL
PostgreSQL	5432	Stores durable records and sessions
Vite preview	4173	Locally verifies the built frontend with API proxy

The browser calls /api on its own origin. Vite forwards those requests to Express, by default at `http://127.0.0.1:3000`. This local arrangement needs no additional CORS middleware. The production static bundle does not contain a working reverse proxy; a future host must route /api separately.

Libraries at the documented checkpoint

Package manifests declare React/React DOM ^19.3.0, Vite ^8.3.0, @vitejs/plugin-react ^6.1.1, Express ^5.2.1, pg ^8.23.0, bcryptjs ^3.0.3, express-session ^1.19.0, connect-pg-simple ^10.0.0, and express-rate-limit ^8.7.0. These are declared ranges; npm ci installs the lockfile resolutions rather than choosing arbitrary latest versions.

Frontend source uses ES modules; backend source uses CommonJS. Node's built-in test runner provides automated tests. Postman supports manual API inspection. Git records local history; GitHub is part of the intended stack, but a remote publication or merge mechanism is not proven by the inspected local history.

3. Project structure & data model

Evidence: backend/database/schema.sql; source tree

```
my-web-app-v2/
  frontend/
    src/App.jsx          shared state, session and page coordination
    src/pages/           Dashboard, Users, AdminTasks, Tasks, Profile, Login
    src/components/      forms, badges, confirmation and data states
    src/services/        HTTP service and filtering/count helpers
    test/                API service and management/personal helpers
    vite.config.js       development and preview proxy
  backend/
    server.js            middleware, routes, health and startup
    config/              database, environment, passwords, sessions, locks
    middleware/          authentication, request bodies and errors
    routes/              auth, users, tasks and profile
    database/            final schema and additive migrations
    scripts/             setup, checks, bootstrap and live verification
    test/                route, security, environment and failure tests
    docs/                setup, API, account and stage verification guides
```

Entity relationships

One user can have many assigned tasks. Each task has at most one assignee and may be unassigned. The nullable tasks.user_id foreign key references users.id. Sessions identify users through their stored JSON payload; the sessions table has no declared user foreign key in this schema.

Table	Key fields	Rules
users	id, name, email, password_hash, active, role, created_at, updated_at	Serial primary key; case-insensitive unique email; admin/user role; legacy password hash may be null
tasks	id, title, description, status, priority, user_id, created_at, updated_at	Status and priority checks; optional user foreign key
sessions	sid, sess, expire	Opaque ID primary key; JSON session data; expiry index

The foreign key uses ON DELETE SET NULL to preserve tasks if users are deleted externally. The application API is stricter: it blocks deletion while a user has assigned tasks, including completed work. Database constraints and application business rules therefore complement each other.

Timestamps are stored without time zone. API mutations explicitly update updated_at; external SQL does not trigger an automatic refresh. Keep local application/database timezone assumptions consistent until a deliberate migration addresses them.

4. Setup & reproducible local startup

Evidence: README.md; docs/database.md; docs/accounts.md

Prerequisites and repository

The README calls for Node.js 22.12+ and recommends Node 24. Stage 5 recorded Windows/PowerShell, Node 24.21.0, and PostgreSQL 18. These are project-recorded requirements and environment details, not a new recommendation about the latest releases.

Start from the existing checkout, or clone your own repository URL into a new directory. The exact original clone URL and shell commands were not preserved in the inspected evidence. For a new empty repository, git init establishes history; do not run project-creation scaffolding over this existing app.

New database only

Create my_web_app_v2 once using an authorized PostgreSQL administrator. Enter the password at the prompt. Do not recreate an existing database.

```
createdb -U postgres my_web_app_v2
```

In backend, copy .env.example to .env only if the private file is absent. Configure DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, and PORT. Set a privately generated SESSION_SECRET of at least 32 characters and private BOOTSTRAP_ADMIN_NAME, BOOTSTRAP_ADMIN_EMAIL, and BOOTSTRAP_ADMIN_PASSWORD for first-Admin setup. Never paste real values into this document.

For an empty database, run the following from backend after configuration:

```
npm ci
npm run db:init
npm run admin:bootstrap
npm run db:check
npm start
```

Existing Stage 4 database

Use additive migrations to preserve records and sequences. db:init creates missing tables; it does not upgrade existing definitions.

```
cd backend
npm ci
npm run db:migrate:accounts
npm run db:migrate:management
npm run admin:bootstrap
npm run db:check
npm start
```

Bootstrap is safe to repeat: it does not reset passwords or duplicate the first active Admin. Remove its private password input afterward if no longer needed. Existing legacy users remain separate from newly bootstrapped accounts.

Frontend in another terminal

```
cd frontend
npm ci
npm run dev
```

Open <http://localhost:5173>. Keep both application servers and PostgreSQL running. If the backend port changes, set `API_PROXY_TARGET` in the frontend's private environment configuration and restart Vite. Never expose database settings through `VITE_` variables or frontend code.

Verification commands

Run `npm test` separately in backend and frontend. Run `npm run build` in frontend; `npm run preview` then serves the production build locally on 4173. `GET http://localhost:3000/api/health` confirms a successful database query. `db:check` does not certify every schema column, and `health` does not certify application-table structure.

5. Stage 1 - Frontend foundations

Evidence: f622594; 12567bd; 2e0c062; 637bb26

The first browser foundation used an HTML page, a stylesheet, and a small JavaScript file. Commit 12567bd corrected the stylesheet filename from `style.sc.ss` to `styles.css`. This is a useful early example of checking asset paths before assuming the styling logic is wrong.

The React foundation introduced Vite, `main.jsx`, `App.jsx`, and `useState`. Its initial Test React button changed a message in component state, proving that an event could update the rendered page without a full reload.

The Work Tracker frontend then added Dashboard, Users, and Tasks pages; reusable `UserForm` and `TaskForm` components; status badges; styling; and mock records. This created a visible workflow before connecting it to a server. Mock data was a development stepping stone, not persistence.

Core code concepts

Props pass data and callbacks into components. State stores the current UI values and selected page. A controlled form keeps its inputs in state, validates them, and calls an action when submitted. Reused forms and badges keep behavior consistent across views.

The current App coordinates the account, shared records, and page selection. Later stages added loading/error states and authenticated views to this foundation rather than introducing an unrelated UI framework.

Learning checkpoint

Explain which values disappear on refresh when they exist only in React state. Identify the event handler that changes a task and trace the callback to shared state. Confirm that a visual change alone does not demonstrate a database write.

The early commits prove the implementation milestones, but no separate comprehensive Stage 1 test report is available. Later Stage 5 browser evidence verifies the final combined application, not every intermediate historical screen.

6. Stage 2 - Backend & REST API

Evidence: 3a2b8fb; e48322b; backend/server.js; docs/api.md

The initial Express backend exposed GET /api/health on port 3000 and returned JSON showing that the backend was running. It established a process outside the browser and a URL that could be checked independently of the React UI.

The fuller user/task routes, environment validation, safe errors, and API tests arrived with the full-stack integration commit e48322b. The repository does not show a separate earlier commit containing the entire mature REST API. This manual groups these concepts under Stage 2 while preserving that historical distinction.

Request lifecycle

Express parses JSON with a 100 KB limit. Health is handled independently of session lookup. Other API requests receive no-store cache behavior, then session middleware, write-request verification, account loading, route-specific access checks, and route logic. Unknown routes return safe 404 JSON; final error middleware handles failures.

Route code validates body shape, strings, IDs, enumerations, and references. Parameterized queries separate submitted values from SQL text. Expected conflicts return meaningful status codes instead of exposing database errors or stack traces.

REST conventions used here

GET reads. POST creates or performs login/logout actions. PUT submits complete editable fields. PATCH changes a narrow field such as task status or account active state. DELETE removes a resource subject to safeguards. A task PUT applies defaults to omitted fields, so clients should send the complete form.

Testing from the outside

Postman can call the API without React, making it possible to isolate backend behavior. For authenticated writes, retain the login cookie, use JSON where required, and send X-Requested-With: WorkTracker. Test both success and rejection paths: malformed input, duplicate email, missing records, unauthorized roles, and unowned tasks.

Node tests import the application without starting the normal server, use an ephemeral port, and mostly substitute database queries and session storage. This makes failures repeatable, but those tests cannot alone prove the real database schema or persistence.

7. Stage 3 - PostgreSQL integration

Evidence: f825961; docs/database.md; backend/config/database.js

The PostgreSQL foundation introduced `pg`, a database configuration module, schema SQL, and a health check that queried the database. The later integration work strengthened environment validation, initialization, timeouts, constraints, and connectivity checks.

The browser never connects to PostgreSQL. Express owns a shared `pg.Pool` and uses it to execute queries. A pool reuses connections; `pool.query` borrows and returns one automatically. Successful writes are committed before the API responds, so a later GET or browser refresh reads saved records.

Transactions and migrations

Schema initialization runs in a transaction and creates missing tables without resetting data. Account and management migrations are separate additive upgrades. A transaction commits all its changes together or rolls them back, preventing a partly upgraded account schema or partly applied account-management operation.

Database constraints enforce primary keys, unique email, valid task status/priority, and assignment references. Application rules add requirements such as preventing inactive assignments and preserving an active usable Admin.

Diagnose one layer at a time

First verify the PostgreSQL service, host, port, database, and role. Then run `db:check` and inspect health. A 28P01 code means authentication failed; check private configuration and shell overrides. Do not disable authentication or reset the database as a troubleshooting shortcut.

Connections time out after five seconds and SQL statements after ten seconds. Idle pool errors are handled. A database outage should produce safe errors while Express remains available to answer requests that do not require a successful database lookup.

Persistence checkpoint

Create uniquely named temporary records through the API, retrieve them, reload the UI, and compare the database independently. Delete only those records afterward. Do not reset sequences: gaps after legitimate testing are normal.

8. Stage 4 - Full-stack integration

Evidence: e48322b; frontend/src/services/api.js; README.md

Commit e48322b removed mockData.js and connected the Work Tracker to real API records. The frontend API service centralizes fetch calls, JSON handling, readable errors, record-shape checks, cancellation, and timeouts. Components deal with records and user feedback rather than repeating transport logic.

Example: saving a task

1. A form collects a title, description, priority, status, and optional assignee.
2. The service sends a JSON request to /api through Vite's proxy.
3. Express verifies the account and permission, validates the fields, and writes PostgreSQL.
4. The API returns the saved record with ID and timestamps.
5. React updates shared state only after success; the dashboard and list reflect the saved record.
6. Refresh reloads the same persisted record from the API.

Loading and failure behavior

Initial failures show a readable error and Retry. Submitted forms preserve input when a request fails and disable pending actions. A 15-second frontend timeout bounds waiting. The timeout message tells the user to reload and check whether a write completed before retrying, because an interrupted response does not prove that the database write failed.

The service avoids showing raw HTML from proxy errors. It validates returned records before components consume them, and treats unexpected JSON or data shapes as readable failures. Cancelled page loads are handled without unnecessary UI changes.

Integration checkpoint

Verify an Admin-created task in the database, refresh the UI, update its status from its assigned account, and confirm the Admin sees the result. Stop and restart the backend during a controlled local check; Retry should recover while preserving saved state. Stage 5 contains the recorded evidence for the final implementation.

9. Stage 4.5 - Authentication & RBAC

Evidence: docs/accounts.md at 8dea970

Admins manage accounts and all tasks. Users see only their assigned tasks, update their status, and edit their own name/email and password. There is no public registration. See Regular User experience for self-service validation and password-change session revocation.

Upgrade an existing Stage 4 database

From backend/, install the locked dependencies with `npm ci`, then run `npm run db:migrate:accounts`. This transactional migration adds `password_hash`, `role`, a case-insensitive email index, and the `sessions` table. It preserves all existing users, tasks, assignments, and sequences. A conflicting legacy email causes rollback rather than automatic merging or deletion.

Existing users become role user with a null password hash and cannot log in. The Admin list labels these accounts "login not enabled". To activate one deliberately, set `LEGACY_ACCOUNT_EMAIL` and `LEGACY_ACCOUNT_PASSWORD` in the ignored local `.env`, run `npm run account:activate`, then remove those temporary settings. The script only enables a legacy User without a password; it cannot overwrite an existing password or grant an Admin role. Assigned tasks remain attached to the same user ID. This does not change the separate active/inactive status; an inactive account must also be reactivated by an Admin.

For an empty future database, `npm run db:init` creates the final schema directly. Do not use schema initialization as a substitute for upgrading existing tables.

Run `npm run db:migrate:management` afterward for account status. See Admin management for the new account safeguards and session revocation.

First Admin

Set these values privately in backend/ `.env`, never `.env.example`:

- `SESSION_SECRET`: a cryptographically random value of at least 32 characters.
- `BOOTSTRAP_ADMIN_NAME`: the first Admin's name.
- `BOOTSTRAP_ADMIN_EMAIL`: an email not already used by a legacy record.
- `BOOTSTRAP_ADMIN_PASSWORD`: your private initial account password.

Generate a session secret without displaying it using Node's `crypto.randomBytes` and write it directly into the ignored local environment file. Keep this secret stable across backend restarts; changing it invalidates existing login cookies. It is unrelated to `DB_PASSWORD`. Do not use the example placeholder.

Run `npm run admin:bootstrap`. The script hashes the password, locks against concurrent bootstrap attempts, and creates the first Admin only if no active Admin exists. Repeating it does not reset passwords or create duplicates. Remove the bootstrap password from `.env` after setup if you no longer need the input.

Passwords must contain at least 12 characters and no more than 72 UTF-8 bytes. The byte limit prevents bcrypt's silent truncation. Passwords are not trimmed. Normal account creation requires name, email, initial password, and role (defaults to user). Deliver initial credentials privately; the application sends no email.

Sessions and authorization

`express-session` signs an opaque cookie ID with `SESSION_SECRET`. `connect-pg-simple` stores session data in PostgreSQL and prunes expired records. Only the user ID and cookie metadata are stored in the session; roles and active status are re-read from users on each request. Passwords use `bcryptjs` with cost 12 and random salts. Neither plaintext passwords nor password hashes are returned by any API.

Cookies use `HttpOnly`, `SameSite=Lax`, and an eight-hour lifetime. Local HTTP works in development. `NODE_ENV=production` requires HTTPS cookies; any future reverse proxy trust configuration must match the actual deployment. This stage adds no deployment infrastructure. Do not run the real server with `NODE_ENV=test`, which is reserved for the isolated tests' in-memory session store.

Login regenerates the session ID. Logout deletes the server-side session and clears the cookie. On refresh, React calls `/api/auth/me` to restore the account. An expired session returns the UI to login; passwords and tokens are not placed in `localStorage`. Authenticated API responses use `Cache-Control: no-store`.

All state-changing API calls require `X-Requested-With: WorkTracker`. Combined with JSON requests, `SameSite` cookies, no cross-origin CORS permission, and rejection of `Sec-Fetch-Site: cross-site`, this prevents cross-site forms/scripts from using the session to perform writes. The frontend service adds the header automatically; Postman callers must add it and preserve the login cookie.

Login is limited to 20 attempts per IP per 15 minutes. This simple limiter lives in the Node process; it resets on restart and is not shared across future replicas.

Authentication identifies the account. Express role checks reject Admin actions from Users. Task lists and status-update SQL include ownership restrictions; supplying another `userId` or `role` in the request cannot change those restrictions. React hides unavailable pages for usability, not as a security boundary.

Verification

Run `npm test` in backend and frontend, then `npm run build` in frontend. Backend tests retain the original API validation cases and add session, role, ownership, invalid-cookie, request-verification, and rate-limit tests.

Browser check: sign in as Admin, create a User, assign a task, and log out. Sign in as that User, verify only assigned work is visible, change status, refresh, check Profile, and log out. Sign back in as Admin to confirm the persisted status. Also call protected endpoints manually with no cookie and with a User cookie; expect 401 and 403 respectively. A User updating an unowned or missing task gets 403 without revealing whether another user's task exists.

Keep the Stage 4 Git checkpoint unchanged. Rolling code back does not reverse the additive database migration; preserve account data and make any rollback decision explicitly rather than dropping columns or tables.

10. Stage 4.6A - Admin management

Evidence: docs/admin-management.md at 8dea970

Upgrade and startup

On the existing feature branch, from backend/, run:

```
npm run db:migrate:accounts
npm run db:migrate:management
npm run db:check
npm start
```

For an empty database, `db:init` includes the final schema. The management migration adds `users.active BOOLEAN NOT NULL DEFAULT true` and ensures `updated_at` exists. It is transactional and repeatable, with a five-second lock timeout. It does not reset data, recreate tables, or reset sequences. Existing account status starts active; legacy records without a password still cannot sign in. The account migration and management migration must both run before bootstrap on older databases.

User management

Admins create accounts with name, email, initial password, and admin/user role. List cards show role, active status, creation date, and whether legacy login is enabled. View loads current details from the API, including updated date and counts of assigned tasks by status. Edit changes only name, email, and role; no password field is shown or accepted in this endpoint. Email uniqueness is case-insensitive.

Deactivate preserves all assignments, rejects new logins with the usual generic login error, and deletes stored sessions. Every authenticated request rereads account status, so revoked access cannot continue. React returns to sign-in on the next 401. Reactivation allows a fresh login; it does not restore old cookies. Legacy login enablement and active/inactive status are separate concepts.

Deletion requires explicit confirmation and is blocked with 409 while any tasks, including completed tasks, are assigned. Reassign or delete those tasks first. No API operation cascades task deletion. The existing foreign key remains `ON DELETE SET NULL` for external database operations; API safeguards are stricter.

An Admin cannot delete, deactivate, or demote their current account. At least one active Admin with a password must remain. A legacy record without login cannot be promoted until its login is deliberately enabled. Role changes revoke the target's sessions. The acting Admin is rechecked within management transactions. Account changes and assignment writes share a short `users-table` lock, making concurrent safeguards understandable for this small local application. This favors correctness over high write throughput; revisit lock granularity only when workload requires it.

Task management

Admins create, view, edit, and delete tasks. Create and Edit support title, description, priority, status, and optional assignment. View includes full description, assignment, and created/updated dates. Delete requires explicit confirmation.

A new assignment cannot target an inactive account. Editing other fields may retain an existing inactive assignment so historical work remains manageable; the select labels that option explicitly. Unassignment and reassignment are allowed. On the next API request the former assignee cannot read/update the task and the new assignee can. Open tabs refresh to see changes from another session; this stage adds no polling or real-time transport. User status updates keep their atomic ownership restriction.

UI and dashboard

Inline detail sections, reused edit forms, and inline delete confirmations avoid a new router or modal focus-management system. Saves update shared React state only after API success. Errors preserve form values and deletion targets; pending actions are disabled. Switching pages cancels the current edit. Search/filter state resets when leaving a management page. Browser refresh reloads persisted API data.

Users support name/email search, role/status filters, and name/newest sorting. Tasks support title search, status/priority/assignee filters (including unassigned), and newest/title/highest-priority sorting. Filters combine, run locally on the API-returned arrays, and do not change database records or dashboard totals.

Dashboard cards show total, active, and inactive users; Open, In Progress, Completed, and unassigned tasks; and task counts by priority. Active includes legacy assignment records marked active, even if login is not enabled. Priority counts include completed work. No charting dependency was added. Regular User pages retain their existing scope.

Manual verification

1. Sign in as Admin; refresh and verify session restoration and dashboard totals.
2. Create two temporary Users with unique test emails. View details, edit name/email, exercise search/filter/sort, and try a duplicate email (expect preserved input).
3. Deactivate one User; login must fail. An already signed-in session must lose access on its next request. Reactivate, then sign in again with the same password.
4. Create an assigned task, view its details, edit title/description/priority/status, and refresh. Verify the User can update only their own task and view Profile.
5. Reassign to the second User. Refresh both accounts: only the new assignee sees it.
6. Try deleting an assigned User: expect 409 and an explanatory message. Cancel a task deletion first, then confirm deletion of the temporary task. Delete only the temporary Users afterward and verify dashboard counts return to baseline.
7. Call management routes with no cookie (401) and a User cookie (403). Try self deletion/deactivation/demotion as Admin (409), invalid IDs/roles/statuses (400), missing records (404), and inactive assignments (400).
8. Check console, failure feedback/retry, small-screen layout, and persistence.

Run `npm test` in each app and `npm run build` in frontend. Automated HTTP tests use mocked PostgreSQL queries; actual database/browser checks remain necessary. No real credentials belong in fixtures, documentation, examples, or screenshots. Verification records must be cleaned up without resetting sequences.

Stage verification - 2026-09-29

Verified on `feature/admin-management`, based on `a987d78`, with no commit or merge. Backend: 42 tests passed. Frontend: 13 tests passed; production build passed. JavaScript syntax and Git whitespace checks passed. Backend production dependencies and the full frontend dependency audit both reported zero vulnerabilities.

Live PostgreSQL/API and browser checks covered account CRUD, blocked deletion, inactive login and session revocation, Admin self-protection, task CRUD, ownership after reassignment, filters, sorting, refresh persistence, regular User regression, and backend outage/retry recovery. Desktop and mobile layouts were checked. Browser console checks found no application warnings/errors during normal flows.

Temporary Stage46 Jane/John accounts and one verification task were deleted through the API/UI. The database retains four existing users and two existing tasks (one Open, one Completed). The temporary credential file was removed. No sequences were reset. Current credentials were absent from tracked/new files; local .env remains ignored, and .env.example contains placeholders. PostgreSQL credentials were unchanged.

Expected validation/authentication failures were observed during negative tests. Git reported only Windows LF/CRLF conversion warnings. The first dependency audit was blocked by the sandbox; rerunning with network permission succeeded. No Stage 4.6A feature remains incomplete; ready for manual review.

11. Stage 4.6B - User experience

Evidence: docs/user-experience.md at 8dea970

Personal workload

The User navigation remains Dashboard, My Tasks, Profile, Logout. Express filters GET `/api/tasks` by the session's account ID; React receives only that User's work. There is no separate `/tasks/my` endpoint. The personal Dashboard shows Open, In Progress, Completed, total assigned tasks, counts by priority, and up to three unfinished tasks ordered by priority (newest first within a priority). Priority totals include completed work. No organization-wide statistics appear.

My Tasks supports case-insensitive title/description search, combined status and priority filters, and newest/oldest/highest-priority/status sorting. Status sorting places Open before In Progress before Completed. Sorting operates on a copy of API data; filters do not change dashboard counts. Filters reset on navigation or refresh. There are separate empty-workload and no-filter-matches messages.

View Details fetches GET `/api/tasks/:id` under the same ownership restriction and shows full description, priority, status, and created/updated dates. The existing PATCH `/api/tasks/:id/status` updates only the status and timestamp. Users may choose Open, In Progress, or Completed; any transition is allowed. The displayed status changes only after a successful save. If the new status no longer matches a filter, the task leaves the list and the feedback explains why.

Users cannot create/delete tasks or change title, description, priority, or assignment. Admin task PUT/DELETE/POST return 403 to Users. An unowned detail request returns 404; an unowned or missing status target returns 403 without revealing existence. User status requests containing any field other than status return 403. These rules are enforced by Express/SQL, regardless of manually supplied IDs or role parameters. Reassignment takes effect on subsequent requests; refresh to see another session's changes. This stage adds no polling or real-time transport.

Profile

GET `/api/auth/me` is reused for safe account data: ID, name, email, role, active status, created date, and updated date. No new profile-read endpoint is needed. PUT `/api/profile` accepts exactly name and email; identity comes only from the session. Extra keys (including role, active, ID, password, and hash) are rejected. Name must be nonblank and at most 100 characters. Email must have a basic valid shape and be at most 255 characters; both fields are trimmed and email lowercased. Case-insensitive duplicate emails return 409. The database updates `updatedAt`.

After success, React refreshes the safe account/header data without unmounting the form. Failed profile requests preserve input. Role and status remain read-only. Changing email does not verify mailbox ownership or send email. Use the new email for the next login; password and active sessions otherwise remain unchanged.

Change password, not password reset

PUT `/api/profile/password` accepts exactly `currentPassword`, `newPassword`, and `confirmPassword`. It requires an authenticated active account, verifies the current bcrypt hash, validates matching confirmation, and uses the existing `bcryptjs` cost-12 hashing helper. Policy remains at least 12 characters and no more than 72 UTF-8 bytes. Passwords are not trimmed; the new password must differ from the current password. Wrong current passwords, malformed values, policy failures, and confirmation mismatch return safe 400 errors. Attempts are limited to 10 per account per 15 minutes using the existing rate-limit package; this local in-process limit resets on restart.

The new hash, updated timestamp, and deletion of all PostgreSQL sessions for the account commit together in the existing management transaction. The request's session is destroyed and its cookie cleared. React clears account/workload state and shows 'Your password has been changed. Please sign in again.' Only the new password works. The password-change form clears password inputs after a submitted request; passwords are never stored in URLs, localStorage, or sessionStorage, or returned/logged by the API.

Login rechecks the credential after saving its session. FOR SHARE waits for an in-flight password transaction, preventing an old-password login racing the change from leaving a surviving session. Password changes are serialized with existing account management so two simultaneous changes cannot verify the same outdated hash.

The profile APIs act only on the authenticated account (including an Admin calling them directly); no Admin profile navigation or management redesign was added. All writes retain the request-verification header, HttpOnly/SameSite cookie behavior, active-account checks, parameterized SQL, and safe JSON errors.

Database and startup

No schema, migrations, dependencies, or environment settings were added. Existing users/password_hash/updated_at and sessions tables suffice. Use the existing README startup commands after the Stage 4.6A migrations. Do not reset the database or sequences. PostgreSQL, bootstrap Admin credentials, and the session secret remain unchanged.

Manual verification

1. As Admin, create a uniquely named temporary User and three assigned tasks with different priorities/statuses. Keep unrelated tasks assigned to another account.
2. Sign in as that User. Verify own dashboard counts and absence of Admin navigation.
3. Search by title/description, combine filters, exercise all sorts, and inspect details. Change own task status; refresh and verify persisted status/counts.
4. View Profile. Try a duplicate email, then save a new name/email. Check the header, profile timestamps, refresh persistence, and new email login.
5. Establish a second session for the temporary account. Try a wrong current password, weak/overlong new password, and confirmation mismatch. Successful change must sign out, reject the old password and copied old cookie, and accept the new password.
6. As Admin, verify saved profile/status changes; exercise CRUD, deactivation, reactivation, reassignment, safe deletion, and existing Admin safeguards.
7. Call APIs directly as User: another task detail must return 404; another task's status and Admin CRUD must return 403; profile role/active/ID fields must return 400. Unauthenticated profile requests return 401 and inactive accounts stay blocked.
8. Check desktop/mobile layout, empty tasks and empty filters, preserved input on failures, backend outage/retry, refresh, and browser console.
9. Delete only the temporary tasks and User. Keep legitimate data and sequences.

Run `npm test` in backend and frontend, plus `npm run build` in frontend. Unit HTTP tests mock SQL (and adapt session deletion to their MemoryStore); live PostgreSQL verification is needed to prove actual cross-session revocation. No additional test framework is required.

Verification - 2026-09-29

Branch: feature/user-experience, HEAD 1295fd4; no commit, branch switch, or merge. 52 backend tests and 19 frontend tests passed. Existing Admin tests remain intact. Production build, JavaScript syntax, and Git whitespace checks passed. Backend production and full frontend dependency audits reported zero vulnerabilities. No dependencies or schema changes were introduced.

Browser verification passed for personal counts, task search/filter/sort/details, status persistence, safe profile fields, duplicate email feedback, successful name/email editing, wrong-current-password rejection, password change with required re-login, old-password failure, and new-password success. A separate pre-change session was rejected against real PostgreSQL. API bypass checks rejected unowned tasks, Admin CRUD, privileged profile fields, and forbidden task fields. Admin views reflected the User changes; management regression and inactive-account checks passed. Desktop/mobile layout, empty workload, form failure/input retention, backend outage/retry recovery, and normal console checks passed. Expected negative requests and intentional outage failures were observed; no normal-flow application console errors were found.

Created one temporary Stage46B User and three High/Medium/Low tasks through Admin management. Renamed the temporary profile and changed only its password. Deleted all three tasks and the temporary User through Admin management after verification. The database retains three legitimate users and two legitimate tasks. The temporary credential file was removed and sequences were not reset. The permanent Admin and PostgreSQL credentials were unchanged. Local .env remains ignored/untracked, examples contain placeholders, and current-secret scanning of tracked/new files found zero matches.

An initial test fixture generated a different hash on every mock read; it was corrected to model a stable stored hash for the new login recheck. All tests then passed. Git reports Windows LF/CRLF conversion notices only. No remaining Stage 4.6B blockers were identified. Ready for manual review; Stage 5 was not started.

12. Git workflow & project chronology

Evidence: local Git history and branches inspected 30 September 2026

The inspected checkout is on main at 8dea970. Local branches also include feature/admin-management, feature/user-experience, and testing/stage-5-stabilization. The visible history is linear; it does not prove whether integration used a fast-forward, another local workflow, or a hosted pull request.

Date	Commit	Milestone
Sep 11	c22e3e0	Initialize project foundation
Sep 11	f622594	Browser frontend foundation
Sep 11	12567bd	Correct stylesheet filename
Sep 11	2e0c062	React frontend foundation
Sep 11	3a2b8fb	Express backend foundation
Sep 11	f825961	PostgreSQL connection
Sep 14	637bb26	Work Tracker frontend with mock data
Sep 28	e48322b	Complete full-stack integration
Sep 29	a987d78	Authentication and role-based access
Sep 29	1295fd4	Admin user and task management
Sep 29	2198120	Regular User experience
Sep 30	8dea970	Stabilization and regression testing

Repeatable working discipline

Inspect status and the diff before editing. Work on a focused branch, preserve unrelated changes, run checks appropriate to the change, review the result, then commit and integrate deliberately. The commands below are a suggested future workflow, not a transcript of the original build:

```
git status --short
git switch -c codex/descriptive-change
git diff
git diff --check
# Run relevant package tests and frontend build.
# Stage only the reviewed files, then commit.
git log --oneline -12
```

Keep .env ignored and secrets out of commits. A code rollback does not reverse a database migration; coordinate schema compatibility and preserve data. Do not rewrite historical commits or drop tables merely to return to an earlier UI checkpoint.

Earlier stage documents identify their starting HEAD and say the working changes were not yet committed. Those are valid snapshots of the testing sessions. The table above records the later completed commits and resolves the apparent conflict with the current main branch.

13. Stage 5 - Testing & stabilization

Evidence: docs/stage-5-testing.md at 8dea970

Verification completed September 30, 2026. Stage 5 is complete and ready for review. The application is suitable as the baseline workload for the next learning stage; this is not a claim of infrastructure or operational production readiness.

Scope and environment

- Branch: testing/stage-5-stabilization; starting HEAD: 2198120.
- Working tree was clean at the initial inspection. Interruptions were resumed without discarding changes.
- Windows / PowerShell, Node.js 24.21.0, PostgreSQL 18, existing my_web_app_v2 database.
- Existing React/Vite frontend on 127.0.0.1:5173, Express backend on port 3000.
- Browser tests used the Codex in-app Chromium browser. Viewports: 1280x900, 768x900, 390x844.
- Feature scope remained frozen. No dependencies, schema changes, credentials, branches, commits, merges, database resets, sequence resets, or DevOps tooling were introduced.
- Existing source, routes, middleware, schema/migrations, configuration, tests, and documentation were reviewed. The four runtime fixes below are intentionally small.

Results and reproducible checks

Check	Initial baseline	Final result
Backend npm test	52 passing	56 passing, 0 failed/skipped
Frontend npm test	19 passing	20 passing, 0 failed/skipped
Frontend npm run build	Previously working	Passed; 32 modules transformed
Backend and frontend npm audit --json	Not repeated as a baseline	Both: 0 known vulnerabilities
Backend npm run db:check	Previously verified	Passed against the existing database
JavaScript syntax and Git whitespace	Reviewed	Passed
Live API verification	Separate from unit-test counts	89 checked requests passed, plus cleanup
Real-record preservation	3 users, 2 tasks	3 users, 2 tasks; unchanged comparison passed

Run tests/build from each package directory with its existing npm scripts. On this Windows machine the npm PowerShell launcher was unreliable; the installed npm CLI was invoked through Node without changing package definitions.

The opt-in backend/scripts/verify-stage5.cjs runs against a running local backend:

```
cd backend
node --env-file=.env scripts/verify-stage5.cjs
```

It reads the local bootstrap Admin credential, generates random disposable account passwords in memory, creates uniquely named test records, checks API responses and PostgreSQL independently, then deletes its own tasks/users and logs out its sessions. It compares all pre-existing user/task rows before and after using a digest without displaying hashes or credentials. It is deliberately outside node - - test: normal automated tests do not mutate the local database. Do not run the live verifier concurrently with someone editing legitimate application data. An interrupted live run can require cleanup of its exact disposable records; do not delete unrelated records.

CRUD and authorization matrix

Operation	Admin	Regular User	Unauthenticated
Create/list/read/edit users	Allowed, validated	403	401
Activate/deactivate/delete users	Allowed subject to safeguards	403	401
Create/edit/reassign/delete tasks	Allowed, validated	403	401
List tasks	All tasks	Assigned tasks only; forged query filters ignored	401
Read task detail	Allowed	Own task allowed; another task returns 404	401
Update task status	Allowed	Own status only; unowned/forbidden fields return 403	401
Read own profile through /api/auth/me	Own safe account	Own safe account	{ "user": null }
Edit own profile / password	Authenticated API allowed	Allowed with validation	401
Change role/active state through self-service profile	Rejected	Rejected	401

Admin User CREATE/READ/UPDATE/DELETE and Task CREATE/READ/UPDATE/DELETE passed both API and browser verification. Assigned users cannot be deleted until work is reassigned/deleted. Self deletion, deactivation, and demotion are blocked. Last-usable-Admin protection and legacy-account promotion safeguards remain covered by automated tests; no permanent Admin was altered.

Direct API attempts covered listing/retrieving/managing other users, arbitrary task access, task creation/deletion/reassignment, privileged status payloads, and self-service privilege changes. Role and ownership decisions are made by Express/SQL, not by the hidden browser controls.

Authentication, sessions, passwords, and negative inputs

- Valid/invalid/malformed login, inactive login, session restoration, logout, copied invalid cookies, and role re-reading passed. Cookies retain HttpOnly and SameSite=Lax; Secure is enabled by the existing production configuration (local HTTP naturally does not use a Secure cookie).

- Deactivation revokes access; reactivation does not revive the old session. Browser access after deactivation returns to Sign In on the next protected request.

- Password change validates the current password, rejects incorrect/mismatched/short/over-72-byte values, stores a bcrypt hash, revokes multiple sessions, rejects the old password, and permits the new password. No permanent account password was changed.
- Validation covered blank/whitespace values, wrong types/arrays/objects, malformed email, duplicate email, excessive lengths, invalid/nonexistent IDs, unsupported role/status/priority, inactive or nonexistent assignees, forbidden fields, malformed JSON, oversized bodies, and query-filter bypasses.
- Hostile-looking SQL/HTML text stayed ordinary data. Browser rendering displayed the literal text; no script ran. Existing query parameterization and React escaping remained intact.
- API responses were checked for password/hash/secret leakage and safe error handling. No plaintext passwords, hashes, session secret, database password, stack traces, or SQL details were returned. Safe account fields and the required signed session cookie remain part of the intended contract.

Browser workflows and persisted state

Admin browser workflow passed: login -> dashboard -> create/view/edit user -> deactivate/reactivate -> create/view/edit task -> assign/unassign/reassign -> search/filter/sort -> delete disposable task -> delete disposable users -> logout. Admin could see the regular user's saved profile and task changes.

Regular-user browser workflow passed: login -> own dashboard/tasks -> search and empty/filter states -> task detail -> status change -> refresh -> persisted completed task and updated dashboard -> profile edit -> duplicate-email failure retaining input -> successful edit -> password change -> mandatory sign-in -> old-password rejection -> new-password login -> logout/revocation.

The dashboard showed one completed assigned task for the disposable user after refresh. Admin counts changed with mutations and returned after cleanup to 3 users, 0 open tasks, 1 in-progress task, 1 completed task, 0 unassigned tasks, and priority totals High 1 / Medium 1 / Low 0.

Pending sign-in/password controls were disabled; success and error feedback appeared. Changing a status that no longer matched the selected filter removed the card and explained why. Detail status updated too. Profile updates changed the header without losing success feedback. Browser refresh restored the PostgreSQL-backed session and saved records. Independent SQL confirmed the completed task, reassignment, edited profile, inactive state, and sensible created/updated timestamps.

Failure/recovery and responsive review

- Backend unavailable: the open page remained usable and showed a readable connection error. A controlled backend stop followed by page refresh showed the service-unavailable message and **Retry loading**. After restarting the backend, Retry restored the same user session and persisted workload. No manual database/session reset was needed.
- Database unavailable: the isolated refused-connection test returned safe 503/500 JSON and no authenticated session. The actual database was never stopped, and its check passed afterward.
- Invalid/revoked session: deactivating the disposable account forced Sign In on its next request.
- Desktop/tablet/mobile: forms, cards, profile details, and action buttons remained usable. Measured page scroll width matched client width. Mobile navigation intentionally scrolls horizontally inside its own container. Password controls and deletion confirmations remained reachable. No redesign or CSS change was necessary; the viewport override was reset after testing.
- Normal workflows showed no uncaught application exceptions or React warnings. The browser's available warn/error log capture returned an empty list. Deliberate duplicate-email, wrong-password, revoked-session, and outage requests were expected failures, not normal-workflow regressions.

Secrets, cleanup, and repository review

- backend/.env remains ignored and untracked; all example secrets remain placeholders, including DB_PASSWORD=your_local_postgres_password.
- Exact-value scanning of tracked and new non-ignored files found no working PostgreSQL password, bootstrap Admin password, or session secret. Generated frontend assets were checked separately.
- No test credential file was created. Disposable passwords existed only in process/browser-test memory; browser variables were cleared. No environment file or permanent credential was changed.
- The live verifier removed its two temporary users and two tasks. Browser testing removed Stage5 Browser Work, Stage5 Edited User, and Stage5 Profile Updated (the latter began as Stage5 Resume User after the interrupted browser's in-memory password was lost).
- Final database count: **3 users and 2 tasks**. A complete-row digest comparison confirmed the legitimate records were unchanged. No sequence was reset. Browser finished signed out.
- Runtime changes are limited to the four fixes above. Test support and this document account for the other changes. No package/lockfile, schema, product-page, or unrelated change was made.

Non-issues and remaining limitations

- Development StrictMode can cancel/restart initial loads; this is expected. The cancellation authentication side effect was the actual defect and has its own regression test.
- Server processes and browser handles did not survive every interrupted tool session. Restarting them restored the app; that tool-lifecycle behavior was not an application defect.
- Git reports Windows LF-to-CRLF conversion notices; whitespace checks pass. No application build warning or dependency vulnerability remained.
- This is local functional/security-boundary verification, not a penetration test, load/stress test, cross-browser certification, or production deployment review. Automated route tests mostly use query/store doubles; the separate real-PostgreSQL checks provide persistence evidence.
- A complete browser network HAR and duplicate-request trace were not available through the browser tool. Successful UI results, direct API checks, and console inspection were verified; expected negative requests are documented. No exhaustive network-trace claim is made.
- HTTPS/proxy deployment behavior, distributed rate limiting, operations, scaling, backups, and infrastructure resilience remain future-stage work. The accepted historical credential issue was not revisited and Git history was not rewritten.

14. The four Stage 5 defects

Evidence: docs/stage-5-testing.md; changes in 8dea970

Each regression was observed failing before its fix and passing afterward. These are narrowly scoped behavioral fixes, with no schema or dependency change.

1. The example session secret was accepted

The validator rejected a differently spelled placeholder but allowed the actual example value. A user could therefore believe a copied example was valid private configuration. The fix rejects both spellings while preserving the minimum length requirement. The regression reads the actual example file and asserts rejection, preventing example/configuration drift.

Files: backend/config/session.js and backend/test/environment.test.js.

2. A canceled response could expire a valid UI session

A request could be canceled while its response body was being read. If the delayed result was a 401, the service still dispatched session-expired because cancellation was only handled later in the catch path. The fix checks the combined abort signal after body parsing and before authentication side effects.

The regression verifies that a canceled response emits no session-expiry event while ordinary 401 behavior remains supported. This distinguishes expected React development cancellation from a real session-state bug.

Files: frontend/src/services/api.js and frontend/test/api.test.js.

3. A failed final login check left a saved session

Login saved a regenerated session before a final password/active-state database check. If that query failed, the response returned 500 but the saved session survived. The fix tracks whether regeneration occurred and destroys the new session on the error path before forwarding the failure.

The regression verifies the 500 response, absence of a new authenticated session, and rejection of a later protected request. An already existing independent session is not accidentally counted as the failed login's session.

Files: backend/routes/auth.js and backend/test/profile.test.js.

4. An occupied port reported successful startup

Express passed a listen error to the callback, but the callback ignored it and printed success. The fix checks the error, logs its code, sets exit status 1, and suppresses the success message.

The regression starts a child process against an occupied port and expects EADDRINUSE, exit 1, and no running-server message. Honest startup signals matter because later operational tooling will need to distinguish healthy processes from failed launches.

Files: backend/server.js and backend/test/startup.test.js.

Additional failure coverage

An isolated refused database connection test checks health 503, login 500, no login cookie, safe responses/logs, and continued unauthenticated service. It targets an unused local port without stopping the real PostgreSQL server.

15. Final verification & capability matrix

Evidence: historical Stage 5 report; current Git checkpoint

Evidence	Recorded Stage 5 result	Interpretation
Backend automated tests	56 passed; none failed/skipped	Repeatable route/config/security regressions, mostly using doubles
Frontend automated tests	20 passed; none failed/skipped	Service and helper coverage; not full browser automation
Production frontend build	Passed, 32 modules transformed	Build artifact generated successfully
Live API verification	89 checked requests passed, plus cleanup	Real backend/database checks, separate from unit tests
Dependency audits	Backend and frontend reported zero known vulnerabilities	Audit-time result only
Data preservation	Three users and two tasks remained unchanged	Compared legitimate rows, not just counts
Browser review	Desktop, tablet and mobile workflows passed	In-app Chromium evidence, not cross-browser certification

Current application capabilities

Admins can manage users, active status, and tasks with safeguards; inspect dashboards; assign, unassign, and reassign work; and search/filter/sort their lists. Users can view only their assigned workload, update task status, inspect details, edit their name/email, and change their password with session revocation.

Authentication uses hashed passwords and PostgreSQL-backed sessions. Server checks enforce roles and task ownership. The UI provides loading/retry states, preserved failed form input, confirmation before deletion, and persisted data after refresh.

What these results do not claim

The completed work is not a penetration test, a load test, a cross-browser certification, or a production deployment review. Historical zero-vulnerability and secret-scan results are not re-certified by generating this PDF. Distributed rate limiting, HTTPS reverse-proxy behavior, backups, and operational resilience remain future work.

16. Troubleshooting & lessons learned

Evidence: database, account, experience and Stage 5 guides

Symptom	Likely check	Practical response
Styles absent in early frontend	Stylesheet name/path	The early stylesc.ss typo was fixed to styles.css; inspect requested assets
Database authentication rejected	PostgreSQL 28P01; environment overrides	Verify host/role and private .env values; quote a password containing #
Health succeeds but application query fails	Missing table/column or old schema	Health checks connectivity; inspect migration state separately
Backend cannot start	EADDRINUSE	Identify the intended process/port; do not assume startup succeeded
Browser cannot reach API	Backend process or proxy target	Start the backend, confirm target port, restart Vite after configuration changes
Request timed out	Response may be lost after a write	Reload and inspect saved state before submitting again
User cannot be deleted	Assigned tasks, including completed work	Reassign or deliberately delete the tasks first
Legacy account cannot log in	Null password hash or inactive state	Enable legacy login deliberately; active status is a separate setting
Old session fails after password change	Intended revocation	Sign in again with the new password
PowerShell npm launcher fails	Shell launcher issue	Use the installed npm CLI through Node, as recorded in Stage 5

Lessons that transfer to future projects

A successful UI click is not proof of persistence; verify after reload and independently in the database. Hiding a button is not authorization; test the endpoint directly as another role. Tests with query doubles are valuable but need live integration evidence. A failed operation must clean up security state, not merely return an error.

Treat business-rule races as part of correctness: account updates and assignments coordinate through short locks, password changes revoke sessions atomically, and login rechecks credentials before leaving a usable session. These choices favor understandable correctness for the current small application.

Keep tests representative of reality. During Stage 4.6B a mock returned a freshly generated password hash on each read; correcting it to a stable stored hash fixed the fixture, rather than weakening the production login check.

Build verification must also preserve user data. Create uniquely labeled disposable records, remove only those records, and compare original rows. Avoid database and sequence resets as a testing convenience.

17. Current state & Stage 6 boundary

Evidence: README.md; docs/stage-5-testing.md; main at 8dea970

The source checkpoint is a complete local full-stack Work Tracker through Stage 5. The application has a clear browser/API/database boundary, a health endpoint, environment-based configuration, repeatable tests, and a build command. These provide concrete inputs for the next learning stage.

Stage 6 has not been started by this documentation task. The inspected records place deployment and operational concerns in future-stage work but do not define a final Stage 6 platform or implementation plan. No cloud account, hosting provider, container setup, pipeline, DNS change, or infrastructure deployment is assumed here.

Suggested entry criteria for the next stage

- Preserve 8dea970 as the Stage 5 reference and make future changes reviewable.
- Agree on the deployment target and the learning goal before selecting tools.
- Plan frontend hosting and /api routing together, including HTTPS and cookie behavior.
- Separate development, testing, and deployed configuration; store real secrets privately.
- Define database persistence, migration, backup, restore, logging, and health-check expectations.
- Carry forward the existing tests and add checks for the actual deployed environment.

Study exercises

Trace a task-status change from the component to the API and SQL ownership condition. Explain why the same User gets 404 for another task's details but 403 for its status update. Describe why a password change revokes all sessions and why reactivation does not revive old cookies. Explain the difference between a successful build, a successful health check, and production readiness.

Small glossary

REST API: an HTTP interface for resources and actions. RBAC: permissions based on account roles. Ownership: permission tied to the assigned account. Session: server-side login state referenced by a cookie. Migration: a controlled schema upgrade. Regression test: a check that a fixed defect does not return. Transaction: changes committed or rolled back together. Proxy: a server that forwards requests to another server.

18. API reference & request examples

Evidence: docs/api.md at 8dea970

The Express API uses JSON. The default local base URL is `http://localhost:3000`. The backend persists users and tasks in PostgreSQL. The React frontend calls these endpoints through its API service and Vite's local proxy. See database setup for connection configuration and the README for local startup.

Authentication and access

Use POST `/api/auth/login` with an email/password JSON object and preserve the returned `HttpOnly` cookie. GET `/api/auth/me` returns `{ "user": null }` when signed out or a safe account when signed in. POST `/api/auth/logout` destroys the session. All writes, including login/logout, require the header `X-Requested-With: WorkTracker`. See Accounts setup for bootstrap, cookies, passwords, and migration.

All users endpoints and task creation/full editing/deletion require Admin access. GET tasks returns all tasks for Admins and only assigned tasks for Users; query parameters cannot override this. PATCH task status permits Admins or the assigned User. Missing/unowned task IDs return 403 for Users, while a missing task returns 404 for Admins. Unauthenticated protected requests return 401; forbidden actions return 403. Login throttling returns 429. All account responses exclude password hashes.

Endpoints

Method	Path	Success	Purpose
GET	/api/health	200	Check database connectivity
GET	/api/auth/me	200	Current safe account, or null
PUT	/api/profile	200	Edit own name/email
PUT	/api/profile/password	200	Change own password and revoke sessions
GET	/api/users	200	Return users ordered by ID
POST	/api/users	201	Create an active user
GET	/api/users/:id	200	User details and task summary (Admin)
PUT	/api/users/:id	200	Edit name, email, role (Admin)
PATCH	/api/users/:id/status	200	Set active boolean (Admin)
DELETE	/api/users/:id	200	Safe user deletion (Admin)
GET	/api/tasks	200	Return tasks ordered by ID
POST	/api/tasks	201	Create an optionally assigned task
GET	/api/tasks/:id	200	Task details (Admin or owner; otherwise 404)
PUT	/api/tasks/:id	200	Replace editable task fields (Admin)
DELETE	/api/tasks/:id	200	Delete task (Admin)
PATCH	/api/tasks/:id/status	200	Change task status

POST, PUT, and PATCH require Content-Type: application/json and a JSON object. The maximum request body size is 100 KB. Unknown fields are ignored except password, password_hash, and active fields in Admin user edits, which are rejected. Self-profile endpoints reject every field outside their allowlist, and User task-status requests reject every field except status. Task creation accepts a valid status and defaults to Open.

Create a user

```
{ "name": "Alice Example", "email": "alice@example.com", "password": "<private initial password>", "role": "user" }
```

Name must be a nonblank string of up to 100 characters after trimming. Email must have a basic name@domain.suffix shape and be at most 255 characters. Email is trimmed and lowercased before saving. A database uniqueness conflict returns 409. This does not verify mailbox ownership or deliverability.

Account creation also requires a password (at least 12 characters, at most 72 UTF-8 bytes). Role is admin or user, defaulting to user. The angle-bracketed password above is a placeholder, not a suggested credential.

User records have `id`, `name`, `email`, `role`, `active`, `canLogin`, `createdAt`, and `updatedAt`. Both listing and creation use camelCase JSON timestamps (previously the user routes returned `created_at`).

Create a task

```
{
  "title": "Prepare documentation",
  "description": "Document the local setup steps.",
  "priority": "High",
  "userId": 1
}
```

Title must be a nonblank string of up to 200 characters after trimming. Description is an optional string of at most 10000 characters, defaulting to empty; explicit null is rejected. Priority is Low, Medium, or High, defaulting to Medium. `userId` is an existing user's integer ID, or null/omitted for an unassigned task. String IDs are rejected. IDs must be between 1 and 2147483647.

Task records have `id`, `title`, `description`, `status`, `priority`, `userId`, `createdAt`, and `updatedAt`. Creating a task returns the saved record.

Change status

Send this body to `/api/tasks/1/status`:

```
{ "status": "In Progress" }
```

Supported values are `Open`, `In Progress`, and `Completed`. Any transition between these values is allowed. The update sets `updatedAt` and returns the saved task. A valid but nonexistent task ID returns 404.

Errors and health

Errors use `{ "error": "Human-readable message" }`. Health failures also include `"status": "error"`. SQL details and stack traces are not sent to clients.

Status	Meaning
400	Invalid JSON, body, field, ID, or nonexistent assigned user
404	Unknown route or record not found
409	Duplicate email, blocked deletion/Admin change, or concurrent management lock timeout
413	Request body too large
415	Unsupported content type, charset, or encoding
500	Unexpected server/database error
503	Health check cannot reach the database

Successful health responses contain `status`, `message`, `database`, and `time`. This endpoint checks database connectivity, not the presence of application tables.

Local commands and verification

From backend/, run `npm start` with the existing local `.env` configured. Do not put real passwords in documentation or source code.

Run `npm test` (or `node --test`) from backend/. Tests use Node's built-in test runner, an ephemeral local HTTP port, and a mocked query method. They do not load `.env`, connect to PostgreSQL, or prove that the live schema matches the queries. Sessions use an isolated in-memory test store; authentication and ownership bypass cases run alongside the original API tests.

For manual Postman checks after database configuration is resolved, use the sample bodies above, verify successful creates appear in GET results, and verify status updates persist. Repeat with invalid fields, duplicate emails, and missing IDs. Creation and update requests write to the configured database; use deliberate test data.

Admin edits and deletion

`PUT /api/users/:id` requires `{ "name": "Alice Example", "email": "alice@example.com", "role": "user" }`. `PATCH /api/users/:id/status` requires `{ "active": false }` or `{ "active": true }`. GET user details adds `taskSummary: { total, open, inProgress, completed }`. Passwords are never editable through PUT. Status changes and edits update `updatedAt`.

`PUT /api/tasks/:id` uses the same editable fields as creation, including status. It is a replacement of editable fields: omitted description/priority/status/userId use the creation defaults, so callers should send the complete form. Assignment must exist and be active, except that an existing inactive assignment may be retained. Both DELETE routes return a 200 JSON `{ "message": "... deleted" }` response.

User deletion with assigned tasks and self deletion/deactivation/demotion return 409. The last usable Admin is protected. Deactivation and role changes revoke sessions. Full rules and manual checks are in Admin management.

Self-service profile and password

`PUT /api/profile`: `{ "name": "My Name", "email": "me@example.com" }`. Only these two keys are accepted; the session supplies the account ID. Returns the saved safe User record. Duplicate emails return 409, validation errors 400. GET `/api/auth/me` and login now include `active` and `updatedAt` in safe account data.

`PUT /api/profile/password` accepts `currentPassword`, `newPassword`, `confirmPassword`. The current password must match, confirmation must match, and the replacement must meet the existing 12-character/72-UTF-8-byte policy and differ from the old password. Success returns `{ "message": "Your password has been changed. Please sign in again." }`. All sessions for that account are revoked, including the current one. Wrong current password/invalid fields return 400; expired/inactive authentication returns 401; more than 10 attempts per account in 15 minutes returns 429. No credentials/hashes are included in responses. See User experience for full behavior.

Clarifications for interpreting the reference

POST `/api/auth/login` and POST `/api/auth/logout` are also supported. Login uses email/password JSON; both require the write-verification header. Preserve cookies between requests.

The general missing-status-target 404 applies to Admins. Users receive 403 for missing/unowned status targets and 404 for unowned task details.

Example IDs are illustrative; choose an existing active assignee or null. Password placeholders are not credentials.

Source register

Evidence: repository-local evidence

Source	What it supports
README.md	Purpose, stack, startup, proxy behavior and project layout
docs/database.md	Environment, initialization, persistence and database troubleshooting
docs/accounts.md	Authentication, migrations, sessions, password policy and RBAC
docs/admin-management.md	Stage 4.6A features, safeguards and historical verification
docs/user-experience.md	Stage 4.6B features, ownership, profile/password behavior and checks
docs/stage-5-testing.md	Historical final results, four fixes, 89-request verification and limitations
docs/api.md	Routes, request bodies, validation and response contracts
backend/database/schema.sql	Tables, columns, indexes, constraints and relationships
backend/server.js; frontend/src/services/api.js	Request flow, startup behavior and frontend transport handling
frontend/vite.config.js; package manifests	Proxy settings, commands and declared dependencies
Local Git history through 8dea970	Chronology, source milestones, current branch and completed checkpoint

Compiled from the local repository on 30 September 2026. The detailed account, Admin, User, Stage 5, and API chapters adapt the project's own guides, with editorial explanations and status clarifications. No external sources are needed to substantiate this repository-specific build history.

The first part records project history; the technical chapters and source appendix that follow document the implementation. The included source snapshot is not a database backup. Keep the repository, lockfiles, migration scripts, and private operational backups separately. No real credentials, session IDs, password hashes, or private environment values are included.

19. Technical edition: scope & navigation

Evidence: immutable Git snapshot 8dea970ba22de0b3c79fe2301a515ab780138815

This edition extends the build manual with the technical details needed to trace, maintain, and reconstruct the application. The following chapters explain runtime boundaries, React state, middleware ordering, SQL ownership, transaction locks, authentication races, database evolution, and test design. Full source listings follow the explanations.

What is included

The appendix prints **59 files**: every tracked first-party backend/frontend source file, SQL schema and migrations, maintenance scripts, test files, CSS, HTML, package manifests, Vite configuration, example environment files, and .gitignore at commit 8dea970. Listings retain source line numbers. Long lines wrap visually; a dot in the line-number gutter marks a continuation of the previous source line.

The companion source ZIP contains **68 tracked files**, including both package-lock.json files and original project documentation. The same ZIP is embedded as an attachment inside this PDF; a PDF reader with an Attachments panel can extract it. Lockfiles are included in the archive rather than printed because they are generated dependency resolution records. No node_modules, dist, .git history, private .env, real database records, or runtime sessions are included.

Exactness and safe examples

All archived file bytes come directly from the named Git commit. SNAPSHOT-MANIFEST.json provides the full commit ID and SHA-256 digest for each file. The ZIP is a source distribution, not an installed environment: npm ci and private database configuration are still required.

Test-only passwords, dummy email addresses, and example placeholders remain in the source because they are part of the executable fixtures and validation tests. They are explicitly synthetic, not real account credentials. Unsupported display glyphs in printed code are shown as Unicode escapes; the embedded source remains byte-for-byte unchanged.

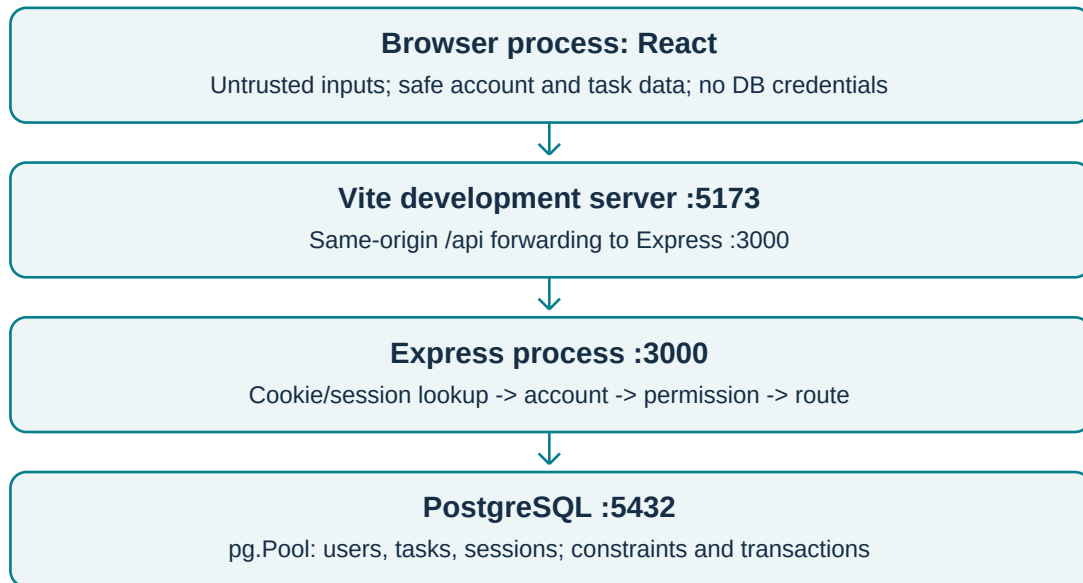
Reading routes through the code

For login, start at Login.jsx, then App.signIn, services/api.js, server.js, middleware/auth.js, routes/auth.js, and config/session.js. For a task status change, start at Tasks.changeStatus, App.changeTaskStatus, updateTaskStatus, then the PATCH handler in routes/tasks.js. For password changes, follow Profile.changePassword through changePassword to routes/profile.js and managementTransaction.

The original test counts remain historical Stage 5 evidence. This edition documents the implementation; it does not rerun the live data-mutating verifier or assert that previously tested deployments exist.

20. Runtime architecture & trust boundaries

Evidence: server.js; vite.config.js; config/database.js; middleware/auth.js



The boundary between UI state and authority

The browser can be modified by its user. React role-based rendering is a usability feature; Express decides whether an operation is allowed. An account ID supplied in a JSON body is not the identity of the caller. `loadAccount` derives identity from `req.session.userId` and rereads a usable active account from PostgreSQL.

There is no separate service process, ORM, Redis cache, message queue, or client-side persistence layer. Express routes call `pg` directly. Helpers centralize connection pooling, safe selected columns, validation, password hashing, and management transactions. Understanding this direct structure is necessary before adding abstraction or infrastructure.

Runtime configuration and process lifetime

Backend startup loads `.env` using Node's `--env-file` flag in the npm script. `DB_*` values configure one shared Pool. `max` is 10; `connectionTimeoutMillis` is 5000; `statement_timeout` is 10000; `idleTimeoutMillis` is 30000. Idle connection failures log a code and do not become unhandled pool errors.

`createSessionMiddleware` runs during application construction and validates `SESSION_SECRET`. The normal entry point validates the database environment and starts listening. Importing `server.js` exports the app for tests without calling `listen`. Session configuration still needs valid test environment values during imports.

PORT defaults to 3000. A listen callback error records only its code, sets `process.exitCode` to 1, and does not print a success message. Health executes `SELECT NOW()`; it tests database reachability, not the full application schema.

Same-origin writes and cookie security

Every non-GET/HEAD/OPTIONS API operation requires X-Requested-With: WorkTracker and rejects Sec-Fetch-Site: cross-site. No permissive CORS layer is configured. Cookies add HttpOnly and SameSite=Lax, with Secure enabled when NODE_ENV is production. These controls assume the documented same-origin arrangement; this source snapshot does not configure a production reverse proxy or trust proxy setting.

The browser receives a signed opaque session ID, not a JSON Web Token. The server-side session contains userId plus cookie metadata. It does not cache the role as the authority. A role or active-status change therefore affects subsequent account lookups, and relevant mutations delete stored sessions.

21. React component tree & state transitions

Evidence: frontend/src/App.jsx; pages; components; services

```
main.jsx -> StrictMode -> App
  header + role-aware navigation
  DataState(checkingSession, sessionError)
    signed out -> Login
    signed in -> DataState(isLoading, loadError)
      Admin -> Dashboard | Users | AdminTasks
      User -> Dashboard | Tasks | Profile
  shared forms: UserForm, TaskForm
  display helpers: StatusBadge, PriorityBadge, ConfirmDelete
```

State ownership

App owns account, users, tasks, currentPage, session restoration state, workload loading state, and logout feedback. Individual pages own transient filters, selected details, pending-operation flags, and messages. Forms own their input values. Page navigation unmounts those page-level values; saved records survive because they are loaded from the API.

accountId is a ref updated from the active account. Each mutation captures its initiating account ID before awaiting the HTTP call. A late response is applied only if the ref still matches. This guards against rendering one account's late mutation response after logout or a switch to another account.

Session and workload effects

The first effect calls getSession with an AbortController. A successful /auth/me result is either an account or null; an unavailable backend is an error with a retry path, not proof that the user is signed out. The cleanup aborts old work.

After authentication, another effect loads users and tasks with Promise.all. Admins request /users; ordinary Users use [account] locally and never request the Admin list. Express already scopes the tasks. The effect depends on account ID, role, and loadAttempt rather than the whole account object. A successful name/email edit can update the header without remounting the form and discarding its success message.

The session-expired event clears the account and workload. password-changed performs the same clearing and adds a re-login message. Login resets old arrays and selects the dashboard before setting the returned account. Logout clears the UI only after its API call succeeds; a failed logout leaves readable feedback.

Mutation contract

Forms await callbacks; App awaits API responses; only then does it append, replace, or remove a record using functional state updates. There is no optimistic database write simulation. Task status returns the saved record so a currently open details panel can also update. React renders text content normally, rather than inserting user content as raw HTML.

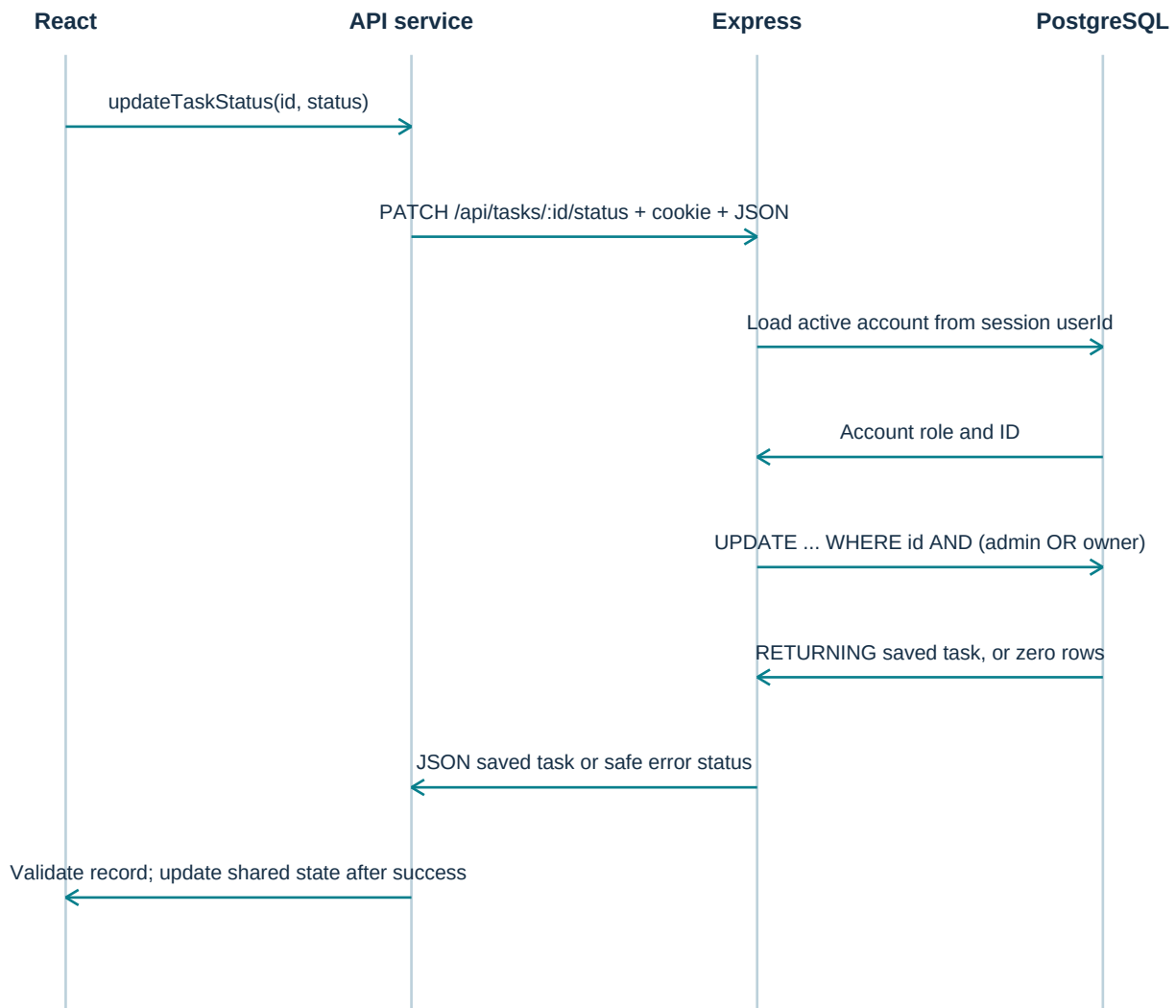
Lists and derived values

management.js filters Admin users by name/email, role, active state and sort; Admin tasks by title, status, priority, assignee and sort. personal.js searches title/description and orders by time, priority, or status. Each filters into a new array before sorting, preserving shared state order. Dashboard totals use the full authorized task array, not the filtered visible subset.

No pagination, subscription, polling, or real-time transport exists. Changes made in another session are reflected on a later load/refresh. This is an explicit property of the current implementation, not a synchronization guarantee.

22. HTTP pipeline & task workflow

Evidence: api.js; server.js; tasks.js



Exact middleware order

1. `express.json({ limit: '100kb' })` parses JSON; parser failures go to the final error handler.
2. `GET /api/health` executes outside session/account middleware.
3. `/api` responses receive `Cache-Control: no-store`.
4. Session middleware reads the cookie and loads server-side state.
5. `protectWrites` verifies state-changing requests; `loadAccount` rereads an active login-enabled account.
6. `/auth` routes handle session discovery/login/logout. `/profile` requires an account. `/users` requires an account and Admin. `/tasks` requires an account and performs per-operation role/ownership checks.
7. Route-level `jsonBody` requires `application/json` and a non-array object where used. Unknown routes return 404; errors finish in `middleware/errors.js`.

SQL authorization is part of the write

The task status query includes `WHERE id = $2 AND ($3::boolean OR user_id = $4)`. The boolean is derived from `req.account.role`, and the owner ID from `req.account.id`. Neither comes from a request query parameter. The same atomic `UPDATE` both checks assignment and writes status, avoiding a separate read-then-write ownership gap.

For a User, extra payload fields are rejected even if status is valid. Zero matching rows return 403 without revealing whether the target exists. Admins receive 404 for a nonexistent target. Task detail instead uses an ownership-filtered `SELECT` and returns 404 when no authorized row is found.

Frontend transport behavior

request composes a caller abort signal with a 15-second timeout. It sends credentials: same-origin, `Accept: application/json`, and the write-verification header. JSON helpers add `Content-Type` for body requests. Responses are parsed only when their content type indicates JSON.

After body parsing, `signal.throwIfAborted` runs before any session-expired event. This ordering is the Stage 5 cancellation fix. A legitimate protected 401 dispatches the event; login 401 remains a login error. Server 5xx responses are converted to a generic service-unavailable message. Network errors, malformed JSON, incompatible record shapes, and timeouts have separate readable feedback.

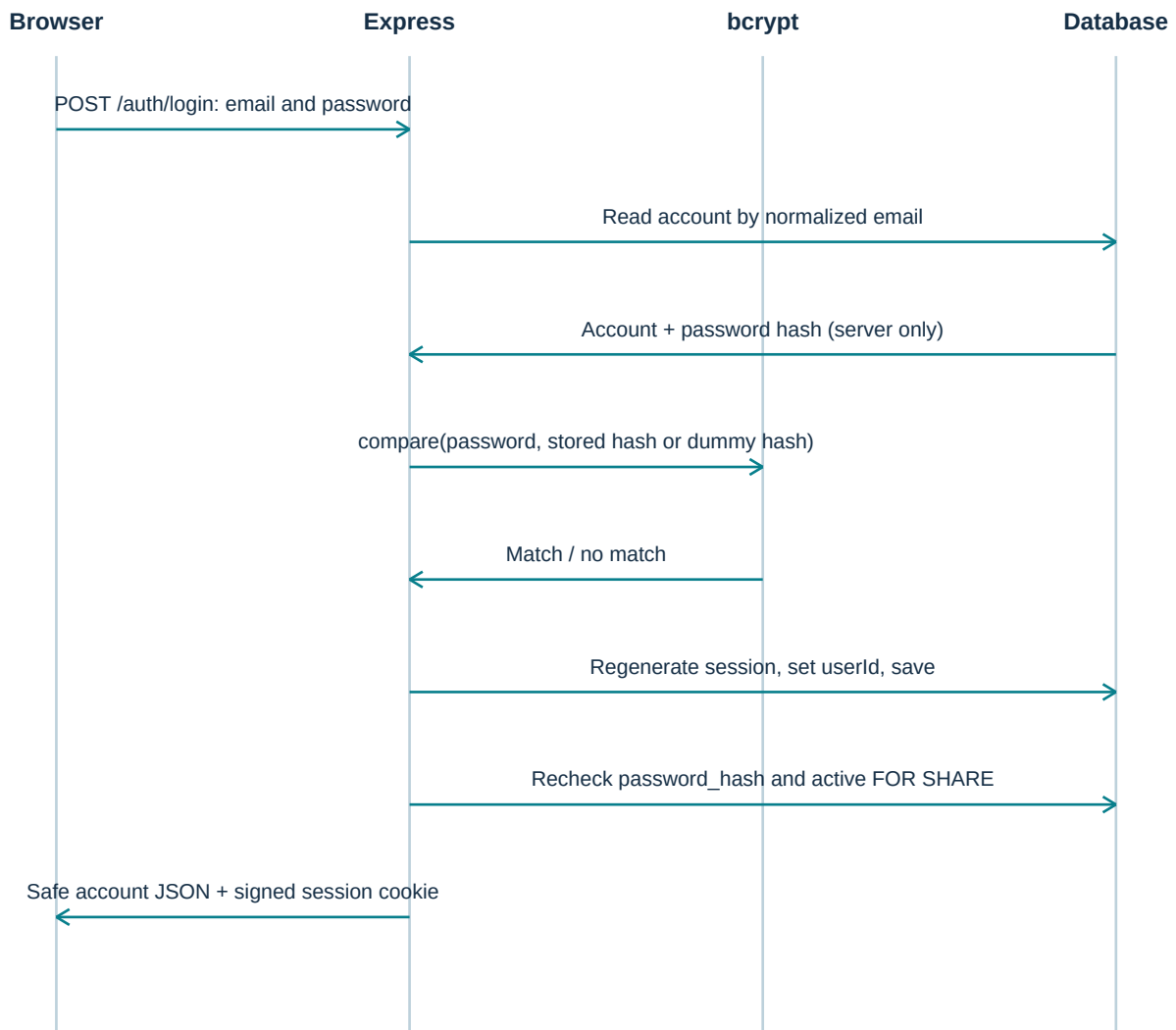
Task creation and full editing

`saveTask` validates and normalizes fields, then uses `managementTransaction`. It rechecks the acting Admin under the `users-table` lock. An edit locks the existing task with `FOR UPDATE`; an assignment requires a real active user, except the existing inactive assignment may be retained. `INSERT/UPDATE` returns `taskFields`, including camelCase aliases for `userId` and timestamps.

`PUT` uses defaults for omitted editable fields. It is not a partial patch: omitted assignment becomes null, for example. The UI sends a complete form. The dedicated `PATCH` endpoint is the narrow status operation.

23. Authentication & revocation sequences

Evidence: routes/auth.js; routes/profile.js; config/session.js



Login branches and failure cleanup

Malformed email/password fields return the same generic 401 as a bad credential. `bcrypt.compare` also runs with a random dummy hash for unknown or legacy accounts, reducing an obvious timing difference; this is not a claim of perfect timing indistinguishability. Login is limited to 20 attempts per IP per 15 minutes by an in-process limiter.

After password verification, `regenerate` replaces any old session ID to prevent fixation. `userId` is then saved. The final database check detects changed credentials or deactivation; FOR SHARE coordinates with concurrent password updates. If the check fails or its query errors, the regenerated session is destroyed rather than left valid. The Stage 5 failure regression covers that error path.

`GET /auth/me` returns safe account data or `user:null`. Logout deletes server-side state and clears `worktracker.sid` with the configured cookie attributes. Replaying a copied old cookie cannot revive a deleted session.

Password-change transaction

The profile password endpoint requires authentication and limits attempts to 10 per account per 15 minutes. It allows exactly `currentPassword`, `newPassword`, and `confirmPassword`. The new password must differ, confirm correctly, be at least 12 JavaScript string units, and remain within bcrypt's 72 UTF-8 byte limit. The UI and normal project wording call this a 12-character minimum; the source uses `.length`, which is UTF-16 code-unit length for non-ASCII strings.

Within `managementTransaction`, the handler rereads the active account hash FOR UPDATE, checks the current password, hashes the new value with cost 12, updates the hash/timestamp, and deletes all matching session rows. The database commit couples credential replacement and stored-session revocation. Then it destroys the request session and clears the cookie; the frontend emits `password-changed` only after success.

Race cases to understand

If a password change commits before a racing old-password login saves, the final login check sees the different hash and deletes the new session. If login completes first, the later password transaction deletes its session. If the final login query itself errors after session save, Stage 5 cleanup destroys that session before forwarding the failure.

After the password transaction commits, a later failure destroying the request session could still return an error although the new password is already stored. Database commit and HTTP delivery are different events. This is why response failure cannot universally imply that no state changed.

Storage and authorization limits

The default session lifetime is eight hours. Production uses PostgreSQL session storage; `NODE_ENV=test` switches to `MemoryStore` for isolated automated checks. This is not a production memory-store fallback. Rate limits remain process-local and reset on restart; there is no shared distributed store or deployed HTTPS proof in this checkpoint.

24. Database design, locks & migrations

Evidence: database/*.sql; config/management.js; routes/users.js

users	tasks
id SERIAL PK <-----	user_id INTEGER NULL FK
name VARCHAR(100)	id SERIAL PK
email VARCHAR(255) UNIQUE	title VARCHAR(200)
password_hash TEXT NULL	description TEXT
role admin user	status Open In Progress Completed
active BOOLEAN	priority Low Medium High
created_at / updated_at	created_at / updated_at

sessions

sid VARCHAR PK

sess JSON -> userId (logical association; no declared FK)

expire TIMESTAMP(6) + expiry index

Mapping SQL to JSON

The backend selects explicit fields instead of `SELECT *`. `userFields` computes `canLogin` from `password_hash IS NOT NULL` without exposing the hash. `taskFields` aliases `user_id` to `userId` and timestamps to `createdAt/updatedAt`. A safe API response contract should not change merely because a private database column is added.

The schema keeps both the original email `UNIQUE` constraint and a unique `lower(email)` index. API inputs normalize email to lowercase, while the index protects case-insensitive uniqueness at the database boundary. Task status/priority `CHECK` constraints reject invalid stored values even outside React.

Management transaction boundaries

`managementTransaction` checks out one pool client, begins, sets a five-second local lock timeout, and locks users `IN SHARE ROW EXCLUSIVE MODE`. All callback queries run on that same client. Success commits; failure rolls back; finally releases the client. Mixing `pool.query` inside that callback would use another connection and lose the intended transaction boundary, so the mutation helpers use `client.query`.

The shared helper serializes participating account safeguards and task assignment operations. It is a coarse lock that favors reasoning about correctness over high write throughput. It does not mean every API request uses a transaction: reads, user creation, profile name/email edits, task deletion, and the atomic status `UPDATE` have their own simpler paths in the source.

`changeUser` rechecks the acting Admin, reads the target, applies `self/last-usable-Admin/legacy-promotion` guards, checks assigned tasks for deletion, performs the mutation, and deletes sessions where access was revoked. A lock timeout code `55P03` becomes a safe `409` asking the user to retry. It is not retried automatically.

Migration ordering

`schema.sql` describes the final empty-database shape. `accounts.sql` adds `password_hash` and `role`, ensures the `role` constraint and `lower(email)` index, and creates sessions. `admin-management.sql` adds `active` and `updated_at`. The migration runners wrap SQL in transactions and bound lock waits. They preserve IDs, tasks, assignments, and sequences.

`IF NOT EXISTS` avoids recreating existing objects; it does not validate arbitrary preexisting definitions. A legacy duplicate email can block the new unique index and roll back the migration. Resolving that data conflict is a separate deliberate decision, not an automatic merge or delete.

Bootstrap and legacy activation

bootstrap-admin validates private inputs, hashes the password, locks users, and creates an Admin only when no active login-enabled Admin exists. It never silently overwrites an existing account. activate-legacy-user is a narrow tool for an existing ordinary account with no password; it is not a password-reset or role-escalation interface. Enabling a password and setting active are separate account properties.

Timestamp and deletion behavior

created_at and updated_at use TIMESTAMP without time zone; mutation SQL explicitly refreshes updated_at. No automatic update trigger exists. The tasks FK uses ON DELETE SET NULL for external SQL deletion, while Admin API deletion rejects assigned users. Those two policies operate at different boundaries and should not be confused during maintenance.

25. Validation, errors & security contracts

Evidence: middleware; routes; password and frontend helpers

Input or boundary	Source-level contract	Observable outcome
JSON body	application/json and object, not array	415 unsupported type; 400 invalid body
Payload size	express.json limit 100kb	413 before route mutation
User name	Nonblank trimmed string; up to 100 code points in shared validation	400 on invalid input
Email	Basic non-whitespace address shape, length <=255, trim/lowercase	400 malformed; 409 duplicate
Task title/description	Title 1-200 code points; description <=10000, explicit null rejected	400 invalid; stored strings trimmed
Task assignee	Integer or null; positive, <=2147483647; existing account	400 invalid/missing/inactive assignment
Profile fields	Exactly name and email	400 rejects role, active, ID and password fields
User status patch	Only status key for regular User	403 on extra fields or unowned target
Admin edit	Explicit name/email/role validation; password/hash/active rejected	Dedicated status action required

Errors are contracts, not debug output

fail throws an Error with publicStatus for intentional business-rule responses. The final handler respects that public status, maps uniqueness 23505 to 409, lock timeout 55P03 to 409, parser failures to 400, large entities to 413, and encoding errors to 415. Unexpected errors log method/path and a code, then return generic 500 JSON.

Request-verification rejection occurs before route access checks, so an unauthenticated malformed write without the required header may return 403 rather than 401. The authorization matrix assumes a correctly formed request that reaches the account/role check.

What is and is not protected by each layer

React escaping prevents ordinary rendered strings from becoming HTML markup. Parameterized SQL prevents those values from becoming query syntax. Neither replaces field validation or authorization. Custom-header verification and cookie settings address cross-site writes under the documented origin setup; they do not prevent a legitimate authenticated caller from directly exercising the API, which is why ownership and role rules remain server-side.

Sessions and hashes are not returned in record JSON. A signed session cookie is intentionally returned through HTTP cookie handling. Test assertions therefore distinguish forbidden secret leakage in bodies/logs from the required authentication cookie.

Unknown-field handling is endpoint-specific: profile and User status requests reject extras; some other routes ignore them. Check the route before extending a payload.

26. Test architecture & engineering workflow

Evidence: backend/test; frontend/test; verify-stage5.cjs

Backend automated tests

api.test.js exercises response contracts, field validation, IDs and failure mapping. auth.test.js exercises session lifecycle, roles, ownership, request verification and rate limits. management.test.js covers account/task management and safeguards. profile.test.js covers profile allowlists, password changes, revocation and the failed-final-login regression.

environment.test.js validates configuration, including the actual example-secret placeholder. startup.test.js uses a child process and an occupied port to test honest startup failure. database-outage.test.js points an isolated child process at a refused connection and checks safe health/login failures without stopping the real database.

Most route tests replace pool methods and use MemoryStore. Some fixtures use lower-cost bcrypt hashing for test speed; production hashPassword uses cost 12. SQL-string-aware mocks test route orchestration but cannot prove PostgreSQL parsing, real lock behavior, indexes, or durable session deletion. Those need the separate live checks.

Frontend automated tests

api.test.js mocks fetch and verifies URL/method/header behavior, record validation, error mapping, legitimate 401 events, and canceled-body behavior. management.test.js checks Admin filters and sorting. personal.test.js checks workload ordering/summary, password validation, and success-only password-change events. These are service/helper tests, not mounted React interaction tests.

Live verification workflow

verify-stage5.cjs is deliberately outside node --test. Against a running local backend, it reads the configured private bootstrap login, generates disposable credentials, creates temporary records, checks API outcomes and PostgreSQL state, compares original-row digests, and cleans up its records and sessions. The earlier 89 checked requests are recorded evidence, not an execution count created by reading the script.

Run it only in a coordinated local verification session without concurrent legitimate edits. An interruption may require inspection and cleanup of precisely its own records. The source listing is provided for study; this documentation run does not execute it.

Rebuild from the archive

1. Extract the ZIP to a new directory; preserve its frontend/backend layout.
2. Install the project-compatible Node/PostgreSQL versions and use npm ci separately in each package. The original lockfiles are in the archive.
3. Create a new database only when needed. Copy the example environment file only when a private .env does not exist; replace placeholders privately.
4. For a fresh database run db:init. For an older Stage 4 database use the account then management migrations. Bootstrap the first Admin and run db:check.
5. Start Express and Vite in separate terminals, open 5173, and verify the complete browser/API/database flow.

6. Run package tests and the frontend build. Schedule live and browser checks separately with disposable records.

Change-review workflow

Begin from a recorded source baseline and inspect Git status. Make a focused branch, implement the smallest coherent change, add a regression when correcting behavior, and run the relevant tests. Compare the API contract, schema compatibility, session behavior and UI states before committing. A passing unit test does not remove the need for persistence checks when SQL changes.

Use the full listings below to locate the exact implementation, rather than reconstructing modules from abbreviated snippets. Stage 6 remains outside this snapshot and requires its own infrastructure and operational verification.

27. Complete source-code appendix

Evidence: all listings are from commit 8dea970

There are 59 numbered listings. Each file starts with its path, source size and content fingerprint. The PDF contents and bookmarks link to individual files. Code is selectable text, not a screenshot. Numbered lines correspond to the original source; continuation rows have a dot instead of a new line number.

Copying line-numbered PDF text is convenient for reading but may add line numbers or visual wraps. For execution, use the companion ZIP or the embedded ZIP attachment, which preserves all original bytes and includes lockfiles and source documentation.

The technical chapters describe the application as implemented. The listings are not an idealized rewrite: existing comments, tests, validation choices and limitations remain visible. Source references use repository-relative paths to describe locations inside the extracted archive.

backend/config/database.js

Backend configuration and server | 23 source lines | 799 bytes

SHA-256: 707e2be52f0632b5d5606d44165e50d5e1ce015489cbbac3895ef55e9996a747

```
1  const { Pool } = require('pg')
2
3  // Reuse connections across requests instead of opening a new connection for every query.
4  const pool = new Pool({
5    host: process.env.DB_HOST,
6    port: Number(process.env.DB_PORT),
7    database: process.env.DB_NAME,
8    user: process.env.DB_USER,
9    password: process.env.DB_PASSWORD,
10   // Bound connection attempts and queries so an unavailable database cannot hang request
11   .
12   ts.
13     connectionTimeoutMillis: 5000,
14     statement_timeout: 10000,
15     idleTimeoutMillis: 30000,
16     max: 10,
17     application_name: 'work-tracker',
18   })
19
20 // Idle connections can fail outside a request; handle that event without crashing Express
21 .
22 pool.on('error', error => {
23   console.error('PostgreSQL idle connection failed:', error.code || 'DATABASE_ERROR')
24 })
25
26 module.exports = pool
```

backend/config/environment.js

Backend configuration and server | 17 source lines | 809 bytes

SHA-256: 3db092a61ddbafd968eab2902ac7f03e4d7be83d9598aed31b3b746269abeea6

```
1 // Fail with variable names only; configuration errors must never reveal credentials.
2 function validateDatabaseEnvironment() {
3   for (const name of ['DB_HOST', 'DB_PORT', 'DB_NAME', 'DB_USER', 'DB_PASSWORD']) {
4     if (!process.env[name] || !process.env[name].trim()) {
5       throw new Error(`Missing required environment variable: ${name}`)
6     }
7   }
8   const port = Number(process.env.DB_PORT)
9   if (!/^\d+$/.test(process.env.DB_PORT) || !Number.isInteger(port) || port < 1 || port
10  > 65535) {
11     throw new Error('DB_PORT must be an integer between 1 and 65535')
12   }
13   if (process.env.DB_PASSWORD === 'your_local_postgres_password') {
14     throw new Error('Replace the DB_PASSWORD placeholder in your local environment')
15   }
16 }
17 module.exports = validateDatabaseEnvironment
```

backend/config/management.js

Backend configuration and server | 32 source lines | 1,641 bytes

SHA-256: cd27d3d8184eaa5a026e103bb5ec90a9587991ef29a878173668c8212e61c4a1

```

1  const pool = require('./database')
2
3  const userFields = `id, name, email, role, active, (password_hash IS NOT NULL) AS "canLogi
.  n",
4    created_at AS "createdAt", updated_at AS "updatedAt"`
5  const taskFields = `id, title, description, status, priority, user_id AS "userId",
6    created_at AS "createdAt", updated_at AS "updatedAt"`
7
8  function fail(status, message) { throw Object.assign(new Error(message), { publicStatus: s
.  tatus }) }
9  function validId(id) { return /^[1-9]\d*$/.test(String(id)) && Number(id) <= 2147483647 }
10 function userInput({ name, email, role }) {
11   if (typeof name !== 'string' || !name.trim() || [...name.trim()].length > 100) fail(40
.  0, 'Name must contain 1 to 100 characters')
12   if (typeof email !== 'string' || email.trim().length > 255 || !/^[^\s@]+@[^\s@]+\.[^\s
.  @]+$/ .test(email.trim())) fail(400, 'A valid email of up to 255 characters is required')
13   if (!['admin', 'user'].includes(role)) fail(400, 'Role must be admin or user')
14 }
15
16 // Serialize account safeguards and assignment changes so concurrent requests cannot bypas
.  s them.
17 async function managementTransaction(work) {
18   const client = await pool.connect()
19   try {
20     await client.query('BEGIN')
21     await client.query("SET LOCAL lock_timeout = '5s'")
22     await client.query('LOCK TABLE users IN SHARE ROW EXCLUSIVE MODE')
23     const result = await work(client)
24     await client.query('COMMIT')
25     return result
26   } catch (error) {
27     await client.query('ROLLBACK')
28     throw error
29   } finally { client.release() }
30 }
31
32 module.exports = { userFields, taskFields, fail, validId, userInput, managementTransaction
.  }
```

backend/config/passwords.js

Backend configuration and server | 13 source lines | 483 bytes

SHA-256: 117422a78f371a1fea343bd05ee9ada301e99671dd0dc07a93d269b65467f016

```
1  const bcrypt = require('bcryptjs')
2
3  function validPassword(password) {
4      // bcrypt only considers 72 UTF-8 bytes; reject longer input rather than silently truncate it.
5      return typeof password === 'string' && password.length >= 12 && !bcrypt.truncates(password)
6  }
7
8  async function hashPassword(password) {
9      // bcrypt generates a random salt and stores it with the adaptive password hash.
10     return bcrypt.hash(password, 12)
11 }
12
13 module.exports = { validPassword, hashPassword }
```

backend/config/session.js

Backend configuration and server | 26 source lines | 1,137 bytes

SHA-256: ef1435d9da827cd8731fcf6f494459e6191acdb0ced70cf02beb9c95b61e602c

```
1  const session = require('express-session')
2  const PgSession = require('connect-pg-simple')(session)
3  const pool = require('./database')
4
5  function createSessionMiddleware() {
6      const secret = process.env.SESSION_SECRET
7      if (!secret || secret.length < 32 || /^replace_with_a_random_secret_(of_)?at_least_32_
.  characters$/.test(secret)) {
8          throw new Error('SESSION_SECRET must be a private random value of at least 32 char
.  acters')
9      }
10     // Only an opaque signed ID reaches the browser. Session data stays in PostgreSQL.
11     // Automated unit tests use MemoryStore; real application startup always uses PostgreS
.  QL.
12     const store = process.env.NODE_ENV === 'test'
13         ? new session.MemoryStore()
14         : new PgSession({ pool, tableName: 'sessions', createTableIfMissing: false })
15     return session({
16         name: 'worktracker.sid', secret, store,
17         resave: false, saveUninitialized: false,
18         cookie: {
19             httpOnly: true, sameSite: 'lax',
20             secure: process.env.NODE_ENV === 'production',
21             maxAge: 8 * 60 * 60 * 1000,
22         },
23     })
24 }
25
26 module.exports = createSessionMiddleware
```

backend/server.js

Backend configuration and server | 80 source lines | 2,550 bytes

SHA-256: 8f6e0efe956f54eb9e8c988835121d7c1fa5610bd781609723f052268b6a6bfe

```
1  const express = require('express')
2  const pool = require('./config/database')
3  const usersRouter = require('./routes/users')
4  const tasksRouter = require('./routes/tasks')
5  const errorHandler = require('./middleware/errors')
6  const validateDatabaseEnvironment = require('./config/environment')
7
8  const createSessionMiddleware = require('./config/session')
9  const authRouter = require('./routes/auth')
10 const profileRouter = require('./routes/profile')
11 const { loadAccount, requireAccount, requireAdmin, protectWrites } = require('./middleware
. /auth')
12
13 const app = express()
14
15 const PORT = process.env.PORT || 3000
16
17 // Limit request size before parsing user-supplied JSON.
18 app.use(express.json({ limit: '100kb' }))
19
20 // Health remains independent of cookie/session lookup, including during database outages.
21 app.get('/api/health', async (req, res) => {
22   try {
23     const result = await pool.query('SELECT NOW()')
24
25     res.json({
26       status: 'ok',
27       message: 'Backend is running',
28       database: 'connected',
29       time: result.rows[0].now,
30     })
31   } catch (error) {
32     console.error('Health check failed:', error.code || 'DATABASE_ERROR')
33
34     res.status(503).json({
35       status: 'error',
36       error: 'Database connection failed',
37     })
38   }
39 })
40
```

backend/server.js (continued)

8dea970 | Listing segment 2 of 2

```
41 // Session cookies identify accounts; these backend checks are the security boundary.
42 app.use('/api', (req, res, next) => {
43     res.set('Cache-Control', 'no-store')
44     next()
45 })
46 app.use('/api', createSessionMiddleware(), protectWrites, loadAccount)
47 app.use('/api/auth', authRouter)
48 app.use('/api/profile', requireAccount, profileRouter)
49 app.use('/api/users', requireAccount, requireAdmin, usersRouter)
50 app.use('/api/tasks', requireAccount, tasksRouter)
51
52
53 app.use((req, res) => {
54     res.status(404).json({
55         error: 'Route not found',
56     })
57 })
58
59 // Error middleware must follow routes and the fallback handler.
60 app.use(errorHandler)
61
62 // Importing the app in tests does not start the normal development server.
63 if (require.main === module) {
64     try {
65         validateDatabaseEnvironment()
66         app.listen(PORT, error => {
67             if (error) {
68                 console.error('Backend startup failed:', error.code || 'LISTEN_ERROR')
69                 process.exitCode = 1
70                 return
71             }
72             console.log(`Server is running on port ${PORT}`)
73         })
74     } catch (error) {
75         console.error('Backend configuration error:', error.message)
76         process.exitCode = 1
77     }
78 }
79
80 module.exports = app
```

backend/middleware/auth.js

Backend middleware and routes | 38 source lines | 1,642 bytes

SHA-256: 1639c7b18d1f983364e6a07f53b5c12967d2b6f0dbc6f1d9f2d1781f074fb016

```

1  const pool = require('../config/database')
2
3  async function loadAccount(req, res, next) {
4      try {
5          if (req.session.userId) {
6              // Re-read the role each time; a client cannot choose its role or retain revoked access.
7              const result = await pool.query(`
8                  SELECT id, name, email, role, active, created_at AS "createdAt", updated_at AS "updatedAt"
9                  FROM users WHERE id = $1 AND password_hash IS NOT NULL AND active = true
10                 `, [req.session.userId])
11              req.account = result.rows[0]
12              // Revoked accounts must sign in again after reactivation; old sessions stay invalid.
13              if (!req.account) await new Promise((resolve, reject) => req.session.regenerate(error => error ? reject(error) : resolve()))
14              }
15              next()
16          } catch (error) { next(error) }
17      }
18
19      function requireAccount(req, res, next) {
20          if (!req.account) return res.status(401).json({ error: 'Please sign in to continue' })
21          next()
22      }
23
24      function requireAdmin(req, res, next) {
25          if (req.account.role !== 'admin') return res.status(403).json({ error: 'Admin access required' })
26          next()
27      }
28
29      function protectWrites(req, res, next) {
30          // Cross-site forms cannot add this header; cross-origin scripts need CORS, which we do not enable.
31          if (!['GET', 'HEAD', 'OPTIONS'].includes(req.method) &&
32              (req.get('X-Requested-With') !== 'WorkTracker' || req.get('Sec-Fetch-Site') === 'cross-site')) {
33              return res.status(403).json({ error: 'Request verification failed' })
34          }
35          next()
36      }
37
38      module.exports = { loadAccount, requireAccount, requireAdmin, protectWrites }

```

backend/middleware/errors.js

Backend middleware and routes | 21 source lines | 1,186 bytes

SHA-256: ea90478fabae61487350dcd5f79e511979d8f1d2ac4d0aaa4b3451d42642b613

```
1 // Express forwards parser and route failures here so clients always receive JSON.
2 function errorHandler(error, req, res, next) {
3   if (res.headersSent) return next(error)
4   if (error.publicStatus) return res.status(error.publicStatus).json({ error: error.message })
5   if (error.code === '23505') return res.status(409).json({ error: 'Email already exists' })
6   if (error.code === '55P03') return res.status(409).json({ error: 'Another management operation is in progress. Please try again.' })
7   if (error.type === 'entity.parse.failed') {
8     return res.status(400).json({ error: 'Request body must be valid JSON' })
9   }
10  if (error.type === 'entity.too.large') {
11    return res.status(413).json({ error: 'Request body exceeds the 100 KB limit' })
12  }
13  if (error.status === 415) {
14    return res.status(415).json({ error: 'Unsupported request encoding or charset' })
15  }
16  // Log a code rather than SQL details or submitted personal information.
17  console.error('API request failed:', req.method, req.path, error.code || 'INTERNAL_ERROR')
18  res.status(500).json({ error: 'An unexpected server error occurred' })
19 }
20
21 module.exports = errorHandler
```

backend/middleware/jsonBody.js

Backend middleware and routes | 12 source lines | 458 bytes

SHA-256: 76928088d67b3dbfed675377c09e443354839fac94ae7dcc800a085fe19f3b81

```
1 // Reject absent bodies and arrays before handlers try to read object fields.
2 function jsonBody(req, res, next) {
3   if (!req.is('application/json')) {
4     return res.status(415).json({ error: 'Content-Type must be application/json' })
5   }
6   if (!req.body || typeof req.body !== 'object' || Array.isArray(req.body)) {
7     return res.status(400).json({ error: 'Request body must be a JSON object' })
8   }
9   next()
10 }
11
12 module.exports = jsonBody
```

backend/routes/auth.js

Backend middleware and routes | 71 source lines | 3,782 bytes

SHA-256: ee9176022816b23539086193f8d6b6fed5ca0868bb3d45a7a7091167a045c87

```

1  const express = require('express')
2  const bcrypt = require('bcryptjs')
3  const { randomBytes } = require('node:crypto')
4  const { rateLimit } = require('express-rate-limit')
5  const pool = require('../config/database')
6  const jsonBody = require('../middleware/jsonBody')
7
8  const router = express.Router()
9  // Unknown accounts still perform a hash comparison, reducing obvious account-enumeration
10 timing differences.
11 const dummyHash = bcrypt.hashSync(randomBytes(32).toString('hex'), 12)
12 const loginLimiter = rateLimit({
13   windowMs: 15 * 60 * 1000, limit: 20,
14   standardHeaders: 'draft-8', legacyHeaders: false,
15   message: { error: 'Too many sign-in attempts. Please try again in 15 minutes.' },
16 })
17 router.get('/me', (req, res) => res.json({ user: req.account || null }))
18
19 router.post('/login', loginLimiter, jsonBody, async (req, res, next) => {
20   let createdSession = false
21   try {
22     const { email, password } = req.body
23     if (typeof email !== 'string' || email.length > 255 ||
24         typeof password !== 'string' || password.length === 0 || bcrypt.truncates(password)) {
25       return res.status(401).json({ error: 'Email or password is incorrect' })
26     }
27     const result = await pool.query(`
28       SELECT id, name, email, role, active, password_hash, created_at AS "createdAt"
29       , updated_at AS "updatedAt"
30       FROM users WHERE lower(email) = $1
31       `, [email.trim().toLowerCase()])
32     const account = result.rows[0]
33     const matches = await bcrypt.compare(password, account?.password_hash || dummyHash)
34     if (!matches || !account?.password_hash || account.active === false) {
35       return res.status(401).json({ error: 'Email or password is incorrect' })
36     }
37     // Replace any previous session ID after login to prevent session fixation.
38     await new Promise((resolve, reject) => req.session.regenerate(error => error ? reject(error) : resolve()))
39     createdSession = true

```

backend/routes/auth.js (continued)

8dea970 | Listing segment 2 of 2

```
39     req.session.userId = account.id
40     await new Promise((resolve, reject) => req.session.save(error => error ? reject(er
.   ror) : resolve()))
41     // If a password change raced with login, its session deletion may have happened b
.   efore this save.
42     // Recheck afterward: either this detects the change, or a later change deletes th
.   e saved session.
43     // FOR SHARE waits for an in-flight password transaction instead of reading its ol
.   d committed hash.
44     const latest = await pool.query('SELECT password_hash, active FROM users WHERE low
.   er(email) = $1 FOR SHARE', [account.email.toLowerCase()])
45     if (latest.rows[0]?.password_hash !== account.password_hash || latest.rows[0]?.act
.   ive === false) {
46       await new Promise((resolve, reject) => req.session.destroy(error => error ? re
.   ject(error) : resolve()))
47       return res.status(401).json({ error: 'Email or password is incorrect' })
48     }
49     const { id, name, role, active, createdAt, updatedAt } = account
50     res.json({ user: { id, name, email: account.email, role, active, createdAt, update
.   dAt } })
51   } catch (error) {
52     // A saved session must not survive an unsuccessful final database check.
53     if (createdSession && req.session) {
54       return req.session.destroy(() => next(error))
55     }
56     next(error)
57   }
58 })
59
60 router.post('/logout', async (req, res, next) => {
61   // Delete the server session, not just the browser cookie; copied cookies stop working
.   too.
62   req.session.destroy(error => {
63     if (error) return next(error)
64     res.clearCookie('worktracker.sid', {
65       path: '/', httpOnly: true, sameSite: 'lax', secure: process.env.NODE_ENV === '
.   production',
66     })
67     res.json({ message: 'Signed out' })
68   })
69 })
70
71 module.exports = router
```

backend/routes/profile.js

Backend middleware and routes | 60 source lines | 3,734 bytes

SHA-256: 09214bbabb0923a8593131a77ed9c0e4d0a603415a98d893a90cdc9b6dafbeae

```
1  const express = require('express')
2  const bcrypt = require('bcryptjs')
3  const { rateLimit } = require('express-rate-limit')
4  const pool = require('../config/database')
5  const jsonBody = require('../middleware/jsonBody')
6  const { validPassword, hashPassword } = require('../config/passwords')
7  const { userFields, userInput, fail, managementTransaction } = require('../config/management')
8
9  const router = express.Router()
10 const passwordLimiter = rateLimit({
11   windowMs: 15 * 60 * 1000, limit: 10,
12   keyGenerator: req => String(req.account.id),
13   standardHeaders: 'draft-8', legacyHeaders: false,
14   message: { error: 'Too many password-change attempts. Please try again in 15 minutes.'
15 },
16 })
17 router.put('/', jsonBody, async (req, res, next) => {
18   try {
19     // The session supplies the identity. Reject privileged fields instead of accepting mass assignment.
20     if (Object.keys(req.body).some(key => ![ 'name', 'email' ].includes(key))) {
21       fail(400, 'Only name and email can be changed in your profile')
22     }
23     userInput({ ...req.body, role: req.account.role })
24     const result = await pool.query(`UPDATE users SET name = $1, email = $2,
25       updated_at = CURRENT_TIMESTAMP WHERE id = $3 AND active = true
26       RETURNING ${userFields}`, [req.body.name.trim(), req.body.email.trim().toLowerCase(), req.account.id])
27     if (!result.rows[0]) fail(401, 'Please sign in to continue')
28     res.json(result.rows[0])
29   } catch (error) { next(error) }
30 })
31
32 router.put('/password', passwordLimiter, jsonBody, async (req, res, next) => {
33   try {
34     const { currentPassword, newPassword, confirmPassword } = req.body
```

backend/routes/profile.js (continued)

8dea970 | Listing segment 2 of 2

```
35     if (Object.keys(req.body).some(key => ![ 'currentPassword', 'newPassword', 'confirm
. Password'].includes(key))) {
36         fail(400, 'Only password-change fields are allowed')
37     }
38     if (typeof currentPassword !== 'string' || !currentPassword || bcrypt.truncates(cu
. rrentPassword)) fail(400, 'Enter a valid current password')
39     if (!validPassword(newPassword)) fail(400, 'New password must be at least 12 chara
. cters and at most 72 UTF-8 bytes')
40     if (typeof confirmPassword !== 'string' || confirmPassword !== newPassword) fail(4
. 00, 'New password confirmation does not match')
41     if (currentPassword === newPassword) fail(400, 'Choose a new password different fr
. om your current password')
42
43     await managementTransaction(async client => {
44         // Re-read under the existing account lock; two changes cannot both verify an
. outdated password.
45         const result = await client.query('SELECT password_hash FROM users WHERE id =
. $1 AND active = true FOR UPDATE', [req.account.id])
46         const hash = result.rows[0]?.password_hash
47         if (!hash) fail(401, 'Please sign in to continue')
48         if (!await bcrypt.compare(currentPassword, hash)) fail(400, 'Current password
. is incorrect')
49         const replacement = await hashPassword(newPassword)
50         await client.query('UPDATE users SET password_hash = $1, updated_at = CURRENT_
. TIMESTAMP WHERE id = $2', [replacement, req.account.id])
51         // Commit the new hash and all-device session revocation together.
52         await client.query("DELETE FROM sessions WHERE sess->>'userId' = $1", [String(
. req.account.id)])
53     })
54     await new Promise((resolve, reject) => req.session.destroy(error => error ? reject
. (error) : resolve()))
55     res.clearCookie('worktracker.sid', { path: '/', httpOnly: true, sameSite: 'lax', s
. ecure: process.env.NODE_ENV === 'production' })
56     res.json({ message: 'Your password has been changed. Please sign in again.' })
57 } catch (error) { next(error) }
58 })
59
60 module.exports = router
```

backend/routes/tasks.js

Backend middleware and routes | 160 source lines | 6,605 bytes

SHA-256: f77e157cac877eb2caa7a904c00bb01528bac29a81521167d636c4d8d68f997e

```
1  const express = require('express')
2  const pool = require('../config/database')
3  const jsonBody = require('../middleware/jsonBody')
4  const { requireAdmin } = require('../middleware/auth')
5
6  const { taskFields, fail, validId, managementTransaction } = require('../config/management')
7  const router = express.Router()
8
9  const validStatuses = [
10     'Open',
11     'In Progress',
12     'Completed',
13 ]
14
15 const validPriorities = [
16     'Low',
17     'Medium',
18     'High',
19 ]
20
21 router.get('/', async (req, res, next) => {
22     try {
23         const result = await pool.query(`
24             SELECT
25                 id,
26                 title,
27                 description,
28                 status,
29                 priority,
30                 user_id AS "userId",
31                 created_at AS "createdAt",
32                 updated_at AS "updatedAt"
33             FROM tasks
34             WHERE (${1}::boolean OR user_id = ${2})
35             ORDER BY id
36         `, [req.account.role === 'admin', req.account.id])
37
38         res.json(result.rows)
39     } catch (error) {
40         next(error)
41     }
42 })
43
44 function taskInput(body) {
```

backend/routes/tasks.js (continued)

8dea970 | Listing segment 2 of 4

```

45     const { title, description = '', priority = 'Medium', status = 'Open', userId = null }
46     = body
47     if (typeof title !== 'string' || !title.trim() || [...title.trim()].length > 200) fail
48     (400, 'Task title must contain 1 to 200 characters')
49     if (typeof description !== 'string' || [...description].length > 10000) fail(400, 'Des
50     cription must be a string of at most 10000 characters')
51     if (!validPriorities.includes(priority)) fail(400, 'Invalid task priority')
52     if (!validStatuses.includes(status)) fail(400, 'Invalid task status')
53     if (userId !== null && (!Number.isInteger(userId) || !validId(userId))) fail(400, 'Ass
54     igned user ID must be a positive integer or null')
55     return [title.trim(), description.trim(), priority, userId, status]
56 }
57
58 async function saveTask(req, editing) {
59     const values = taskInput(req.body)
60     return managementTransaction(async client => {
61         const actor = await client.query("SELECT id FROM users WHERE id = $1 AND active =
62         true AND role = 'admin' AND password_hash IS NOT NULL", [req.account.id])
63         if (!actor.rows.length) fail(403, 'Admin access required')
64         let existing
65         if (editing) {
66             existing = (await client.query('SELECT user_id AS "userId" FROM tasks WHERE id
67             = $1 FOR UPDATE', [Number(req.params.id)]).rows[0])
68             if (!existing) fail(404, 'Task not found')
69         }
70         if (values[3] !== null) {
71             const user = (await client.query('SELECT id, active FROM users WHERE id = $1',
72             [values[3]]).rows[0])
73             if (!user) fail(400, 'Assigned user does not exist')
74             // Preserve an existing inactive assignment, but never assign new work to it.
75             if (user.active === false && (!editing || existing.userId !== values[3])) fail
76             (400, 'Cannot assign a task to an inactive account')
77         }
78         const result = editing
79         ? await client.query(`UPDATE tasks SET title = $1, description = $2, priority
80         = $3,
81         user_id = $4, status = $5, updated_at = CURRENT_TIMESTAMP WHERE id = $6 RE
82         TURNING ${taskFields}`, [...values, Number(req.params.id)])
83         : await client.query(`INSERT INTO tasks (title, description, priority, user_id
84         , status)
85         VALUES ($1, $2, $3, $4, $5) RETURNING ${taskFields}`, values)
86         return result.rows[0]
87     })
88 }

```

backend/routes/tasks.js (continued)

8dea970 | Listing segment 3 of 4

```

79 router.param('id', (req, res, next, id) => {
80   if (!validId(id)) return res.status(400).json({ error: 'Task ID must be a positive int
.   eger' })
81   next()
82 })
83 router.get('/:id', async (req, res, next) => {
84   try {
85     const result = await pool.query(`SELECT ${taskFields} FROM tasks WHERE id = $1 AND
.     ($2::boolean OR user_id = $3)`, [Number(req.params.id), req.account.role === 'admin', req
.     .account.id])
86     if (!result.rows[0]) fail(404, 'Task not found')
87     res.json(result.rows[0])
88   } catch (error) { next(error) }
89 })
90 router.post('/', requireAdmin, jsonBody, async (req, res, next) => {
91   try { res.status(201).json(await saveTask(req, false)) }
92   catch (error) {
93     if (error.code === '23503') return res.status(400).json({ error: 'Assigned user do
.   es not exist' })
94     next(error)
95   }
96 })
97 router.put('/:id', requireAdmin, jsonBody, async (req, res, next) => {
98   try { res.json(await saveTask(req, true)) } catch (error) { next(error) }
99 })
100 router.delete('/:id', requireAdmin, async (req, res, next) => {
101   try {
102     const result = await pool.query('DELETE FROM tasks WHERE id = $1 RETURNING id', [N
.   umber(req.params.id)])
103     if (!result.rows.length) fail(404, 'Task not found')
104     res.json({ message: 'Task deleted' })
105   } catch (error) { next(error) }
106 })
107
108 router.patch('/:id/status', jsonBody, async (req, res, next) => {
109   try {
110     const { id } = req.params
111     const { status } = req.body
112     // Users can only change status; a valid status must not disguise forbidden field
.   edits.
113     if (req.account.role !== 'admin' && Object.keys(req.body).some(key => key !== 'sta
.   tus')) {
114       return res.status(403).json({ error: 'You may only change task status' })
115     }
116     if (! /^[1-9]\d*$/.test(id) || Number(id) > 2147483647) {

```

backend/routes/tasks.js (continued)

8dea970 | Listing segment 4 of 4

```
117         return res.status(400).json({ error: 'Task ID must be a positive integer' })
118     }
119
120     if (!validStatuses.includes(status)) {
121         return res.status(400).json({
122             error: 'Invalid task status',
123         })
124     }
125
126     const result = await pool.query(
127         `
128         UPDATE tasks
129         SET
130             status = $1,
131             updated_at = CURRENT_TIMESTAMP
132         WHERE id = $2 AND ($3::boolean OR user_id = $4)
133         RETURNING
134             id,
135             title,
136             description,
137             status,
138             priority,
139             user_id AS "userId",
140             created_at AS "createdAt",
141             updated_at AS "updatedAt"
142         `,
143         [status, Number(id), req.account.role === 'admin', req.account.id]
144     )
145
146     if (result.rows.length === 0) {
147         // Use 403 for regular users without revealing whether an unowned task exists.
148         if (req.account.role !== 'admin') return res.status(403).json({ error: 'You ma
149 . y only update your assigned tasks' })
150         return res.status(404).json({
151             error: 'Task not found',
152         })
153     }
154     res.json(result.rows[0])
155 } catch (error) {
156     next(error)
157 }
158 })
159
160 module.exports = router
```

backend/routes/users.js

Backend middleware and routes | 146 source lines | 6,841 bytes

SHA-256: b74d6c98714a7d531c4e155bb127b263fdcf8142cfb290565776ac27f2610250

```

1  const express = require('express')
2  const pool = require('../config/database')
3  const jsonBody = require('../middleware/jsonBody')
4  const { validPassword, hashPassword } = require('../config/passwords')
5  const { userFields, fail, validId, userInput, managementTransaction } = require('../config
  /management')
6
7  const router = express.Router()
8
9  router.get('/', async (req, res, next) => {
10     try {
11         const result = await pool.query(`
12             SELECT
13                 id,
14                 name,
15                 email,
16                 role,
17                 active,
18                 updated_at AS "updatedAt",
19                 (password_hash IS NOT NULL) AS "canLogin",
20                 created_at AS "createdAt"
21             FROM users
22             ORDER BY id
23         `)
24
25         res.json(result.rows)
26     } catch (error) {
27         next(error)
28     }
29 })
30
31 router.post('/', jsonBody, async (req, res, next) => {
32     try {
33         const { name, email, password, role = 'user' } = req.body
34
35         // Validate types before trimming; match the database's field limits.
36         if (typeof name !== 'string' || !name.trim() || [...name.trim()].length > 100) {
37             return res.status(400).json({ error: 'Name must contain 1 to 100 characters' })
38         }
39         if (typeof email !== 'string' || email.trim().length > 255 ||
40             !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.trim())) {
41             return res.status(400).json({ error: 'A valid email of up to 255 characters is
  required' })
42         }
43
44         if (!validPassword(password)) {
45             return res.status(400).json({ error: 'Password must be at least 12 characters
  and at most 72 UTF-8 bytes' })
46         }
47         if (!['admin', 'user'].includes(role)) {
48             return res.status(400).json({ error: 'Role must be admin or user' })
49         }
50         const passwordHash = await hashPassword(password)
51         const result = await pool.query(
52             `
53             INSERT INTO users (name, email, password_hash, role)

```

backend/routes/users.js (continued)

8dea970 | Listing segment 2 of 3

```

54         VALUES ($1, $2, $3, $4)
55         RETURNING ${userFields}
56     `,
57     [
58         name.trim(),
59         email.trim().toLowerCase(),
60         passwordHash,
61         role,
62     ]
63 )
64
65     res.status(201).json(result.rows[0])
66 } catch (error) {
67     if (error.code === '23505') {
68         return res.status(409).json({
69             error: 'Email already exists',
70         })
71     }
72
73     next(error)
74 }
75 })
76
77 router.param('id', (req, res, next, id) => {
78     if (!validId(id)) return res.status(400).json({ error: 'User ID must be a positive integer' })
79     next()
80 })
81
82 router.get('/:id', async (req, res, next) => {
83     try {
84         const result = await pool.query(`SELECT ${userFields} FROM users WHERE id = $1`, [
85             Number(req.params.id)]
86         )
87         if (!result.rows[0]) fail(404, 'User not found')
88         const summary = await pool.query(`SELECT count(*)::int AS total,
89             count(*) FILTER (WHERE status = 'Open')::int AS open,
90             count(*) FILTER (WHERE status = 'In Progress')::int AS "inProgress",
91             count(*) FILTER (WHERE status = 'Completed')::int AS completed
92             FROM tasks WHERE user_id = $1`, [Number(req.params.id)])
93         res.json({ ...result.rows[0], taskSummary: summary.rows[0] })
94     } catch (error) { next(error) }
95 })
96
97 async function changeUser(req, action) {
98     const id = Number(req.params.id)
99     return managementTransaction(async client => {
100         // Recheck the acting Admin after acquiring the lock, including concurrent revocations.
101         const actor = await client.query("SELECT id FROM users WHERE id = $1 AND active = true AND role = 'admin' AND password_hash IS NOT NULL", [req.account.id])
102         if (!actor.rows.length) fail(403, 'Admin access required')
103         const result = await client.query(`SELECT ${userFields} FROM users WHERE id = $1`, [id])
104         const user = result.rows[0]
105         if (!user) fail(404, 'User not found')
106         const removesAccess = action === 'delete' || (action === 'status' && !req.body.active) || (action === 'edit' && req.body.role !== 'admin')

```

backend/routes/users.js (continued)

8dea970 | Listing segment 3 of 3

```

105     if (id === req.account.id && removesAccess) fail(409, 'You cannot delete, deactiva
. te, or demote your own Admin account')
106     if (user.role === 'admin' && user.active && user.canLogin && removesAccess) {
107         const others = await client.query("SELECT id FROM users WHERE id <> $1 AND rol
. e = 'admin' AND active = true AND password_hash IS NOT NULL", [id])
108         if (!others.rows.length) fail(409, 'At least one usable Admin account must rem
. ain')
109     }
110     if (action === 'edit' && req.body.role === 'admin' && !user.canLogin) fail(409, 'E
. nable login for this legacy account before granting Admin access')
111     if (action === 'delete') {
112         const assigned = await client.query('SELECT id FROM tasks WHERE user_id = $1 L
. IMIT 1', [id])
113         if (assigned.rows.length) fail(409, 'This user cannot be deleted while tasks a
. re assigned. Reassign or delete those tasks first.')
114         await client.query('DELETE FROM users WHERE id = $1', [id])
115     } else if (action === 'status') {
116         await client.query('UPDATE users SET active = $1, updated_at = CURRENT_TIMESTA
. MP WHERE id = $2', [req.body.active, id])
117     } else {
118         await client.query('UPDATE users SET name = $1, email = $2, role = $3, updated
. _at = CURRENT_TIMESTAMP WHERE id = $4', [req.body.name.trim(), req.body.email.trim().toLow
. erCase(), req.body.role, id])
119     }
120     // Remove sessions when access is revoked, so reactivation cannot revive a copied
. cookie.
121     if (action === 'delete' || (action === 'status' && !req.body.active) || (action ==
. = 'edit' && user.role !== req.body.role)) {
122         await client.query("DELETE FROM sessions WHERE sess->'userId' = $1", [String(
. id)])
123     }
124     if (action === 'delete') return { message: 'User deleted' }
125     return (await client.query(`SELECT ${userFields} FROM users WHERE id = $1`, [id]))
. .rows[0]
126     })
127 }
128
129 router.put('/:id', jsonBody, async (req, res, next) => {
130     try {
131         userInput(req.body)
132         if ('password' in req.body || 'password_hash' in req.body || 'active' in req.body)
. fail(400, 'Use the account status action separately; passwords cannot be edited here')
133         res.json(await changeUser(req, 'edit'))
134     } catch (error) { next(error) }
135 })
136 router.patch('/:id/status', jsonBody, async (req, res, next) => {
137     try {
138         if (typeof req.body.active !== 'boolean') fail(400, 'Active must be true or false'
. )
139         res.json(await changeUser(req, 'status'))
140     } catch (error) { next(error) }
141 })
142 router.delete('/:id', async (req, res, next) => {
143     try { res.json(await changeUser(req, 'delete')) } catch (error) { next(error) }
144 }
145
146 module.exports = router

```

backend/database/accounts.sql

Database and maintenance scripts | 17 source lines | 925 bytes

SHA-256: 5e89d3a3b467ce33aab9ef9dca9067bc7f5ac2deb0396b3177f61ea6532fef21

```
1  -- Additive migration: preserve existing users, task assignments, and sequences.
2  -- Legacy users have no password and cannot log in until deliberately activated.
3  ALTER TABLE users ADD COLUMN IF NOT EXISTS password_hash TEXT;
4  ALTER TABLE users ADD COLUMN IF NOT EXISTS role VARCHAR(10) NOT NULL DEFAULT 'user';
5  DO $$ BEGIN
6      IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conrelid = 'users'::regclass AND conname = 'users_role_check') THEN
7          ALTER TABLE users ADD CONSTRAINT users_role_check CHECK (role IN ('admin', 'user'))
8      );
9      END IF;
10     END $$;
11 -- Fail safely if legacy case-insensitive email collisions need manual resolution.
12 CREATE UNIQUE INDEX IF NOT EXISTS users_email_lower_key ON users (lower(email));
13 CREATE TABLE IF NOT EXISTS sessions (
14     sid VARCHAR NOT NULL PRIMARY KEY,
15     sess JSON NOT NULL,
16     expire TIMESTAMP(6) NOT NULL
17 );
18 CREATE INDEX IF NOT EXISTS sessions_expire_idx ON sessions (expire);
```

backend/database/admin-management.sql

Database and maintenance scripts | 3 source lines | 256 bytes

SHA-256: b069d430fd7d513bde10ba2dd30e8b6b20e57186d94d85dcb4b947a218c6743d

```
1  -- Existing accounts remain active; legacy records still need a password to log in.  
2  ALTER TABLE users ADD COLUMN IF NOT EXISTS active BOOLEAN NOT NULL DEFAULT true;  
3  ALTER TABLE users ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

backend/database/schema.sql

Database and maintenance scripts | 43 source lines | 1,850 bytes

SHA-256: 6a63ab08c9192a528f472ff6d317f3ee8cbf62395e9025545df813687b12719d

```

1  -- Run against an already-created database. These statements never replace existing tables
2  .
3  -- IF NOT EXISTS does not upgrade older table definitions; inspect differences separately.
4  CREATE TABLE IF NOT EXISTS users (
5      id SERIAL PRIMARY KEY,
6      name VARCHAR(100) NOT NULL,
7      email VARCHAR(255) UNIQUE NOT NULL,
8      -- NULL is reserved for preserved legacy assignment records that cannot log in.
9      password_hash TEXT,
10     active BOOLEAN NOT NULL DEFAULT true,
11     role VARCHAR(10) NOT NULL DEFAULT 'user' CONSTRAINT users_role_check CHECK (role IN ('
12     admin', 'user')),
13     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
14     -- Management edits explicitly refresh this timestamp.
15     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
16 );
17
18 CREATE TABLE IF NOT EXISTS tasks (
19     id SERIAL PRIMARY KEY,
20     title VARCHAR(200) NOT NULL,
21     description TEXT,
22     status VARCHAR(20) NOT NULL DEFAULT 'Open',
23     priority VARCHAR(20) NOT NULL DEFAULT 'Medium',
24     -- Preserve tasks if a user is removed through an external database operation.
25     user_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
26     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
27     -- The API explicitly updates this timestamp when task status changes.
28     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
29
30     CONSTRAINT tasks_status_check
31         CHECK (status IN ('Open', 'In Progress', 'Completed')),
32
33     CONSTRAINT tasks_priority_check
34         CHECK (priority IN ('Low', 'Medium', 'High'))
35 );
36
37 CREATE UNIQUE INDEX IF NOT EXISTS users_email_lower_key ON users (lower(email));
38
39 -- Opaque session IDs refer to server-side data; expiration supports automatic pruning.
40 CREATE TABLE IF NOT EXISTS sessions (
41     sid VARCHAR NOT NULL PRIMARY KEY,
42     sess JSON NOT NULL,
43     expire TIMESTAMP(6) NOT NULL
44 );
45
46 CREATE INDEX IF NOT EXISTS sessions_expire_idx ON sessions (expire);

```

backend/scripts/activate-legacy-user.js

Database and maintenance scripts | 21 source lines | 1,188 bytes

SHA-256: 45305c1d86ef569e64a3f6f388595c6918bea31d778bbb304cef672579676b4b

```
1  const pool = require('../config/database')
2  const { validPassword, hashPassword } = require('../config/passwords')
3  const validate = require('../config/environment')
4
5  // Deliberately activate a preserved assignment record without replacing its ID or existin
6  . g credentials.
7  async function activate() {
8      try {
9          validate()
10         const email = process.env.LEGACY_ACCOUNT_EMAIL?.trim().toLowerCase()
11         const password = process.env.LEGACY_ACCOUNT_PASSWORD
12         if (!email || !validPassword(password)) throw new Error('Set LEGACY_ACCOUNT_EMAIL
13         . and a valid LEGACY_ACCOUNT_PASSWORD privately')
14         const result = await pool.query(`UPDATE users SET password_hash = $1, updated_at =
15         . CURRENT_TIMESTAMP
16         WHERE lower(email) = $2 AND password_hash IS NULL AND role = 'user' RETURNING
17         . id`,
18         [await hashPassword(password), email])
19         console.log(result.rowCount ? 'Legacy User login enabled; assignments preserved.'
20         . : 'No legacy User without a password matched; nothing changed.')
21         } catch (error) {
22             console.error('Legacy activation failed:', error.code || error.message)
23             process.exitCode = 1
24         } finally { await pool.end() }
25     }
26     activate()
```

backend/scripts/bootstrap-admin.js

Database and maintenance scripts | 42 source lines | 2,091 bytes

SHA-256: ca2ccf916961a8fd6f85cb672a91e7c585222e82ab5aee3a8e005ee522918c1a

```

1  const pool = require('../config/database')
2  const validate = require('../config/environment')
3  const { validPassword, hashPassword } = require('../config/passwords')
4
5  async function bootstrap() {
6      let client
7      try {
8          validate()
9          const name = process.env.BOOTSTRAP_ADMIN_NAME?.trim()
10         const email = process.env.BOOTSTRAP_ADMIN_EMAIL?.trim().toLowerCase()
11         const password = process.env.BOOTSTRAP_ADMIN_PASSWORD
12         if (!name || name.length > 100 || !email || email.length > 255 ||
13             !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || !validPassword(password) ||
14             password === 'your_private_initial_admin_password') {
15             throw new Error('Set valid BOOTSTRAP_ADMIN_NAME, BOOTSTRAP_ADMIN_EMAIL and BOO
16             TSTRAP_ADMIN_PASSWORD (12 characters minimum, 72 bytes maximum)')
17         }
18         const hash = await hashPassword(password)
19         client = await pool.connect()
20         await client.query('BEGIN')
21         // Serialize bootstrap attempts so two processes cannot both create the first admin.
22         await client.query('LOCK TABLE users IN SHARE ROW EXCLUSIVE MODE')
23         const admins = await client.query("SELECT id FROM users WHERE role = 'admin' AND password_hash IS NOT NULL AND active = true")
24         if (admins.rowCount) {
25             await client.query('ROLLBACK')
26             console.log('An active Admin already exists; no account was changed.')
27             return
28         }
29         // Never overwrite an existing user's credentials or silently promote an assignment record.
30         await client.query(`INSERT INTO users (name, email, password_hash, role)
31             VALUES ($1, $2, $3, 'admin')`, [name, email, hash])
32         await client.query('COMMIT')
33         console.log('Initial Admin created. Password was not logged.')
34     } catch (error) {
35         if (client) await client.query('ROLLBACK').catch(() => {})
36         console.error('Admin bootstrap failed:', error.code || error.message)
37         process.exitCode = 1
38     } finally {
39         if (client) client.release()
40         await pool.end()
41     }
42 }
bootstrap()

```

backend/scripts/check-database.js

Database and maintenance scripts | 26 source lines | 956 bytes

SHA-256: 349887d5a13a7070a9719e6f4e8188b441e8a236ec2a86e178de240775d769d8

```
1  const validateDatabaseEnvironment = require('../config/environment')
2  const pool = require('../config/database')
3
4  // Check the actual connection identity and table presence without changing data.
5  async function checkDatabase() {
6      try {
7          validateDatabaseEnvironment()
8          const result = await pool.query(`
9              SELECT current_database() AS database, current_user AS role,
10                 to_regclass('public.users') AS users,
11                 to_regclass('public.tasks') AS tasks
12            `)
13          const info = result.rows[0]
14          console.log(info)
15          if (info.database !== process.env.DB_NAME || !info.users || !info.tasks) {
16              throw new Error('Configured database identity or required tables do not match'
17              .)
18          }
19      } catch (error) {
20          console.error('Database check failed:', error.code || error.message)
21          process.exitCode = 1
22      } finally {
23          await pool.end()
24      }
25  }
26  checkDatabase()
```

backend/scripts/init-database.js

Database and maintenance scripts | 29 source lines | 1,178 bytes

SHA-256: 8517b85a5d3735af927ffa4594ff3b8f08ea8a55c9a629cf138976613b288feb

```
1  const fs = require('node:fs/promises')
2  const path = require('node:path')
3  const validateDatabaseEnvironment = require('../config/environment')
4  const pool = require('../config/database')
5
6  // Initialize missing tables only. Existing tables require inspection, not automatic repla
7  cement.
8  async function initializeDatabase() {
9      let client
10     try {
11         validateDatabaseEnvironment()
12         client = await pool.connect()
13         const schema = await fs.readFile(path.join(__dirname, '../database/schema.sql'), '
14         utf8')
15         await client.query('BEGIN')
16         // Stop rather than wait indefinitely for another session's schema lock.
17         await client.query("SET LOCAL lock_timeout = '5s'")
18         await client.query(schema)
19         await client.query('COMMIT')
20         console.log('Schema initialization finished. Existing tables and records were pres
21         erved.')
22     } catch (error) {
23         if (client) await client.query('ROLLBACK').catch(() => {})
24         console.error('Schema initialization failed:', error.code || error.message)
25         process.exitCode = 1
26     } finally {
27         if (client) client.release()
28         await pool.end()
29     }
30 }
31
32 initializeDatabase()
```

backend/scripts/migrate-accounts.js

Database and maintenance scripts | 25 source lines | 886 bytes

SHA-256: 8ed71ab5f5a177a900d51a535645cd9a51899394da86b79603722d9a4062922a

```
1  const fs = require('node:fs/promises')
2  const path = require('node:path')
3  const pool = require('../config/database')
4  const validate = require('../config/environment')
5
6  async function migrate() {
7    let client
8    try {
9      validate()
10     client = await pool.connect()
11     await client.query('BEGIN')
12     await client.query("SET LOCAL lock_timeout = '5s'")
13     await client.query(await fs.readFile(path.join(__dirname, '../database/accounts.sql'), 'utf8'))
14     await client.query('COMMIT')
15     console.log('Account migration complete; existing records preserved.')
16   } catch (error) {
17     if (client) await client.query('ROLLBACK').catch(() => {})
18     console.error('Account migration failed:', error.code || error.message)
19     process.exitCode = 1
20   } finally {
21     if (client) client.release()
22     await pool.end()
23   }
24 }
25 migrate()
```

backend/scripts/migrate-management.js

Database and maintenance scripts | 25 source lines | 912 bytes

SHA-256: 8227ee697d2b5020b8eafdf17da0bf5a85a31be1398bb11c1b859305fd5812de

```
1  const fs = require('node:fs/promises')
2  const path = require('node:path')
3  const pool = require('../config/database')
4  const validate = require('../config/environment')
5
6  async function migrate() {
7      let client
8      try {
9          validate()
10         client = await pool.connect()
11         await client.query('BEGIN')
12         await client.query("SET LOCAL lock_timeout = '5s'")
13         await client.query(await fs.readFile(path.join(__dirname, '../database/admin-manag
14         .ement.sql'), 'utf8'))
15         await client.query('COMMIT')
16         console.log('Admin management migration complete; existing records preserved.')
17     } catch (error) {
18         if (client) await client.query('ROLLBACK').catch(() => {})
19         console.error('Admin management migration failed:', error.code || error.message)
20         process.exitCode = 1
21     } finally {
22         if (client) client.release()
23         await pool.end()
24     }
25     migrate()
}
```

backend/scripts/verify-stage5.cjs

Database and maintenance scripts | 121 source lines | 9,509 bytes

SHA-256: 703b3729eaf8fa0ca896756f8156cb11c908a8fe49dc9498220a4493c018a257

```

1 // Opt-in live verification: uses disposable records, never resets data or sequences.
2 const assert = require('node:assert/strict')
3 const { randomBytes, createHash } = require('node:crypto')
4 const bcrypt = require('bcryptjs')
5 const pool = require('../config/database')
6 const base = process.env.STAGE5_API_URL || 'http://127.0.0.1:3000/api'
7 const marker = `stage5-${randomBytes(6).toString('hex')}`
8 const password = randomBytes(24).toString('hex'), nextPassword = randomBytes(24).toString(
  . 'hex')
9 const users = [], tasks = [], cookies = []
10 let admin, checks = 0
11 async function call(path, method = 'GET', body, cookie = admin, expected = 200, raw = fals
  . e) {
12     const r = await fetch(base + path, { method, headers: { Cookie: cookie || '',
13       'Content-Type': 'application/json', 'X-Requested-With': 'WorkTracker' },
14       body: body === undefined ? undefined : raw ? body : JSON.stringify(body) })
15     const data = await r.json(), text = JSON.stringify(data)
16     for (const secret of [password, nextPassword, process.env.DB_PASSWORD, process.env.SES
  . SION_SECRET,
17       process.env.Bootstrap_ADMIN_PASSWORD, 'password_hash', 'stack', 'SELECT ']) {
18       if (secret) assert.ok(!text.includes(secret), 'Response must not expose private da
  . ta')
19     }
20     assert.equal(r.status, expected, `${method} ${path}: ${r.status}, expected ${expected}
  . `)
21     checks++
22     return { data, cookie: r.headers.get('set-cookie')?.split(';')[0], headers: r.headers
  . }
23 }
24 async function login(email, value = password) {
25     const r = await call('/auth/login', 'POST', { email, password: value }, '', 200)
26     assert.match(r.headers.get('set-cookie'), /HttpOnly/)
27     assert.match(r.headers.get('set-cookie'), /SameSite=Lax/)
28     cookies.push(r.cookie)
29     return r.cookie
30 }
31 async function snapshot() {
32     const records = await pool.query('SELECT to_jsonb(u) AS record FROM users u ORDER BY i
  . d')
33     const work = await pool.query('SELECT to_jsonb(t) AS record FROM tasks t ORDER BY id')
34     return { users: records.rowCount, tasks: work.rowCount,
35       digest: createHash('sha256').update(JSON.stringify([records.rows, work.rows])).dig
  . est('hex') }
36 }
37 async function main() {
38     const before = await snapshot()
39     try {
40         admin = await login(process.env.Bootstrap_ADMIN_EMAIL, process.env.Bootstrap_ADMIN
  . _PASSWORD)
41         const me = (await call('/auth/me')).data.user
42         assert.equal(me.role, 'admin')
43         await call('/health')
44         for (const path of ['/users', '/tasks', '/users/1', '/tasks/1']) await call(path,
  . 'GET', undefined, '', 401)
45         await call('/tasks', 'GET', undefined, 'worktracker.sid=forged', 401)
46         await call('/auth/login', 'POST', { email: me.email, password: 'incorrect-test-onl

```

backend/scripts/verify-stage5.cjs (continued)

8dea970 | Listing segment 2 of 3

```

.   y' }, '', 401)
47     await call('/auth/login', 'POST', { email: [], password: {} }, '', 401)
48     for (const suffix of ['a', 'b']) {
49         const data = (await call('/users', 'POST', { name: `${marker}-${suffix}`, email:
.   1: `${marker}-${suffix}@example.invalid`, password }, admin, 201)).data
50         users.push(data)
51     }
52     const [a, b] = users
53     const userCookie = await login(a.email), secondCookie = await login(a.email)
54     const hashed = (await pool.query('SELECT password_hash FROM users WHERE id=$1', [a
.   .id])).rows[0].password_hash
55     assert.notEqual(hashed, password); assert.equal(await bcrypt.compare(password, hashed), true)
.   )
56     assert.equal((await call('/users')).data.some(u => u.id === a.id), true)
57     await call(`/users/${a.id}`)
58     await call('/users', 'POST', { ...a, password }, admin, 409)
59     for (const change of [{ name: ' ' }, { name: 'x'.repeat(101) }, { email: [] }, { email: 'bad' }, { role: 'owner' }, { password: 'short' }, { password: '\U0001f600'.repeat(19) }]) {
.   . 9) ]]) {
60         await call('/users', 'POST', { name: marker, email: `${marker}-invalid@example.invalid`, password, ...change }, admin, 400)
.   )
61     }
62     for (const [path, method, body] of [['/users/${me.id}', 'DELETE'], ['/users/${me.id}/status', 'PATCH', { active: false }], ['/users/${me.id}', 'PUT', { name: me.name, email: me.email, role: 'user' }]]) await call(path, method, body, admin, 409)
.   .
63     const edited = (await call(`/users/${a.id}`, 'PUT', { name: `${marker} Edited`, email: a.email, role: 'user' })).data
.   .
64     assert.ok(new Date(edited.updatedAt) >= new Date(a.updatedAt))
65     for (const owner of [a.id, b.id]) {
66         tasks.push((await call('/tasks', 'POST', { title: `${marker} Work`, description: "<script>alert('test')</script> ' OR 1=1 --", priority: 'High', userId: owner }, admin, 201)).data)
.   .
67     }
68     const [ta, tb] = tasks
69     assert.equal((await call('/tasks?role=admin&userId=1', 'GET', undefined, userCookie)).data.length, 1)
.   .
70     await call(`/tasks/${ta.id}`, 'GET', undefined, userCookie)
71     await call(`/tasks/${tb.id}`, 'GET', undefined, userCookie, 404)
72     await call(`/tasks/${tb.id}/status`, 'PATCH', { status: 'Completed' }, userCookie, 403)
.   .
73     for (const [path, method] of [['/users', 'GET'], ['/users', 'POST'], ['/users/${b.id}', 'GET'], ['/users/${b.id}', 'PUT'], ['/users/${b.id}/status', 'PATCH'], ['/users/${b.id}', 'DELETE'], ['/tasks', 'POST'], ['/tasks/${ta.id}', 'PUT'], ['/tasks/${ta.id}', 'DELETE']]) await call(path, method, method === 'GET' ? undefined : {}, userCookie, 403)
.   .
74     for (const key of ['title', 'description', 'priority', 'userId', 'role']) await call(`/tasks/${ta.id}/status`, 'PATCH', { status: 'Completed', [key]: b.id }, userCookie, 403)
.   .
75     for (const key of ['role', 'active', 'id', 'password_hash']) await call('/profile', 'PUT', { name: a.name, email: a.email, [key]: 'forged' }, userCookie, 400)
76     for (const change of [{ title: ' ' }, { title: 'x'.repeat(201) }, { description: { }, { description: 'x'.repeat(10001) }, { status: 'Done' }, { priority: 'Urgent' }, { userId: '1' }, { userId: 2147483647 }]) await call('/tasks', 'POST', { title: marker, ...change }, admin, 400)
.   .
77     for (const id of ['0', '-1', 'abc', '2147483648']) await call(`/tasks/${id}`, 'GET', undefined, admin, 400)
.   .
78     await call('/tasks/2147483647', 'GET', undefined, admin, 404)

```

backend/scripts/verify-stage5.cjs (continued)

8dea970 | Listing segment 3 of 3

```

79     await call('/tasks', 'POST', '{', admin, 400, true)
80     await call('/tasks', 'POST', [], admin, 400)
81     await call('/tasks', 'POST', { title: 'x'.repeat(110000) }, admin, 413)
82     await call(`/tasks/${ta.id}/status`, 'PATCH', { status: 'Completed' }, userCookie)
83     assert.equal((await pool.query('SELECT status FROM tasks WHERE id=$1', [ta.id])).r
.   ows[0].status, 'Completed')
84     await call(`/users/${a.id}`, 'DELETE', undefined, admin, 409)
85     await call(`/tasks/${ta.id}`, 'PUT', { ...ta, title: `${marker} Reassigned`, statu
.   s: 'In Progress', priority: 'Low', userId: b.id })
86     await call(`/tasks/${ta.id}`, 'GET', undefined, userCookie, 404)
87     assert.equal((await pool.query('SELECT user_id FROM tasks WHERE id=$1', [ta.id])).
.   rows[0].user_id, b.id)
88     await call('/profile', 'PUT', { name: a.name, email: b.email }, userCookie, 409)
89     a.email = `${marker}-updated@example.invalid`
90     await call('/profile', 'PUT', { name: `${marker} Profile`, email: a.email }, userC
.   ookie)
91     assert.equal((await pool.query('SELECT email FROM users WHERE id=$1', [a.id])).row
.   s[0].email, a.email)
92     const body = { currentPassword: password, newPassword: nextPassword, confirmPasswo
.   rd: nextPassword }
93     await call('/profile/password', 'PUT', { ...body, currentPassword: 'incorrect-curr
.   ent' }, userCookie, 400)
94     await call('/profile/password', 'PUT', body, userCookie)
95     for (const cookie of [userCookie, secondCookie]) await call('/tasks', 'GET', undef
.   ined, cookie, 401)
96     await call('/auth/login', 'POST', { email: a.email, password }, '', 401)
97     const fresh = await login(a.email, nextPassword)
98     await call(`/users/${a.id}/status`, 'PATCH', { active: false })
99     assert.equal((await pool.query('SELECT active FROM users WHERE id=$1', [a.id])).ro
.   ws[0].active, false)
100    await call('/tasks', 'GET', undefined, fresh, 401)
101    await call('/auth/login', 'POST', { email: a.email, password: nextPassword }, '',
.   401)
102    await call('/tasks', 'POST', { title: marker, userId: a.id }, admin, 400)
103    await call(`/users/${a.id}/status`, 'PATCH', { active: true })
104    await call('/tasks', 'GET', undefined, fresh, 401)
105    const active = await login(a.email, nextPassword)
106    await call('/auth/logout', 'POST', {}, active)
107    await call('/tasks', 'GET', undefined, active, 401)
108    const orphans = await pool.query('SELECT count(*)::int AS n FROM tasks t LEFT JOIN
.   users u ON u.id=t.user_id WHERE t.user_id IS NOT NULL AND u.id IS NULL')
109    assert.equal(orphans.rows[0].n, 0)
110    console.log(JSON.stringify({ liveApiChecks: checks, result: 'PASS' }))
111  } finally {
112    for (const task of tasks) await call(`/tasks/${task.id}`, 'DELETE')
113    for (const user of users) await call(`/users/${user.id}`, 'DELETE')
114    for (const cookie of cookies) await call('/auth/logout', 'POST', {}, cookie)
115    const after = await snapshot()
116    assert.deepEqual(after, before, 'Legitimate records must remain unchanged')
117    console.log(JSON.stringify({ cleanup: 'PASS', remainingUsers: after.users, remaini
.   ngTasks: after.tasks, legitimateDataUnchanged: true }))
118    await pool.end()
119  }
120 }
121 main().catch(error => { console.error(error.message); process.exitCode = 1; pool.end().cat
.   ch(() => {}) })

```

frontend/src/App.jsx

Frontend application and views | 208 source lines | 9,138 bytes

SHA-256: 528884f1274a0f72b08d7ceff9dd6473910c65961bb884c7a1a6c506e5ee734b

```
1  import { useEffect, useRef, useState } from 'react'
2  import * as api from './services/api.js'
3  import DataState from './components/DataState.jsx'
4  import Dashboard from './pages/Dashboard.jsx'
5  import Users from './pages/Users.jsx'
6  import Tasks from './pages/Tasks.jsx'
7  import AdminTasks from './pages/AdminTasks.jsx'
8  import Login from './pages/Login.jsx'
9  import Profile from './pages/Profile.jsx'
10
11  function App() {
12    const [account, setAccount] = useState(null)
13    const [checkingSession, setCheckingSession] = useState(true)
14    const [sessionError, setSessionError] = useState('')
15    const [signInMessage, setSignInMessage] = useState('')
16    const [sessionAttempt, setSessionAttempt] = useState(0)
17    const [currentPage, setCurrentPage] = useState('dashboard')
18    const [users, setUsers] = useState([])
19    const [tasks, setTasks] = useState([])
20    const [isLoading, setIsLoading] = useState(true)
21    const [loadError, setLoadError] = useState('')
22    const [loadAttempt, setLoadAttempt] = useState(0)
23    const [logoutError, setLogoutError] = useState('')
24    const [loggingOut, setLoggingOut] = useState(false)
25    const accountId = useRef(null)
26    accountId.current = account?.id
27
28    // Restore the HttpOnly-cookie session without storing credentials in the browser.
29    useEffect(() => {
30      const controller = new AbortController()
31      setCheckingSession(true)
32      setSessionError('')
33      api.getSession(controller.signal).then(user => {
34        if (!controller.signal.aborted) setAccount(user)
35      }).catch(error => {
36        if (!controller.signal.aborted) setSessionError(error.message)
37      }).finally(() => {
38        if (!controller.signal.aborted) setCheckingSession(false)
39      })
40      return () => controller.abort()
41    }, [sessionAttempt])
42
43    useEffect(() => {
44      function sessionExpired() {
45        accountId.current = null
46        setAccount(null)
47        setUsers([])
48        setTasks([])
49        setCurrentPage('dashboard')
50      }
51      window.addEventListener('session-expired', sessionExpired)
52      function passwordChanged() {
53        sessionExpired()
54        setSignInMessage('Your password has been changed. Please sign in again.')
55      }
56      window.addEventListener('password-changed', passwordChanged)
57      return () => {
```

frontend/src/App.jsx (continued)

8dea970 | Listing segment 2 of 4

```

58         window.removeEventListener('session-expired', sessionExpired)
59         window.removeEventListener('password-changed', passwordChanged)
60     }
61 }, [])
62
63 // The API filters regular users' tasks. Never request the Admin user list for them.
64 useEffect(() => {
65     if (!account) return
66     const controller = new AbortController()
67     setIsLoading(true)
68     setLoadError('')
69     Promise.all([
70         account.role === 'admin' ? api.getUsers(controller.signal) : Promise.resolve([
71             . account]),
72         api.getTasks(controller.signal),
73     ]).then(([savedUsers, savedTasks]) => {
74         if (!controller.signal.aborted) {
75             setUsers(savedUsers)
76             setTasks(savedTasks)
77         }
78     }).catch(error => {
79         if (!controller.signal.aborted) setLoadError(error.message)
80     }).finally(() => {
81         if (!controller.signal.aborted) setIsLoading(false)
82     })
83     return () => controller.abort()
84 // A profile edit updates the header without unmounting the form and losing its succe
85 s feedback.
86 }, [account?.id, account?.role, loadAttempt])
87
88 async function signIn(credentials) {
89     const user = await api.login(credentials)
90     setUsers([])
91     setTasks([])
92     setIsLoading(true)
93     setCurrentPage('dashboard')
94     setAccount(user)
95     setLogoutError('')
96     setSignInMessage('')
97 }
98
99 async function signOut() {
100     if (loggingOut) return
101     setLoggingOut(true)
102     setLogoutError('')
103     try {
104         await api.logout()
105         setAccount(null)
106         setUsers([])
107         setTasks([])
108         setCurrentPage('dashboard')
109     } catch (error) { setLogoutError(error.message) }
110     finally { setLoggingOut(false) }
111 }
112
113 // Ignore late mutation responses after logout or switching to another account.
114 async function createUser(user) {

```

frontend/src/App.jsx (continued)

8dea970 | Listing segment 3 of 4

```

113     const owner = account.id
114     const saved = await api.createUser(user)
115     if (accountId.current === owner) setUsers(current => [...current, saved])
116   }
117
118   async function createTask(task) {
119     const owner = account.id
120     const saved = await api.createTask(task)
121     if (accountId.current === owner) setTasks(current => [...current, saved])
122   }
123
124   async function changeTaskStatus(id, status) {
125     const owner = account.id
126     const saved = await api.updateTaskStatus(id, status)
127     if (accountId.current === owner) setTasks(current => current.map(task => task.id =
128       . == id ? saved : task))
129     return saved
130   }
131
132   async function saveProfile(fields) {
133     const owner = account.id
134     const saved = await api.updateProfile(fields)
135     if (accountId.current !== owner) return
136     setAccount(saved)
137     setUsers(current => current.map(user => user.id === owner ? saved : user))
138   }
139
140   const isAdmin = account?.role === 'admin'
141   async function updateUser(id, fields) {
142     const owner = account.id
143     const saved = await api.updateUser(id, fields)
144     if (accountId.current !== owner) return
145     setUsers(current => current.map(user => user.id === id ? saved : user))
146     if (id === owner) setAccount(saved)
147   }
148   async function statusUser(id, active) {
149     const owner = account.id
150     const saved = await api.updateUserStatus(id, active)
151     if (accountId.current === owner) setUsers(current => current.map(user => user.id =
152       . == id ? saved : user))
153   }
154   async function deleteUser(id) {
155     const owner = account.id
156     await api.deleteUser(id)
157     if (accountId.current === owner) setUsers(current => current.filter(user => user.i
158       . d !== id))
159   }
160   async function updateTask(id, fields) {
161     const owner = account.id
162     const saved = await api.updateTask(id, fields)
163     if (accountId.current === owner) setTasks(current => current.map(task => task.id =
164       . == id ? saved : task))
165   }
166   async function deleteTask(id) {
167     const owner = account.id
168     await api.deleteTask(id)
169     if (accountId.current === owner) setTasks(current => current.filter(task => task.i

```

frontend/src/App.jsx (continued)

8dea970 | Listing segment 4 of 4

```

    d !== id))
166   }
167   // Role-based rendering improves UX; Express authorization is the actual security bound-
    dary.
168   function renderPage() {
169     if (currentPage === 'users' && isAdmin) return <Users users={users} account={accou-
    nt} onCreateUser={createUser}
170       onUpdateUser={updateUser} onStatusUser={statusUser} onDeleteUser={deleteUser}
    />
171     if (currentPage === 'profile') return <Profile account={account} onSave={saveProfi-
    le} onPasswordChange={api.changePassword} />
172     if (currentPage === 'tasks' && isAdmin) return <AdminTasks tasks={tasks} users={us-
    ers}
173       onCreateTask={createTask} onUpdateTask={updateTask} onDeleteTask={deleteTask}
    />
174     if (currentPage === 'tasks') return <Tasks tasks={tasks} onChangeTaskStatus={chang-
    eTaskStatus} />
175     return <Dashboard isAdmin={isAdmin} users={users} tasks={tasks} />
176   }
177
178   const pages = isAdmin
179     ? [['dashboard', 'Dashboard'], ['users', 'User Management'], ['tasks', 'Task Manag-
    ement']]
180     : [['dashboard', 'Dashboard'], ['tasks', 'My Tasks'], ['profile', 'Profile']]
181
182   return (
183     <div className="app">
184       <header className="app-header">
185         <div><h1>My Web App V2</h1><span>Work Tracker{account ? ` · ${account.name
    } (${account.role})` : ''}</span></div>
186         {account && <nav aria-label="Main navigation">
187           {pages.map(([page, label]) => <button key={page}
188             aria-current={currentPage === page ? 'page' : undefined}
189             className={currentPage === page ? 'nav-active' : ''}
190             onClick={() => setCurrentPage(page)}>{label}</button>)}
191         <button onClick={signOut} disabled={loggingOut}>{loggingOut ? 'Signing
    out...' : 'Logout'}</button>
192       </nav>
193     </header>
194     <main className="container">
195       {logoutError && <p role="alert" className="message message-error">{logoutE-
    rror}</p>}
196       <DataState isLoading={checkingSession} error={sessionError}
197         onRetry={() => setSessionAttempt(value => value + 1)}>
198         {!account ? <><p className="message-success" role="status">{signInMess-
    age}</p><Login onLogin={signIn} /></> :
199         <DataState isLoading={isLoading} error={loadError}
200         onRetry={() => setLoadAttempt(value => value + 1)}>{renderPage
    ()}</DataState>
201       </DataState>
202     </main>
203     <footer className="app-footer"><p>&copy; 2026 My Web App V2</p></footer>
204   </div>
205 )
206 }
207
208 export default App

```

frontend/src/components/ConfirmDelete.jsx

Frontend application and views | 10 source lines | 525 bytes

SHA-256: c8a728fc3a9339d66bd4f98a5770ea795074e67f34d3a6bdb9ae84a037fa4db8

```
1  function ConfirmDelete({ name, busy, onConfirm, onCancel }) {
2      return <div className="confirmation" role="group" aria-label={`Confirm deletion of ${name}`}>
3          <p>Delete <strong>{name}</strong>? This cannot be undone.</p>
4          <div className="actions">
5              <button className="danger" disabled={busy} onClick={onConfirm}>{busy ? 'Delete' : 'Confirm Delete'}</button>
6              <button disabled={busy} onClick={onCancel}>Cancel Delete</button>
7          </div>
8      </div>
9  }
10 export default ConfirmDelete
```

frontend/src/components/DataState.jsx

Frontend application and views | 13 source lines | 538 bytes

SHA-256: 74756c771f205a14ccbaae9e6f95961396b45c453e98bbb49c7832cc1719bbb3

```
1 // Keep failed loading separate from empty data, and let the user retry explicitly.
2 function DataState({ isLoading = false, error = '', onRetry, children }) {
3   if (isLoading) return <p className="card" role="status">Loading work tracker data...</p>
4   if (error) return (
5     <div className="card">
6       <p className="message message-error" role="alert">{error}</p>
7       {onRetry && <button type="button" onClick={onRetry}>Retry loading</button>}
8     </div>
9   )
10   return children
11 }
12
13 export default DataState
```

frontend/src/components/PriorityBadge.jsx

Frontend application and views | 5 source lines | 174 bytes

SHA-256: d7e62f0469291558bae674f1afbafcb4b16ceb552c78dd5acf65c16811d1dfe

```
1 function PriorityBadge({ priority }) {  
2   return <span className={`priority priority-${priority.toLowerCase()}`}>Priority: {prio  
3   rity}</span>  
4 }  
5 export default PriorityBadge
```

frontend/src/components/StatusBadge.jsx

Frontend application and views | 13 source lines | 256 bytes

SHA-256: 0acc392daadf804f5a373d67389f6012a80c7142d95bf76d0574d239b6a8a710

```
1  function StatusBadge({ status }) {  
2      const className = `status status-${status  
3          .toLowerCase()  
4          .replace(' ', '-')}`  
5  
6      return (  
7          <span className={className}>  
8              {status}  
9          </span>  
10     )  
11 }  
12  
13 export default StatusBadge
```

frontend/src/components/TaskForm.jsx

Frontend application and views | 152 source lines | 5,972 bytes

SHA-256: 7ae5074bb50d3f89497e3c946a51d84c835e9aa481e97e4fa6e4d2201477186c

```
1 import { useState } from 'react'
2 import { taskPriorities, taskStatuses } from '../data/taskOptions.js'
3
4 function TaskForm({ users, onCreateTask, task, onCancel }) {
5   const [title, setTitle] = useState(task?.title || '')
6   const [description, setDescription] = useState(task?.description || '')
7   const [priority, setPriority] = useState(task?.priority || 'Medium')
8   const [userId, setUserId] = useState(task?.userId ? String(task.userId) : '')
9   const [status, setStatus] = useState(task?.status || 'Open')
10
11   const [error, setError] = useState('')
12   const [success, setSuccess] = useState('')
13   const [isSubmitting, setIsSubmitting] = useState(false)
14
15   // Select values are strings; resolve the record to send its numeric database ID.
16   async function handleSubmit(event) {
17     event.preventDefault()
18
19     if (isSubmitting) return
20     setError('')
21     setSuccess('')
22     if (!title.trim()) {
23       setError('Enter a task title containing more than spaces.')
24       return
25     }
26     const assignedUser = users.find(user => String(user.id) === userId)
27     if (userId && !assignedUser) {
28       setError('Choose an available user or leave the task unassigned.')
29       return
30     }
31     if (!taskPriorities.includes(priority)) {
32       setError('Choose a valid priority.')
33       return
34     }
35     setIsSubmitting(true)
36     try {
37       await onCreateTask({
38         title: title.trim(),
39         description: description.trim(),
40         priority,
41         status,
42         userId: assignedUser ? assignedUser.id : null,
43       })
44
45       if (task) return
46       setStatus('Open')
47       setTitle('')
48       setDescription('')
49       setPriority('Medium')
50       setUserId('')
51       setSuccess('Task created.')
52     } catch (error) {
53       setError(error.message || 'Unable to create task. Please try again.')
54     } finally {
55     }
```

frontend/src/components/TaskForm.jsx (continued)

8dea970 | Listing segment 2 of 3

```

55         setIsSubmitting(false)
56     }
57 }
58
59 return (
60     <form className="form" onSubmit={handleSubmit}>
61         <h3>{task ? 'Edit Task' : 'Create Task'}</h3>
62         <p className="form-hint">Title is required. Choose status, priority, and an op
        . tional assignment.</p>
63         {error && <p className="message message-error" role="alert">{error}</p>}
64         {success && <p className="message message-success" role="status">{success}</p>}
        .
65     <fieldset disabled={isSubmitting}>
66
67         <div className="form-group">
68             <label htmlFor="task-title">Title</label>
69             <input
70                 id="task-title"
71                 type="text"
72                 maxLength={200}
73                 value={title}
74                 onChange={event => setTitle(event.target.value)}
75                 required
76             />
77         </div>
78
79         <div className="form-group">
80             <label htmlFor="task-description">
81                 Description
82             </label>
83
84             <textarea
85                 id="task-description"
86                 maxLength={10000}
87                 value={description}
88                 onChange={event =>
89                     setDescription(event.target.value)
90                 }
91             />
92         </div>
93
94         <div className="form-group">
95             <label htmlFor="task-priority">
96                 Priority
97             </label>
98
99             <select
100                 id="task-priority"
101                 value={priority}
102                 onChange={event =>
103                     setPriority(event.target.value)
104                 }
105             >
106                 {taskPriorities.map(value => <option key={value}>{value}</option>)}

```

frontend/src/components/TaskForm.jsx (continued)

8dea970 | Listing segment 3 of 3

```

    }
107         </select>
108     </div>
109
110     <div className="form-group"><label htmlFor="task-status">Task status</label>
111     <select id="task-status" value={status} onChange={event => setStatus(event.target.value)}>
112         {taskStatuses.map(value => <option key={value}>{value}</option>)}
113     </select></div>
114     <div className="form-group">
115         <label htmlFor="task-user">
116             Assigned User
117         </label>
118
119         <select
120             id="task-user"
121             aria-describedby="assignment-hint"
122             value={userId}
123             onChange={event =>
124                 setUserId(event.target.value)
125             }
126         >
127             <option value="">
128                 Unassigned
129             </option>
130
131             {users.filter(user => user.active !== false || user.id === task?.userId).map(user => (
132                 <option
133                     key={user.id}
134                     value={user.id}
135                 >
136                     {user.name}{user.active === false ? ' (inactive; existing
137                     assignment)' : ''}
138                 </option>
139             ))}
140         </select>
141     </div>
142
143     <p id="assignment-hint" className="form-hint">{users.length === 0 ? 'Add a
144     user on the Users page to assign tasks.' : 'Assignment is optional.'}</p>
145     <button type="submit">
146         {isSubmitting ? 'Saving...' : task ? 'Save Task' : 'Create Task'}
147     </button>
148     {onCancel && <button type="button" className="secondary" onClick={onCancel}>Cancel</button>}
149 </fieldset>
150 </form>
151 )
152 }
153
154 export default TaskForm

```

frontend/src/components/UserForm.jsx

Frontend application and views | 101 source lines | 4,382 bytes

SHA-256: 7570a6ee9412cc979304f95b418b201a7b474761b0701cdb47d1b558a81bd479

```

1  import { useState } from 'react'
2
3  function UserForm({ onCreateUser, user, onCancel }) {
4      const [name, setName] = useState(user?.name || '')
5      const [email, setEmail] = useState(user?.email || '')
6      const [password, setPassword] = useState('')
7      const [role, setRole] = useState(user?.role || 'user')
8
9
10     const [error, setError] = useState('')
11     const [success, setSuccess] = useState('')
12     const [isSubmitting, setIsSubmitting] = useState(false)
13
14     // Preserve entered values on failure and clear the form only after a successful save.
15     async function handleSubmit(event) {
16         event.preventDefault()
17
18         if (isSubmitting) return
19         setError('')
20         setSuccess('')
21         // HTML required fields still allow spaces; reject them before saving.
22         if (!name.trim()) {
23             setError('Enter a name containing more than spaces.')
24             return
25         }
26         if (!user && (password.length < 12 || new TextEncoder().encode(password).length >
27             72)) {
28             setError('Password must be at least 12 characters and at most 72 UTF-8 bytes.')
29             return
30         }
31         setIsSubmitting(true)
32         try {
33             await onCreateUser({ name: name.trim(), email: email.trim().toLowerCase(), ...
34                 (user ? {} : { password }), role })
35             if (user) return
36             setName('')
37             setEmail('')
38             setPassword('')
39             setRole('user')
40             setSuccess('User added. They are now available for task assignment.')
41         } catch (error) {
42             setError(error.message || 'Unable to add user. Please try again.')
43         } finally {
44             setIsSubmitting(false)
45         }
46     }
47
48     return (
49         <form className="form" onSubmit={handleSubmit}>
50             <h3>{user ? 'Edit User' : 'Add User'}</h3>
51             <p className="form-hint">{user ? 'Update account information. Passwords are ma
52                 naged separately.' : 'All fields are required. Share the initial password privately with t
53                 he account owner.'}</p>
54             {error && <p className="message message-error" role="alert">{error}</p>}
55             {success && <p className="message message-success" role="status">{success}</p>}
56         </form>
57     )
58 }

```

frontend/src/components/UserForm.jsx (continued)

8dea970 | Listing segment 2 of 2

```

51         <fieldset disabled={isSubmitting}>
52
53             <div className="form-group">
54                 <label htmlFor="user-name">Name</label>
55                 <input
56                     id="user-name"
57                     type="text"
58                     maxLength={100}
59                     autoComplete="name"
60                     value={name}
61                     onChange={event => setName(event.target.value)}
62                     required
63                 />
64             </div>
65
66             <div className="form-group">
67                 <label htmlFor="user-email">Email</label>
68                 <input
69                     id="user-email"
70                     type="email"
71                     maxLength={255}
72                     autoComplete="email"
73                     value={email}
74                     onChange={event => setEmail(event.target.value)}
75                     required
76                 />
77             </div>
78
79             {!user && <div className="form-group">
80                 <label htmlFor="user-password">Initial password</label>
81                 <input id="user-password" type="password" autoComplete="new-password"
82                     value={password} onChange={event => setPassword(event.target.value
83 . }} minLength={12} required />
84                 <span className="form-hint">At least 12 characters; maximum 72 UTF-8 b
85 . ytes.</span>
86             </div>}
87             <div className="form-group">
88                 <label htmlFor="user-role">Role</label>
89                 <select id="user-role" value={role} onChange={event => setRole(event.t
90 . arget.value)}>
91                     <option value="user">User</option>
92                     <option value="admin">Admin</option>
93                 </select>
94             </div>
95             <button type="submit">
96                 {isSubmitting ? 'Saving...' : user ? 'Save User' : 'Add User'}
97             </button>
98             {onCancel && <button type="button" className="secondary" onClick={onCancel}>Ca
99 . ncel</button>}
100         </fieldset>
101     </form>
102 )
103 }
104
105 export default UserForm

```

frontend/src/data/taskOptions.js

Frontend application and views | 3 source lines | 197 bytes

SHA-256: 21e05b1d3404ec05e03773b37d72a1333f0a4903c583fe1874dc2acbede2e196

```
1 // Keep form options and frontend validation aligned with the API's values.  
2 export const taskStatuses = ['Open', 'In Progress', 'Completed']  
3 export const taskPriorities = ['Low', 'Medium', 'High']
```

frontend/src/index.css

Frontend application and views | 300 source lines | 6,157 bytes

SHA-256: 5dc713dbd61b95f271f47a85bbb11759ebee390e9ebdf2d3a25b10d2e7a57cb2

```
1  * {
2    box-sizing: border-box;
3  }
4
5  body {
6    margin: 0;
7    font-family: Arial, sans-serif;
8    line-height: 1.5;
9    background: #f5f6f8;
10   color: #202124;
11 }
12
13 button,
14 input,
15 textarea,
16 select {
17   font: inherit;
18 }
19
20 button {
21   cursor: pointer;
22 }
23
24 .app {
25   min-height: 100vh;
26   display: flex;
27   flex-direction: column;
28 }
29
30 .app-header {
31   background: #ffffff;
32   border-bottom: 1px solid #dddddd;
33   padding: 20px 24px;
34   display: flex;
35   justify-content: space-between;
36   align-items: center;
37   gap: 24px;
38 }
39
40 .app-header h1 {
41   margin: 0;
42   font-size: 24px;
43 }
44
45 .app-header span {
46   color: #666666;
47 }
48
49 nav {
50   display: flex;
51   gap: 8px;
```

frontend/src/index.css (continued)

8dea970 | Listing segment 2 of 6

```
52  }
53
54  nav button {
55    border: 0;
56    background: transparent;
57    padding: 10px 14px;
58    border-radius: 6px;
59  }
60
61  nav button:hover,
62  .nav-active {
63    background: #e8e8e8;
64  }
65
66  .container {
67    width: 100%;
68    max-width: 1200px;
69    margin: 0 auto;
70    padding: 32px 24px;
71    flex: 1;
72  }
73
74  .page-header {
75    margin-bottom: 24px;
76  }
77
78  .page-header h2 {
79    margin-bottom: 4px;
80  }
81
82  .page-header p {
83    margin: 0;
84    color: #666666;
85  }
86
87  .stats-grid {
88    display: grid;
89    grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
90    gap: 16px;
91  }
92
93  .content-grid {
94    display: grid;
95    grid-template-columns: minmax(280px, 400px) 1fr;
96    gap: 24px;
97  }
98
99  .card,
100  .form {
101    background: #ffffff;
102    border: 1px solid #dddddd;
```

frontend/src/index.css (continued)

8dea970 | Listing segment 3 of 6

```
103     border-radius: 8px;
104     padding: 20px;
105 }
106
107 .card h3,
108 .form h3 {
109     margin-top: 0;
110 }
111
112 .stats-grid strong {
113     font-size: 32px;
114 }
115
116 .form {
117     margin-bottom: 24px;
118 }
119
120 .form-group {
121     display: flex;
122     flex-direction: column;
123     gap: 6px;
124     margin-bottom: 16px;
125 }
126
127 input,
128 textarea,
129 select {
130     width: 100%;
131     padding: 10px;
132     border: 1px solid #cccccc;
133     border-radius: 5px;
134 }
135
136 textarea {
137     min-height: 100px;
138     resize: vertical;
139 }
140
141 .form button {
142     border: 0;
143     border-radius: 5px;
144     padding: 10px 16px;
145 }
146
147 .list {
148     display: flex;
149     flex-direction: column;
150 }
151
152 .list-item {
153     padding: 14px 0;
```

frontend/src/index.css (continued)

8dea970 | Listing segment 4 of 6

```
154     border-bottom: 1px solid #eeeeee;
155     display: flex;
156     flex-direction: column;
157   }
158
159   .list-item:last-child {
160     border-bottom: 0;
161   }
162
163   .list-item span {
164     color: #666666;
165   }
166
167   .task-list {
168     display: grid;
169     gap: 16px;
170   }
171
172   .task-header {
173     display: flex;
174     justify-content: space-between;
175     gap: 16px;
176   }
177
178   .task-header h3 {
179     margin-bottom: 4px;
180   }
181
182   .task-header p {
183     margin-top: 0;
184     color: #666666;
185   }
186
187   .task-meta {
188     display: flex;
189     gap: 24px;
190     margin: 16px 0;
191     color: #555555;
192   }
193
194   .task-actions {
195     max-width: 220px;
196   }
197
198   .status {
199     height: fit-content;
200     padding: 4px 10px;
201     border-radius: 20px;
202     background: #eeeeee;
203     white-space: nowrap;
204   }
```

frontend/src/index.css (continued)

8dea970 | Listing segment 5 of 6

```
205
206 .status-open {
207     background: #fff3cd;
208 }
209
210 .status-in-progress {
211     background: #cfe2ff;
212 }
213
214 .status-completed {
215     background: #d1e7dd;
216 }
217
218 .empty-state {
219     color: #666666;
220 }
221
222 .app-footer {
223     padding: 20px 24px;
224     text-align: center;
225     border-top: 1px solid #dddddd;
226     background: #ffffff;
227 }
228
229 @media (max-width: 700px) {
230     .app-header {
231         flex-direction: column;
232         align-items: flex-start;
233     }
234
235     nav {
236         width: 100%;
237         overflow-x: auto;
238     }
239
240     .content-grid {
241         grid-template-columns: 1fr;
242     }
243
244     .task-header,
245     .task-meta {
246         flex-direction: column;
247     }
248 }
249
250 /* Shared feedback and controls keep every page consistent and keyboard accessible. */
251 fieldset { border: 0; padding: 0; margin: 0; min-width: 0; }
252 button, select { min-height: 44px; }
253 button:focus-visible, input:focus-visible, textarea:focus-visible, select:focus-visible {
254     outline: 3px solid #275dad; outline-offset: 3px;
255 }
```

frontend/src/index.css (continued)

8dea970 | Listing segment 6 of 6

```

256 button:disabled, select:disabled { cursor: wait; opacity: .65; }
257 .form button { background: #275dad; color: white; }
258 .form button:hover:not(:disabled) { background: #1c467f; }
259 .form-hint, .demo-notice { color: #555; font-size: .9rem; }
260 .demo-notice { margin: 0 0 24px; }
261 .message { padding: 12px 16px; border-radius: 6px; overflow-wrap: anywhere; }
262 .message-error { background: #fce8e6; color: #8b1a15; }
263 .message-success { background: #e3f2e7; color: #20552d; }
264 .status-feedback { min-height: 1.5em; }
265 .priority { padding: 4px 10px; border-radius: 6px; width: fit-content; }
266 .priority-high { background: #fce8e6; color: #8b1a15; }
267 .priority-medium { background: #fff3cd; color: #654d00; }
268 .priority-low { background: #e3f2e7; color: #20552d; }
269 /* Long user content must wrap instead of widening the page on small screens. */
270 .card, .form, .task-header > div { min-width: 0; overflow-wrap: anywhere; }
271 .task-header p { white-space: pre-wrap; }
272 .task-meta { align-items: flex-start; flex-wrap: wrap; }
273 @media (max-width: 400px) {
274   .container { padding: 24px 16px; }
275   .app-header { padding: 16px; }
276   .card, .form { padding: 16px; }
277 }
278
279 .login-form { max-width: 440px; margin: 24px auto; }
280 /* Admin management keeps the existing cards and adds consistent action groups. */
281 .management button, .detail-panel button { border: 1px solid #b8c1ce; border-radius: 5px;
. padding: 8px 14px; background: white; color: #202124; }
282 .management .form button[type=submit] { background: #275dad; color: white; }
283 .management button.danger { color: #8b1a15; border-color: #c68985; }
284 .actions { display: flex; flex-wrap: wrap; gap: 8px; }
285 .filters { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); gap
: 12px; margin-top: 24px; }
286 .filters label { display: flex; flex-direction: column; gap: 6px; }
287 .management-list { display: grid; gap: 16px; }
288 .confirmation { margin-top: 16px; padding: 16px; border: 1px solid #c68985; border-radius:
6px; background: #fff5f4; }
289 .detail-panel, .workload { margin-top: 24px; margin-bottom: 24px; }
290 .description { white-space: pre-wrap; }
291 .form .secondary { margin-left: 8px; }
292 /* Personal account forms stay separate while sharing the existing visual language. */
293 .profile-details { margin-bottom: 24px; }
294 .profile-details dl { display: grid; grid-template-columns: minmax(90px, 130px) 1fr; gap:
8px 16px; }
295 .profile-details dt { font-weight: bold; }
296 .profile-details dd { margin: 0; overflow-wrap: anywhere; }
297 .profile-forms { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 24p
x; align-items: start; }
298 .personal-task-actions { align-items: end; }
299 .personal-task-actions label { min-width: 180px; }
300 @media (max-width: 700px) { .profile-forms { grid-template-columns: 1fr; } }

```

frontend/src/main.jsx

Frontend application and views | 10 source lines | 242 bytes

SHA-256: 6315ec8facf24654683fa69275f0c3001984656d7632cd1431954f814ed6445d

```
1  import React from 'react'
2  import ReactDOM from 'react-dom/client'
3  import App from './App.jsx'
4  import './index.css'
5
6  ReactDOM.createRoot(document.getElementById('root')).render(
7    <React.StrictMode>
8      <App />
9    </React.StrictMode>,
10 )
```

frontend/src/pages/AdminTasks.jsx

Frontend application and views | 64 source lines | 5,187 bytes

SHA-256: f1052e05cecb9aa8350f2fe55bcde128f5faa56d5288163c33f554147db91b4c

```

1  import { useState } from 'react'
2  import TaskForm from '../components/TaskForm.jsx'
3  import ConfirmDelete from '../components/ConfirmDelete.jsx'
4  import PriorityBadge from '../components/PriorityBadge.jsx'
5  import StatusBadge from '../components/StatusBadge.jsx'
6  import { taskPriorities, taskStatuses } from '../data/taskOptions.js'
7  import { filterTasks, formatDate } from '../services/management.js'
8  import * as api from '../services/api.js'
9
10 function AdminTasks({ tasks, users, onCreateTask, onUpdateTask, onDeleteTask }) {
11   const [filters, setFilters] = useState({ search: '', status: '', priority: '', userId:
.   '', sort: 'newest' })
12   const [panel, setPanel] = useState(null)
13   const [busy, setBusy] = useState(false)
14   const [error, setError] = useState('')
15   const [message, setMessage] = useState('')
16   const visible = filterTasks(tasks, filters)
17   const filter = (key, value) => setFilters(current => ({ ...current, [key]: value }))
18   const assignee = id => users.find(user => user.id === id)?.name || 'Unassigned'
19   async function perform(work, success) {
20     setBusy(true); setError(''); setMessage('')
21     try { await work(); setMessage(success) }
22     catch (error) { setError(error.message) }
23     finally { setBusy(false) }
24   }
25   return <div className="management">
26     <div className="page-header"><h2>Task Management</h2><p>Plan, assign, and maintain
.     application work.</p></div>
27     {error && <p role="alert" className="message message-error">{error}</p>}
28     <p role="status">{busy ? 'Working...' : message}</p>
29     <TaskForm key={panel?.mode === 'edit' ? panel.task.id : 'create'} users={users}
30       task={panel?.mode === 'edit' ? panel.task : undefined}
31       onCancel={panel?.mode === 'edit' ? () => setPanel(null) : undefined}
32       onCreateTask={async fields => {
33         if (panel?.mode === 'edit') { await onUpdateTask(panel.task.id, fields); s
.         etPanel(null); setMessage('Task updated.') }
34         else await onCreateTask(fields)
35       } } />
36     {panel?.mode === 'view' && <section className="card detail-panel" aria-label="Task
.     details">
37       <h3>{panel.task.title}</h3><p className="description">{panel.task.description
.       || 'No description'}</p>
38       <p>{panel.task.status} · {panel.task.priority} priority · Assigned: {assignee(
.       panel.task.userId)}</p>
39       <p>Created: {formatDate(panel.task.createdAt)}<br />Updated: {formatDate(panel

```

frontend/src/pages/AdminTasks.jsx (continued)

8dea970 | Listing segment 2 of 2

```

.      .task.updatedAt))</p>
40      <button onClick={() => setPanel(null)}>Close Details</button>
41    </section>
42    <div className="filters">
43      <label>Search tasks<input value={filters.search} onChange={e => filter('search
.      ', e.target.value)} placeholder="Task title" /></label>
44      <label>Filter status<select value={filters.status} onChange={e => filter('stat
.      us', e.target.value)}><option value="">All statuses</option>{taskStatuses.map(value => <op
.      tion key={value}>{value}</option>)}</select></label>
45      <label>Filter priority<select value={filters.priority} onChange={e => filter('
.      priority', e.target.value)}><option value="">All priorities</option>{taskPriorities.map(va
.      lue => <option key={value}>{value}</option>)}</select></label>
46      <label>Filter assignee<select value={filters.userId} onChange={e => filter('us
.      erId', e.target.value)}><option value="">All assignees</option><option value="unassigned">
.      Unassigned</option>{users.map(user => <option key={user.id} value={user.id}>{user.name}</o
.      ption>)}</select></label>
47      <label>Sort tasks<select value={filters.sort} onChange={e => filter('sort', e.
.      target.value)}><option value="newest">Newest first</option><option value="title">Title A-Z
.      </option><option value="priority">Highest priority</option></select></label>
48    </div>
49    <p>{visible.length} of {tasks.length} tasks</p>
50    {!visible.length && <p className="card empty-state">No tasks match these filters.<
.    /p>}
51    <div className="management-list">{visible.map(task => <article key={task.id} class
.    Name="card" aria-label={`Task ${task.title}`}>
52      <div className="task-header"><h3>{task.title}</h3><StatusBadge status={task.st
.    atus} /></div>
53      <p className="description">{task.description || 'No description'}</p>
54      <div className="task-meta"><PriorityBadge priority={task.priority} /><span>Ass
.    igned: {assignee(task.userId)}</span></div>
55      <div className="actions">
56        {[ 'view', 'edit' ].map(mode => <button key={mode} disabled={busy} onClick={
.        () => perform(async () => setPanel({ mode, task: await api.getTask(task.id) }), '')}>{mode
.        === 'view' ? 'View' : 'Edit'}</button>)}
57        <button className="danger" disabled={busy} onClick={() => { setPanel({ mod
.        e: 'delete', task }); setError('') }}>Delete</button>
58      </div>
59      {panel?.mode === 'delete' && panel.task.id === task.id && <ConfirmDelete name=
.      {task.title} busy={busy} onCancel={() => setPanel(null)}
60        onConfirm={() => perform(async () => { await onDeleteTask(task.id); setPan
.      el(null) }, 'Task deleted.')} />}
61    </article>)}</div>
62  </div>
63  }
64  export default AdminTasks

```

frontend/src/pages/Dashboard.jsx

Frontend application and views | 83 source lines | 4,136 bytes

SHA-256: 1743fa1655ef92e76d046a5f0fd4eba676d82e40f2ff63396f52c5507078cf27

```

1  import DataState from '../components/DataState.jsx'
2  import { personalSummary } from '../services/personal.js'
3  function Dashboard({ users, tasks, isAdmin = true, isLoading = false, error = '' }) {
4      const personal = !isAdmin ? personalSummary(tasks) : null
5      // Counts derive from shared state, so creating tasks and changing status update the o
6      .
7      verview.
8      const openTasks = tasks.filter(
9          task => task.status === 'Open'
10     ).length
11
12     const inProgressTasks = tasks.filter(
13         task => task.status === 'In Progress'
14     ).length
15
16     const completedTasks = tasks.filter(
17         task => task.status === 'Completed'
18     ).length
19
20     return (
21         <DataState isLoading={isLoading} error={error}>
22             <div>
23                 <div className="page-header">
24                     <div>
25                         <h2>{isAdmin ? 'Admin Dashboard' : 'My Dashboard'}</h2>
26                         <p>
27                             {isAdmin ? 'Application overview and current workload.' : 'You
28                             r assigned workload.'}
29                         </p>
30                     </div>
31                 </div>
32                 <div>
33                     <div className="stats-grid">
34                         {isAdmin && <div className="card">
35                             <h3>Total Users</h3>
36                             <strong>{users.length}</strong>
37                         </div>}
38
39                         <div className="card">
40                             <h3>{isAdmin ? 'Open' : 'My Open Tasks'}</h3>
41                             <strong>{openTasks}</strong>
42                         </div>
43
44                         <div className="card">
45                             <h3>{isAdmin ? 'In Progress' : 'My In-Progress Tasks'}</h3>
46                             <strong>{inProgressTasks}</strong>
47                         </div>
48
49                         <div className="card">

```

frontend/src/pages/Dashboard.jsx (continued)

8dea970 | Listing segment 2 of 2

```

47         <h3>{isAdmin ? 'Completed' : 'My Completed Tasks'}</h3>
48         <strong>{completedTasks}</strong>
49     </div>
50 </div>
51     {isAdmin && <>
52         <div className="stats-grid workload">
53             <div className="card"><h3>Active Users</h3><strong>{users.filter(u
.   ser => user.active).length}</strong></div>
54             <div className="card"><h3>Inactive Users</h3><strong>{users.filter
.   (user => !user.active).length}</strong></div>
55             <div className="card"><h3>Unassigned Tasks</h3><strong>{tasks.filt
.   er(task => task.userId === null).length}</strong></div>
56         </div>
57         <section className="card workload"><h3>Workload by Priority</h3>
58         <p>All tasks, including completed work.</p>
59         <div className="stats-grid">{['High', 'Medium', 'Low'].map(priorit
.   y => <div key={priority}>
60             <span>{priority}</span><p><strong>{tasks.filter(task => task.p
.   riority === priority).length}</strong></p>
61             </div>)}</div>
62         </section>
63     </>
64     {!isAdmin && <>
65         <div className="stats-grid workload">
66             <div className="card"><h3>Total Assigned Tasks</h3><strong>{person
.   al.total}</strong></div>
67             {Object.entries(personal.priorities).map(([priority, count]) => <d
.   iv className="card" key={priority}>
68                 <h3>My {priority}-Priority Tasks</h3><strong>{count}</strong>
69                 </div>)}
70             </div>
71             <section className="card"><h3>Needs My Attention</h3>
72             <p>Up to three unfinished tasks, highest priority first. Priority
.   totals above include completed work.</p>
73             {personal.attention.length ? <ul>{personal.attention.map(task => <
.   li key={task.id}>
74                 <strong>{task.title}</strong> – {task.priority} priority · {ta
.   sk.status}
75                 </li>}</ul> : <p className="empty-state">No unfinished tasks are
.   assigned to you.</p>}
76             </section>
77         </>
78     </div>
79     </DataState>
80 )
81 }
82
83 export default Dashboard

```

frontend/src/pages/Login.jsx

Frontend application and views | 45 source lines | 1,838 bytes

SHA-256: 2a0f8cdb737317ceaa7f32c683a83a437d3a63940d03ccc65090d282d1774fa

```

1  import { useState } from 'react'
2
3  function Login({ onLogin }) {
4      const [email, setEmail] = useState('')
5      const [password, setPassword] = useState('')
6      const [error, setError] = useState('')
7      const [isSubmitting, setIsSubmitting] = useState(false)
8
9      async function handleSubmit(event) {
10         event.preventDefault()
11         if (isSubmitting) return
12         setIsSubmitting(true)
13         setError('')
14         try {
15             // Passwords exist only in this form while signing in, never localStorage or U
16             . RLs.
17             await onLogin({ email: email.trim(), password })
18             setPassword('')
19         } catch (error) {
20             setError(error.message)
21         } finally { setIsSubmitting(false) }
22     }
23
24     return (
25         <form className="form login-form" onSubmit={handleSubmit}>
26             <h2>Sign In</h2>
27             <p>Use the account provided by your Admin.</p>
28             {error && <p className="message message-error" role="alert">{error}</p>}
29             <fieldset disabled={isSubmitting}>
30                 <div className="form-group">
31                     <label htmlFor="login-email">Email</label>
32                     <input id="login-email" type="email" autoComplete="username" required
33                         value={email} onChange={event => setEmail(event.target.value)} />
34                 </div>
35                 <div className="form-group">
36                     <label htmlFor="login-password">Password</label>
37                     <input id="login-password" type="password" autoComplete="current-passw
38                         ord" required
39                         value={password} onChange={event => setPassword(event.target.value
40                             )} />
41                 </div>
42                 <button type="submit">{isSubmitting ? 'Signing in...' : 'Sign In'}</button>
43             </fieldset>
44         </form>
45     )
46 }
47
48 export default Login

```

frontend/src/pages/Profile.jsx

Frontend application and views | 86 source lines | 5,169 bytes

SHA-256: df646d0ffc2f842594f200532b1d9219dee5bacdfb3ed96e61c0f72886a0a5b0

```

1  import { useState } from 'react'
2  import { formatDate } from '../services/management.js'
3  import { passwordError } from '../services/personal.js'
4
5  function Profile({ account, onSave, onPasswordChange }) {
6      const [name, setName] = useState(account.name)
7      const [email, setEmail] = useState(account.email)
8      const [currentPassword, setCurrentPassword] = useState('')
9      const [newPassword, setNewPassword] = useState('')
10     const [confirmPassword, setConfirmPassword] = useState('')
11     const [busy, setBusy] = useState('')
12     const [error, setError] = useState('')
13     const [passwordMessage, setPasswordMessage] = useState('')
14     const [success, setSuccess] = useState('')
15
16     async function save(event) {
17         event.preventDefault()
18         if (busy) return
19         setError(''); setSuccess('')
20         if (!name.trim()) { setError('Enter a name containing more than spaces.');
```

frontend/src/pages/Profile.jsx (continued)

8dea970 | Listing segment 2 of 2

```

51         <dt>Role</dt><dd>{account.role}</dd>
52         <dt>Account status</dt><dd>{account.active ? 'Active' : 'Inactive'}</dd>
53         <dt>Created</dt><dd>{formatDate(account.createdAt)}</dd>
54         <dt>Updated</dt><dd>{formatDate(account.updatedAt)}</dd>
55     </dl>
56 </section>
57 <div className="profile-forms">
58     <form className="form" onSubmit={save}>
59         <h3>Edit My Profile</h3><p className="form-hint">Your role and account sta
60 tus are managed by an Admin.</p>
61         {error && <p className="message message-error" role="alert">{error}</p>}
62         {success && <p className="message message-success" role="status">{success}
63         </p>}
64         <fieldset disabled={!busy}>
65             <div className="form-group"><label htmlFor="profile-name">Name</label>
66             <input id="profile-name" autoComplete="name" required maxLength={1
67 00} value={name} onChange={e => setName(e.target.value)} /></div>
68             <div className="form-group"><label htmlFor="profile-email">Email</labe
69 1>
70             <input id="profile-email" type="email" autoComplete="email" requir
71 ed maxLength={255} value={email} onChange={e => setEmail(e.target.value)} /></div>
72             <button type="submit">{busy === 'profile' ? 'Saving...' : 'Save Profile'
73 }</button>
74         </fieldset>
75     </form>
76     <form className="form" onSubmit={changePassword}>
77         <h3>Change Password</h3><p className="form-hint">Use 12 or more characters
78 , up to 72 UTF-8 bytes. A successful change signs you out on every device.</p>
79         {passwordMessage && <p className="message message-error" role="alert">{pas
80 swordMessage}</p>}
81         <fieldset disabled={!busy}>
82             <div className="form-group"><label htmlFor="current-password">Current
83 Password</label>
84             <input id="current-password" type="password" autoComplete="current
85 -password" required value={currentPassword} onChange={e => setCurrentPassword(e.target.val
86 ue)} /></div>
87             <div className="form-group"><label htmlFor="new-password">New Password
88 </label>
89             <input id="new-password" type="password" autoComplete="new-passwor
90 d" required minLength={12} value={newPassword} onChange={e => setNewPassword(e.target.valu
91 e)} /></div>
92             <div className="form-group"><label htmlFor="confirm-password">Confirm
93 New Password</label>
94             <input id="confirm-password" type="password" autoComplete="new-pas
95 sword" required value={confirmPassword} onChange={e => setConfirmPassword(e.target.value)}
96 /></div>
97             <button type="submit">{busy === 'password' ? 'Changing password...' : 'C
98 hange Password'}</button>
99         </fieldset>
100     </form>
101 </div>
102 </div>
103 }
104 export default Profile

```

frontend/src/pages/Tasks.jsx

Frontend application and views | 67 source lines | 4,802 bytes

SHA-256: 83ff22dd5cf41e152b7c611bffa0c237614a1fb0c76e354e6c18b91afd6b2484

```

1  import { useState } from 'react'
2  import PriorityBadge from '../components/PriorityBadge.jsx'
3  import StatusBadge from '../components/StatusBadge.jsx'
4  import { taskPriorities, taskStatuses } from '../data/taskOptions.js'
5  import { formatDate } from '../services/management.js'
6  import { filterMyTasks } from '../services/personal.js'
7  import { getTask } from '../services/api.js'
8
9  function Tasks({ tasks, onChangeTaskStatus }) {
10     const [filters, setFilters] = useState({ search: '', status: '', priority: '', sort: '
    . newest' })
11     const [busy, setBusy] = useState(null)
12     const [detail, setDetail] = useState(null)
13     const [error, setError] = useState('')
14     const [message, setMessage] = useState('')
15     const visible = filterMyTasks(tasks, filters)
16     const filter = (key, value) => setFilters(current => ({ ...current, [key]: value }))
17
18     async function changeStatus(id, status) {
19         if (busy !== null) return
20         setBusy(id); setError(''); setMessage('')
21         try {
22             // Shared state changes only after the ownership-checked API confirms the save
    . d record.
23             const saved = await onChangeTaskStatus(id, status)
24             if (detail?.id === id) setDetail(saved)
25             setMessage('Task status updated. If it no longer matches your filters, it is h
    . idden from this list.')
26             } catch (error) { setError(error.message) }
27             finally { setBusy(null) }
28         }
29         async function view(id) {
30             if (busy !== null) return
31             setBusy(id); setError(''); setDetail(null)
32             try { setDetail(await getTask(id)) }
33             catch (error) { setError(error.message) }
34             finally { setBusy(null) }
35         }
36         return <div className="personal management">
37             <div className="page-header"><h2>My Tasks</h2><p>Only work assigned to you. You ca
    . n update status; other task changes are managed by an Admin.</p></div>
38             {error && <p role="alert" className="message message-error">{error}</p>}
39             <p role="status">{busy !== null ? 'Working...' : message}</p>
40             {detail && <section className="card detail-panel" aria-label="Task details">

```

frontend/src/pages/Tasks.jsx (continued)

8dea970 | Listing segment 2 of 2

```

41         <h3>{detail.title}</h3><p className="description">{detail.description || 'No d
.     escription'}</p>
42         <div className="task-meta"><StatusBadge status={detail.status} /><PriorityBadg
.     e priority={detail.priority} /></div>
43         <p>Created: {formatDate(detail.createdAt)}<br />Updated: {formatDate(detail.up
.     datedAt)}</p>
44         <button onClick={() => setDetail(null)}>Close Details</button>
45     </section>
46     <div className="filters">
47         <label>Search my tasks<input value={filters.search} onChange={e => filter('sea
.     rch', e.target.value)} placeholder="Title or description" /></label>
48         <label>Filter status<select value={filters.status} onChange={e => filter('stat
.     us', e.target.value)}><option value="">All statuses</option>{taskStatuses.map(value => <op
.     tion key={value}>{value}</option>)}</select></label>
49         <label>Filter priority<select value={filters.priority} onChange={e => filter('
.     priority', e.target.value)}><option value="">All priorities</option>{taskPriorities.map(va
.     lue => <option key={value}>{value}</option>)}</select></label>
50         <label>Sort my tasks<select value={filters.sort} onChange={e => filter('sort',
.     e.target.value)}><option value="newest">Newest first</option><option value="oldest">Oldest
.     first</option><option value="priority">Highest priority</option><option value="status">S
.     tatus: Open first</option></select></label>
51     </div>
52     <p>{visible.length} of {tasks.length} assigned tasks</p>
53     {!visible.length && <p className="card empty-state">{tasks.length ? 'No tasks matc
.     h these filters.' : 'No tasks are assigned to you yet.'}</p>}
54     <div className="task-list">{visible.map(task => <article className="card task-card
.     " key={task.id} aria-label={`Task ${task.title}`}>
55         <div className="task-header"><h3>{task.title}</h3><StatusBadge status={task.st
.     atus} /></div>
56         <p className="description">{task.description || 'No description'}</p>
57         <div className="task-meta"><PriorityBadge priority={task.priority} /><span>Cre
.     ated: {formatDate(task.createdAt)}<br />Updated: {formatDate(task.updatedAt)}</span></div>
58         <div className="actions personal-task-actions">
59             <button disabled={busy !== null} onClick={() => view(task.id)}>View Detail
.     s</button>
60             <label>Status<select aria-label={`Status for ${task.title}`} disabled={bus
.     y !== null} value={task.status} onChange={e => changeStatus(task.id, e.target.value)}>
61                 {taskStatuses.map(value => <option key={value}>{value}</option>)}
62             </select></label>
63         </div>
64     </article>)}</div>
65 </div>
66 }
67 export default Tasks

```

frontend/src/pages/Users.jsx

Frontend application and views | 66 source lines | 5,310 bytes

SHA-256: 3abb3863479818defebbc251b6bb6063f040283650a66729a365b2720b6f9ccf

```

1  import { useState } from 'react'
2  import UserForm from '../components/UserForm.jsx'
3  import ConfirmDelete from '../components/ConfirmDelete.jsx'
4  import * as api from '../services/api.js'
5  import { filterUsers, formatDate } from '../services/management.js'
6
7  function Users({ users, account, onCreateUser, onUpdateUser, onStatusUser, onDeleteUser })
8  {
9      const [filters, setFilters] = useState({ search: '', role: '', active: '', sort: 'name
10     ' })
11     const [panel, setPanel] = useState(null)
12     const [busy, setBusy] = useState(false)
13     const [error, setError] = useState('')
14     const [message, setMessage] = useState('')
15     const visible = filterUsers(users, filters)
16     const filter = (key, value) => setFilters(current => ({ ...current, [key]: value }))
17
18     async function perform(work, success) {
19         setBusy(true); setError(''); setMessage('')
20         try { await work(); setMessage(success) }
21         catch (error) { setError(error.message) }
22         finally { setBusy(false) }
23     }
24     async function open(user, mode) {
25         // Load current details before editing so a stale list is not the source of truth.
26         await perform(async () => setPanel({ mode, user: await api.getUser(user.id) }), ''
27     )
28     }
29     return <div className="management">
30         <div className="page-header"><h2>User Management</h2><p>Manage accounts, access, a
31         nd assigned workload.</p></div>
32         {error && <p role="alert" className="message message-error">{error}</p>}
33         <p role="status">{busy ? 'Working...' : message}</p>
34         {panel?.mode === 'edit' ? <UserForm key={panel.user.id} user={panel.user}
35         onCancel={() => setPanel(null)} onCreateUser={async fields => {
36             await onUpdateUser(panel.user.id, fields); setPanel(null); setMessage('Use
37             r updated.')
38         }} /> : <UserForm onCreateUser={onCreateUser} />}
39         {panel?.mode === 'view' && <section className="card detail-panel" aria-label="User
40         details">
41             <h3>{panel.user.name}</h3><p>{panel.user.email} · {panel.user.role} · {panel.u
42             ser.active ? 'Active' : 'Inactive'}</p>
43             <p>Created: {formatDate(panel.user.createdAt)}<br />Updated: {formatDate(panel
44             .user.updatedAt)}</p>
45             <p>Assigned tasks: {panel.user.taskSummary.total} · Open: {panel.user.taskSumm
46             ary.open} · In progress: {panel.user.taskSummary.inProgress} · Completed: {panel.user.task
47             Summary.completed}</p>
48             {panel.user.canLogin === false && <p>Login not enabled for this legacy account

```

frontend/src/pages/Users.jsx (continued)

8dea970 | Listing segment 2 of 2

```

.    .</p>}
39      <button onClick={() => setPanel(null)}>Close Details</button>
40    </section>}
41    <div className="filters">
42      <label>Search users<input value={filters.search} onChange={e => filter('search
.    ', e.target.value)} placeholder="Name or email" /></label>
43      <label>Filter role<select value={filters.role} onChange={e => filter('role', e
.    .target.value)}><option value="">All roles</option><option value="admin">Admin</option><op
.    tion value="user">User</option></select></label>
44      <label>Filter account status<select value={filters.active} onChange={e => filt
.    er('active', e.target.value)}><option value="">All statuses</option><option value="true">A
.    ctive</option><option value="false">Inactive</option></select></label>
45      <label>Sort users<select value={filters.sort} onChange={e => filter('sort', e.
.    target.value)}><option value="name">Name A-Z</option><option value="newest">Newest first</
.    option></select></label>
46    </div>
47    <p>{visible.length} of {users.length} users</p>
48    {!visible.length && <p className="card empty-state">No users match these filters.<
.    /p>}
49    <div className="management-list">{visible.map(user => <article className="card" ke
.    y={user.id} aria-label={`User ${user.name}`}>
50      <div className="task-header"><div><h3>{user.name}</h3><p>{user.email}</p></div>
.    ><span className={`status ${user.active ? 'status-completed' : ''}`}>{user.active ? 'Activ
.    e' : 'Inactive'}</span></div>
51      <p>{user.role} · Created: {formatDate(user.createdAt)}{user.canLogin === false
.    && ' · Login not enabled'}</p>
52      <div className="actions">
53        <button disabled={busy} onClick={() => open(user, 'view')}>View</button>
54        <button disabled={busy} onClick={() => open(user, 'edit')}>Edit</button>
55        <button disabled={busy || user.id === account.id} onClick={() => perform(a
.    sync () => {
56          await onStatusUser(user.id, !user.active); setPanel(null)
57          }, user.active ? 'User deactivated. Existing sessions revoked.' : 'User re
.    activated.')}>{user.active ? 'Deactivate' : 'Activate'}</button>
58        <button className="danger" disabled={busy || user.id === account.id} onCli
.    ck={() => { setPanel({ mode: 'delete', user }); setError('') }}>Delete</button>
59      </div>
60      {user.id === account.id && <p className="form-hint">Your current Admin account
.    cannot be deleted, deactivated, or demoted.</p>}
61      {panel?.mode === 'delete' && panel.user.id === user.id && <ConfirmDelete name=
.    {user.name} busy={busy} onCancel={() => setPanel(null)}
62      onConfirm={() => perform(async () => { await onDeleteUser(user.id); setPan
.    el(null) }, 'User deleted.')} />}
63    </article>)}</div>
64  </div>
65  }
66  export default Users

```

frontend/src/services/api.js

Frontend services | 129 source lines | 5,826 bytes

SHA-256: a3377b9f7cdc2c5aaff2a35b4945c19b25202e030dc7bbfcf43c3b1d69fb4683

```

1  import { taskPriorities, taskStatuses } from '../data/taskOptions.js'
2
3  // Keep HTTP details here so components only deal with records and readable errors.
4  async function request(endpoint, options = {}) {
5      const timeout = AbortSignal.timeout(15000)
6      const signal = options.signal
7          ? AbortSignal.any([options.signal, timeout])
8          : timeout
9
10     try {
11         const response = await fetch(`/api${endpoint}`, {
12             ...options,
13             signal,
14             credentials: 'same-origin',
15             headers: { Accept: 'application/json', 'X-Requested-With': 'WorkTracker', ...o
16         ptions.headers },
17         })
18         // A proxy error can be HTML or empty, so never display raw server content.
19         const isJson = response.headers.get('content-type')?.includes('application/json')
20         const data = isJson ? await response.json() : null
21         // Effect cleanup can cancel a load while its response body is being read.
22         signal.throwIfAborted()
23         if (!response.ok) {
24             // A session that expired elsewhere returns the UI to login; passwords never e
25             nter browser storage.
26             if (response.status === 401 && endpoint !== '/auth/login' && typeof window !==
27             'undefined') {
28                 window.dispatchEvent(new Event('session-expired'))
29             }
30             if (response.status >= 500) {
31                 throw new Error('The service is unavailable. Please try again shortly.')
32             }
33             throw new Error(typeof data?.error === 'string'
34             ? data.error
35             : 'The request could not be completed. Please try again.')
36         }
37         if (!isJson || data === null) throw new Error('The service returned an unexpected
38         response. Please reload and try again.')
39         return data
40     } catch (error) {
41         // Canceled page loads are handled silently by App's effect cleanup.
42         if (options.signal?.aborted) throw error
43         if (timeout.aborted) {
44             throw new Error('The request timed out. Reload to check whether changes were s
45             aved before trying again.')
46         }
47         if (error instanceof TypeError) {
48             throw new Error('Unable to reach the service. Check your connection and try ag
49             ain.')
50         }
51     }
52 }

```

frontend/src/services/api.js (continued)

8dea970 | Listing segment 2 of 3

```

44     }
45     if (error instanceof SyntaxError) {
46         throw new Error('The service returned an unexpected response. Please reload an
. d try again.')
47     }
48     throw error
49 }
50 }
51
52 function isUser(user) {
53     return user && Number.isInteger(user.id) && user.id > 0 &&
54         typeof user.name === 'string' && typeof user.email === 'string' &&
55         ['admin', 'user'].includes(user.role)
56 }
57
58 function isTask(task) {
59     return task && Number.isInteger(task.id) && task.id > 0 &&
60         typeof task.title === 'string' &&
61         (task.description === null || typeof task.description === 'string') &&
62         taskStatuses.includes(task.status) && taskPriorities.includes(task.priority) &&
63         (task.userId === null || (Number.isInteger(task.userId) && task.userId > 0))
64 }
65
66 // Reject incompatible records before they reach components that expect these fields.
67 function validate(data, isRecord, isList = false) {
68     const valid = isList ? Array.isArray(data) && data.every(isRecord) : isRecord(data)
69     if (!valid) throw new Error('The service returned unexpected data. Please reload and t
. ry again.')
70     return data
71 }
72
73 function jsonOptions(method, body) {
74     return { method, headers: { 'Content-Type': 'application/json' }, body: JSON.stringify
. (body) }
75 }
76
77 export async function getUsers(signal) {
78     return validate(await request('/users', { signal }), isUser, true)
79 }
80
81 export async function getTasks(signal) {
82     return validate(await request('/tasks', { signal }), isTask, true)
83 }
84
85 export async function createUser(user) {
86     return validate(await request('/users', jsonOptions('POST', user)), isUser)
87 }
88
89 export async function createTask(task) {

```

frontend/src/services/api.js (continued)

8dea970 | Listing segment 3 of 3

```

90     return validate(await request('/tasks', jsonOptions('POST', task)), isTask)
91   }
92
93   export async function updateTaskStatus(id, status) {
94     return validate(await request(`/tasks/${id}/status`, jsonOptions('PATCH', { status })),
95       isTask)
96   }
97
98   export async function getSession(signal) {
99     const data = await request('/auth/me', { signal })
100     if (data.user !== null) validate(data.user, isUser)
101     return data.user
102   }
103
104   export async function login(credentials) {
105     const data = await request('/auth/login', jsonOptions('POST', credentials))
106     return validate(data.user, isUser)
107   }
108
109   export async function logout() {
110     return request('/auth/logout', jsonOptions('POST', {}))
111   }
112
113   export async function getUser(id) { return validate(await request(`/users/${id}`), isUser) }
114   export async function updateUser(id, user) { return validate(await request(`/users/${id}`,
115     jsonOptions('PUT', user)), isUser) }
116   export async function updateUserStatus(id, active) { return validate(await request(`/users
117     /${id}/status`, jsonOptions('PATCH', { active })), isUser) }
118   export async function deleteUser(id) { return request(`/users/${id}`, { method: 'DELETE' }
119     ) }
120   export async function getTask(id) { return validate(await request(`/tasks/${id}`), isTask)
121     }
122   export async function updateTask(id, task) { return validate(await request(`/tasks/${id}`,
123     jsonOptions('PUT', task)), isTask) }
124   export async function deleteTask(id) { return request(`/tasks/${id}`, { method: 'DELETE' }
125     ) }
126
127   export async function updateProfile(fields) {
128     return validate(await request('/profile', jsonOptions('PUT', fields)), isUser)
129   }
130
131   export async function changePassword(fields) {
132     const result = await request('/profile/password', jsonOptions('PUT', fields))
133     // Notify App only after a successful change, clearing the revoked session and private
134     // workload.
135     if (typeof window !== 'undefined') window.dispatchEvent(new Event('password-changed'))
136     return result
137   }

```

frontend/src/services/management.js

Frontend services | 17 source lines | 1,173 bytes

SHA-256: 2cc723b530327f2cf2459efc427742c06efaa4146c5a6b93f001f94aaf6ca930

```
1 export function filterUsers(users, { search = '', role = '', active = '', sort = 'name' })
2   {
3     const text = search.trim().toLowerCase()
4     return users.filter(user => `${user.name} ${user.email}`.toLowerCase().includes(text)
5     &&
6       (!role || user.role === role) && (!active || String(user.active) === active))
7     .sort((a, b) => sort === 'newest' ? b.id - a.id : a.name.localeCompare(b.name) ||
8     a.id - b.id)
9   }
10
11 export function filterTasks(tasks, { search = '', status = '', priority = '', userId = '',
12   sort = 'newest' }) {
13   const rank = { High: 0, Medium: 1, Low: 2 }
14   return tasks.filter(task => task.title.toLowerCase().includes(search.trim().toLowerCase()) &&
15   (!status || task.status === status) && (!priority || task.priority === priority) &
16   &
17   (!userId || (userId === 'unassigned' ? task.userId === null : String(task.userId)
18   === userId)))
19   .sort((a, b) => sort === 'title' ? a.title.localeCompare(b.title) || a.id - b.id :
20   sort === 'priority' ? rank[a.priority] - rank[b.priority] || b.id - a.id : b.i
21   d - a.id)
22 }
23
24 export function formatDate(value) { return value ? new Date(value).toLocaleString() : 'Un
25   available' }
```

frontend/src/services/personal.js

Frontend services | 31 source lines | 1,834 bytes

SHA-256: 4ffe24f84c32647f734b3e77dc1508cd265e76e3be292f12ceee49871c562b36

```

1  export function filterMyTasks(tasks, { search = '', status = '', priority = '', sort = 'newest' }) {
2      const text = search.trim().toLowerCase()
3      const priorities = { High: 0, Medium: 1, Low: 2 }
4      const statuses = { Open: 0, 'In Progress': 1, Completed: 2 }
5      // Sort a filtered copy so list controls never reorder shared dashboard state.
6      return tasks.filter(task => `${task.title} ${task.description || ''}`.toLowerCase().includes(text) &&
7          (!status || task.status === status) && (!priority || task.priority === priority))
8          .sort((a, b) => {
9              const age = new Date(a.createdAt || 0) - new Date(b.createdAt || 0) || a.id -
10             b.id
11             if (sort === 'oldest') return age
12             if (sort === 'priority') return priorities[a.priority] - priorities[b.priority]
13             if (sort === 'status') return statuses[a.status] - statuses[b.status] || -age
14             return -age
15         })
16     }
17
18     export function personalSummary(tasks) {
19         return {
20             total: tasks.length,
21             priorities: Object.fromEntries(['High', 'Medium', 'Low'].map(priority => [priority, tasks.filter(task => task.priority === priority).length])),
22             attention: filterMyTasks(tasks.filter(task => task.status !== 'Completed'), { sort : 'priority' }).slice(0, 3),
23         }
24     }
25
26     export function passwordError({ currentPassword, newPassword, confirmPassword }) {
27         if (!currentPassword) return 'Enter your current password.'
28         if (newPassword.length < 12 || new TextEncoder().encode(newPassword).length > 72) return 'New password must be at least 12 characters and at most 72 UTF-8 bytes.'
29         if (newPassword !== confirmPassword) return 'New password confirmation does not match.'
30         if (currentPassword === newPassword) return 'Choose a new password different from your current password.'
31         return ''
32     }

```

backend/test/api.test.js

Automated tests | 234 source lines | 10,843 bytes

SHA-256: ce7c480adabc8179899d35b46d2e771922ac845a62e88d8227d1668f85ed11fe

```

1  const { test, before, after, beforeEach, afterEach } = require('node:test')
2  const assert = require('node:assert/strict')
3  process.env.NODE_ENV = 'test'
4  process.env.SESSION_SECRET = require('node:crypto').randomBytes(48).toString('hex')
5  const bcrypt = require('bcryptjs')
6  const testPassword = 'unit-test-password-only'
7  const app = require('../server')
8  const pool = require('../config/database')
9
10 let server
11 let baseUrl
12 let queryMock
13 let cookie
14
15 // Exercise real HTTP parsing and routing while keeping tests independent of PostgreSQL.
16 before(async () => {
17   server = app.listen(0, '127.0.0.1')
18   await new Promise(resolve => server.once('listening', resolve))
19   baseUrl = `http://127.0.0.1:${server.address().port}`
20   const original = pool.query
21   const loginHash = await bcrypt.hash(testPassword, 4)
22   pool.query = async () => ({ rows: [{ id: 900, name: 'Test Admin', email: 'admin@exampl
   e.invalid', role: 'admin', password_hash: loginHash }] })
23   const response = await fetch(baseUrl + '/api/auth/login', {
24     method: 'POST', headers: { 'Content-Type': 'application/json', 'X-Requested-With':
   'WorkTracker' },
25     body: JSON.stringify({ email: 'admin@example.invalid', password: testPassword }),
26   })
27   assert.equal(response.status, 200)
28   cookie = response.headers.get('set-cookie').split(';')[0]
29   pool.query = original
30 })
31
32 beforeEach(context => {
33   queryMock = context.mock.fn(async () => {
34     throw new Error('Unexpected database query')
35   })
36   context.mock.method(pool, 'query', (sql, values) => {
37     if (sql.includes('WHERE id = $1 AND password_hash IS NOT NULL')) {
38       return Promise.resolve({ rows: [{ id: 900, name: 'Test Admin', email: 'admin@e
   xample.invalid', role: 'admin' }] })
39     }
40     return queryMock(sql, values)
41   })
42   context.mock.method(pool, 'connect', async () => ({
43     query: async (sql, values) => {
44       if (['BEGIN', 'COMMIT', 'ROLLBACK'].includes(sql) || sql.startsWith('SET LOCAL
   ') || sql.startsWith('LOCK TABLE')) return { rows: [] }
45       if (sql.includes("active = true AND role = 'admin'")) return { rows: [{ id: 90
   0 }] }
46       return queryMock(sql, values)

```

backend/test/api.test.js (continued)

8dea970 | Listing segment 2 of 5

```

47     }, release() {}),
48   )))
49   context.mock.method(console, 'error', () => {})
50 })
51
52 afterEach(() => queryMock.mock.restore())
53 after(async () => {
54   await new Promise(resolve => server.close(resolve))
55   await pool.end()
56 })
57
58 async function request(path, method = 'GET', body) {
59   if (path === '/api/users' && method === 'POST' && body && !Array.isArray(body) && type
    of body === 'object') {
60     body = { password: testPassword, role: 'user', ...body }
61   }
62   const response = await fetch(`${baseUrl}${path}`, {
63     method,
64     headers: { Cookie: cookie, 'X-Requested-With': 'WorkTracker', ...(body === undefin
    ed ? {} : { 'Content-Type': 'application/json' }) },
65     body: body === undefined ? undefined : JSON.stringify(body),
66   })
67   assert.match(response.headers.get('content-type'), /application\/json/)
68   return { status: response.status, body: await response.json() }
69 }
70
71 test('user validation rejects invalid fields without querying PostgreSQL', async () => {
72   for (const body of [
73     {}, { name: 3, email: 'a@example.com' },
74     { name: ' ', email: 'a@example.com' },
75     { name: 'a'.repeat(101), email: 'a@example.com' },
76     { name: 'A', email: null }, { name: 'A', email: 'invalid' },
77     { name: 'A', email: `${'a'.repeat(250)}@example.com` },
78   ]) {
79     assert.equal((await request('/api/users', 'POST', body)).status, 400)
80   }
81   assert.equal(queryMock.mock.callCount(), 0)
82 })
83
84 test('user creation normalizes input and returns the created record', async () => {
85   const user = { id: 1, name: 'Alice', email: 'alice@example.com', createdAt: '2026-09-2
    8' }
86   queryMock.mock.mockImplementation(async (sql, values) => {
87     assert.match(sql, /created_at AS "createdAt"/)
88     assert.deepEqual(values.slice(0, 2), ['Alice', 'alice@example.com'])
89     assert.equal(await bcrypt.compare(testPassword, values[2]), true)
90     assert.equal(values[3], 'user')
91     return { rows: [user] }
92   })
93   assert.deepEqual(await request('/api/users', 'POST', {
94     name: ' Alice ', email: ' ALICE@example.com ',

```

backend/test/api.test.js (continued)

8dea970 | Listing segment 3 of 5

```

95     }), { status: 201, body: user })
96   })
97
98   test('duplicate email returns 409', async () => {
99     queryMock.mock.mockImplementation(async () => { throw { code: '23505' } })
100     assert.equal((await request('/api/users', 'POST', { name: 'A', email: 'a@example.com'
101       })).status, 409)
102   })
103
104   test('account creation rejects invalid passwords and roles before SQL', async () => {
105     for (const fields of [
106       { password: '' }, { password: null }, { password: 'short' },
107       { password: 'a'.repeat(73) }, { password: '\U0001f600'.repeat(19) }, { role: 'supe
108       ruser' },
109     ]) {
110       assert.equal((await request('/api/users', 'POST', { name: 'Account', email: 'accou
111         nt@example.invalid', ...fields })).status, 400)
112     }
113     assert.equal(queryMock.mock.callCount(), 0)
114   })
115
116   test('task validation rejects invalid fields before any query', async () => {
117     for (const fields of [
118       { title: null }, { title: 42 }, { title: '' }, { title: 'a'.repeat(201) },
119       { description: null }, { description: {} }, { priority: 'Urgent' },
120       { userId: '1' }, { userId: 0 }, { userId: -1 }, { userId: 1.5 },
121       { userId: 2147483648 },
122     ]) {
123       assert.equal((await request('/api/tasks', 'POST', { title: 'Task', ...fields })).s
124         tatus, 400)
125     }
126     assert.equal(queryMock.mock.callCount(), 0)
127   })
128
129   test('task creation applies defaults and uses SQL parameters', async () => {
130     const task = { id: 1, title: "Task ' title", description: '', priority: 'Medium', stat
131       us: 'Open', userId: null }
132     queryMock.mock.mockImplementation(async (sql, values) => {
133       assert.match(sql, /VALUES \(\(\$1, \$2, \$3, \$4, \$5\)/)
134       assert.deepEqual(values, ["Task ' title", '', 'Medium', null, 'Open'])
135       return { rows: [task] }
136     })
137     assert.deepEqual(await request('/api/tasks', 'POST', { title: " Task ' title " }), { s
138       tatus: 201, body: task })
139   })
140
141   test('assigned task checks the user and preserves their numeric ID', async () => {
142     queryMock.mock.mockImplementation(async (sql, values) => {
143       if (sql.startsWith('SELECT id, active FROM users')) {
144         assert.deepEqual(values, [2])
145         return { rows: [{ id: 2 }] }
146       }
147     })
148   })

```

backend/test/api.test.js (continued)

8dea970 | Listing segment 4 of 5

```

140     }
141     assert.deepEqual(values, ['Task', 'Details', 'High', 2, 'Open'])
142     return { rows: [{ id: 1, userId: 2 }] }
143   })
144   assert.equal((await request('/api/tasks', 'POST', {
145     title: 'Task', description: 'Details ', priority: 'High', userId: 2,
146   })).status, 201)
147   assert.equal(queryMock.mock.callCount(), 2)
148 })
149
150 test('missing assigned user and assignment race both return 400', async () => {
151   queryMock.mock.mockImplementation(async () => ({ rows: [] }))
152   assert.equal((await request('/api/tasks', 'POST', { title: 'Task', userId: 5 })).status, 400)
153   queryMock.mock.mockImplementation(async sql => {
154     if (sql.startsWith('SELECT')) return { rows: [{ id: 5 }] }
155     throw { code: '23503' }
156   })
157   assert.equal((await request('/api/tasks', 'POST', { title: 'Task', userId: 5 })).status, 400)
158 })
159
160 test('status update rejects malformed IDs and unsupported statuses', async () => {
161   for (const id of ['abc', '0', '-1', '1.5', '2147483648']) {
162     assert.equal((await request(`/api/tasks/${id}/status`, 'PATCH', { status: 'Open' })).status, 400)
163   }
164   assert.equal((await request('/api/tasks/1/status', 'PATCH', { status: 'Done' })).status, 400)
165   assert.equal(queryMock.mock.callCount(), 0)
166 })
167
168 test('status update handles every supported status and missing tasks', async () => {
169   for (const status of ['Open', 'In Progress', 'Completed']) {
170     queryMock.mock.mockImplementation(async (sql, values) => {
171       assert.match(sql, /updated_at = CURRENT_TIMESTAMP/)
172       assert.deepEqual(values, [status, 1, true, 900])
173       return { rows: [{ id: 1, status }] }
174     })
175     assert.deepEqual(await request('/api/tasks/1/status', 'PATCH', { status }), {
176       status: 200, body: { id: 1, status },
177     })
178   }
179   queryMock.mock.mockImplementation(async () => ({ rows: [] }))
180   assert.equal((await request('/api/tasks/99/status', 'PATCH', { status: 'Open' })).status, 404)
181 })
182
183 test('list routes return populated and empty arrays', async () => {
184   for (const path of ['/api/users', '/api/tasks']) {
185     for (const rows of [[], [{ id: 1 }]]) {

```

backend/test/api.test.js (continued)

8dea970 | Listing segment 5 of 5

```

186         queryMock.mock.mockImplementation(async () => ({ rows }))
187         assert.deepEqual(await request(path), { status: 200, body: rows })
188     }
189 }
190 })
191
192 test('request parser failures and unknown routes remain JSON', async () => {
193     for (const path of ['/api/users', '/api/tasks', '/api/tasks/1/status']) {
194         const method = path.endsWith('/status') ? 'PATCH' : 'POST'
195         for (const body of [[], null, 'text', 1]) {
196             assert.equal((await request(path, method, body)).status, 400)
197         }
198         assert.equal((await request(path, method)).status, 415)
199     }
200     for (const [body, status] of [['', 400], [JSON.stringify({ title: 'a'.repeat(110000)
201 .    }), 413]]) {
202         const response = await fetch(`${baseUrl}/api/tasks`, {
203             method: 'POST', headers: { 'Content-Type': 'application/json' }, body,
204         })
205         assert.equal(response.status, status)
206         assert.equal(typeof (await response.json()).error, 'string')
207     }
208     assert.equal(await request('/api/missing').status, 404)
209     assert.equal(queryMock.mock.callCount(), 0)
210 })
211
212 test('database failures do not expose internal error details', async () => {
213     queryMock.mock.mockImplementation(async () => { throw new Error('private database deta
214 .    ils') })
215     for (const [path, method, body] of [
216         ['/api/users', 'GET'], ['/api/tasks', 'GET'],
217         ['/api/users', 'POST', { name: 'A', email: 'a@example.com' }],
218         ['/api/tasks', 'POST', { title: 'Task' }],
219         ['/api/tasks/1/status', 'PATCH', { status: 'Open' }],
220     ]) {
221         assert.deepEqual(await request(path, method, body), {
222             status: 500, body: { error: 'An unexpected server error occurred' },
223         })
224     }
225 })
226
227 test('health returns 200 when available and 503 on database failure', async () => {
228     queryMock.mock.mockImplementation(async () => ({ rows: [{ now: '2026-09-28' }] }))
229     const healthy = await request('/api/health')
230     assert.equal(healthy.status, 200)
231     assert.equal(healthy.body.database, 'connected')
232     queryMock.mock.mockImplementation(async () => { throw { code: '28P01' } })
233     assert.deepEqual(await request('/api/health'), {
234         status: 503, body: { status: 'error', error: 'Database connection failed' },
235     })
236 })

```

backend/test/auth.test.js

Automated tests | 174 source lines | 8,810 bytes

SHA-256: c6b70d10653bb5624ade8c5747d4cf4889de0965c38c1fadb2c4879bc78efcee

```

1  const { test, before, after, beforeEach } = require('node:test')
2  const assert = require('node:assert/strict')
3  const { randomBytes } = require('node:crypto')
4  const bcrypt = require('bcryptjs')
5  process.env.NODE_ENV = 'test'
6  process.env.SESSION_SECRET = randomBytes(48).toString('hex')
7  const app = require('../server')
8  const pool = require('../config/database')
9  const password = 'unit-test-credentials-only'
10 let server, base, accounts, tasks
11
12 before(async () => {
13   server = app.listen(0, '127.0.0.1')
14   await new Promise(resolve => server.once('listening', resolve))
15   base = `http://127.0.0.1:${server.address().port}`
16 })
17 after(async () => {
18   await new Promise(resolve => server.close(resolve))
19   await pool.end()
20 })
21
22 beforeEach(async context => {
23   const hash = await bcrypt.hash(password, 4)
24   accounts = [
25     { id: 1, name: 'Admin', email: 'admin@example.invalid', role: 'admin', password_hash: hash },
26     { id: 2, name: 'User', email: 'user@example.invalid', role: 'user', password_hash: hash },
27     { id: 3, name: 'Other', email: 'other@example.invalid', role: 'user', password_hash: hash },
28     { id: 4, name: 'Legacy', email: 'legacy@example.invalid', role: 'user', password_hash: null },
29   ]
30   tasks = [{ id: 10, userId: 2, status: 'Open' }, { id: 11, userId: 3, status: 'Open' }]
31   // Query stubs enforce the same filters as SQL; live verification separately exercises PostgreSQL.
32   context.mock.method(pool, 'query', async (sql, values) => {
33     if (sql.includes('WHERE lower(email) = $1')) return { rows: accounts.filter(user => user.email === values[0]) }
34     if (sql.includes('WHERE id = $1 AND password_hash IS NOT NULL')) {
35       return { rows: accounts.filter(user => user.id === values[0] && user.password_hash && user.active !== false).map(({ password_hash, ...safe }) => safe) }
36     }
37     if (sql.includes('FROM tasks')) {
38       assert.match(sql, /WHERE \(\(\$1::boolean OR user_id = \$2\)/)
39       return { rows: tasks.filter(task => values[0] || task.userId === values[1]) }
40     }
41     if (sql.includes('UPDATE tasks')) {
42       assert.match(sql, /WHERE id = \$2 AND \(\(\$3::boolean OR user_id = \$4\)/)
43       const task = tasks.find(task => task.id === values[1] && (values[2] || task.userId === values[3]))

```

backend/test/auth.test.js (continued)

8dea970 | Listing segment 2 of 4

```

44         if (task) task.status = values[0]
45         return { rows: task ? [task] : [] }
46     }
47     if (sql.includes('FROM users')) {
48         assert.ok(!/SELECT\s+\*/.test(sql))
49         return { rows: accounts.map(({ password_hash, ...safe }) => safe) }
50     }
51     throw new Error('Unexpected query')
52 })
53 })
54
55 async function request(path, { cookie, method = 'GET', body, verified = true } = {}) {
56     const response = await fetch(base + path, {
57         method,
58         headers: {
59             ...(verified ? { 'X-Requested-With': 'WorkTracker' } : {}),
60             ...(cookie ? { Cookie: cookie } : {}),
61             ...(body === undefined ? {} : { 'Content-Type': 'application/json' }),
62         },
63         body: body === undefined ? undefined : JSON.stringify(body),
64     })
65     const data = await response.json()
66     assert.ok(!JSON.stringify(data).includes('password_hash'))
67     assert.ok(!JSON.stringify(data).includes(password))
68     return { status: response.status, data, cookie: response.headers.get('set-cookie') }
69 }
70
71 async function signIn(email = 'user@example.invalid') {
72     const result = await request('/api/auth/login', { method: 'POST', body: { email, password } })
73     assert.equal(result.status, 200)
74     assert.match(result.cookie, /HttpOnly/)
75     assert.match(result.cookie, /SameSite=Lax/)
76     return result.cookie.split(';')[0]
77 }
78
79 test('unauthenticated protected requests are rejected', async () => {
80     for (const path of ['/api/users', '/api/tasks']) assert.equal((await request(path)).status, 401)
81     assert.equal((await request('/api/users', { method: 'POST', body: {} })).status, 401)
82     assert.deepEqual((await request('/api/auth/me')).data, { user: null })
83 })
84
85 test('invalid and legacy logins fail with the same safe message', async () => {
86     for (const email of ['user@example.invalid', 'missing@example.invalid', 'legacy@example.invalid']) {
87         const result = await request('/api/auth/login', { method: 'POST', body: { email, password: 'incorrect-password' } })
88         assert.equal(result.status, 401)
89         assert.deepEqual(result.data, { error: 'Email or password is incorrect' })
90         assert.equal(result.cookie, null)

```

backend/test/auth.test.js (continued)

8dea970 | Listing segment 3 of 4

```

91     }
92   })
93
94   test('login restores account via cookie and logout invalidates copied cookie', async () =>
95     {
96       const cookie = await signIn()
97       assert.equal((await request('/api/auth/me', { cookie })).data.user.id, 2)
98       assert.equal((await request('/api/auth/logout', { cookie, method: 'POST', body: {} })))
99         .status, 200)
100       assert.equal((await request('/api/tasks', { cookie })).status, 401)
101       assert.deepEqual((await request('/api/auth/me', { cookie })).data, { user: null })
102     })
103
104   test('regular user cannot use Admin management endpoints', async () => {
105     const cookie = await signIn()
106     assert.equal((await request('/api/users', { cookie })).status, 403)
107     assert.equal((await request('/api/users', { cookie, method: 'POST', body: { role: 'adm
108       in' } })).status, 403)
109     assert.equal((await request('/api/tasks', { cookie, method: 'POST', body: { title: 'By
110       pass' } })).status, 403)
111   })
112
113   test('users see only their tasks and cannot override filters with query parameters', async
114     () => {
115     const cookie = await signIn()
116     const result = await request('/api/tasks?userId=3&role=admin', { cookie })
117     assert.deepEqual(result.data.map(task => task.id), [10])
118   })
119
120   test('users can update their own task but not another user task', async () => {
121     const cookie = await signIn()
122     const own = await request('/api/tasks/10/status', { cookie, method: 'PATCH', body: { s
123       tatus: 'Completed' } })
124     assert.equal(own.status, 200)
125     assert.equal(own.data.status, 'Completed')
126     const other = await request('/api/tasks/11/status', { cookie, method: 'PATCH', body: {
127       status: 'Completed', role: 'admin', userId: 2 } })
128     assert.equal(other.status, 403)
129     assert.equal(tasks[1].status, 'Open')
130   })
131
132   test('Admin sees all records and can update any task', async () => {
133     const cookie = await signIn('admin@example.invalid')
134     assert.equal((await request('/api/users', { cookie })).status, 200)
135     assert.equal((await request('/api/tasks', { cookie })).data.length, 2)
136     assert.equal((await request('/api/tasks/11/status', { cookie, method: 'PATCH', body: {
137       status: 'Completed' } })).status, 200)
138   })
139
140   test('write verification rejects cross-site style requests', async () => {
141     assert.equal((await request('/api/auth/login', { method: 'POST', body: {}, verified: f

```

backend/test/auth.test.js (continued)

8dea970 | Listing segment 4 of 4

```

    .   else })).status, 403)
134     const cookie = await signIn()
135     assert.equal((await request('/api/auth/logout', { cookie, method: 'POST', body: {}, ve
    .   rified: false })).status, 403)
136   })
137
138   test('login replaces session IDs and roles are re-read from the database', async () => {
139     const cookie = await signIn('admin@example.invalid')
140     const second = await request('/api/auth/login', { cookie, method: 'POST', body: { emai
    .   l: 'admin@example.invalid', password } })
141     const newer = second.cookie.split(';')[0]
142     assert.notEqual(newer, cookie)
143     assert.equal((await request('/api/users', { cookie })).status, 401)
144     accounts[0].role = 'user'
145     assert.equal((await request('/api/users', { cookie: newer })).status, 403)
146   })
147
148   test('invalid cookie cannot forge authentication', async () => {
149     assert.equal((await request('/api/users', { cookie: 'worktracker.sid=admin' })).status
    .   , 401)
150   })
151
152   test('inactive accounts cannot log in or retain authenticated access', async () => {
153     const cookie = await signIn()
154     accounts[1].active = false
155     assert.equal((await request('/api/tasks', { cookie })).status, 401)
156     const denied = await request('/api/auth/login', { method: 'POST', body: { email: 'user
    .   @example.invalid', password } })
157     assert.equal(denied.status, 401)
158     accounts[1].active = true
159     assert.equal((await request('/api/tasks', { cookie })).status, 401)
160   })
161
162   test('reassignment immediately changes ownership on subsequent reads and updates', async (
    .   ) => {
163     const cookie = await signIn()
164     tasks[0].userId = 3
165     assert.deepEqual((await request('/api/tasks', { cookie })).data, [])
166     assert.equal((await request('/api/tasks/10/status', { cookie, method: 'PATCH', body: {
    .   status: 'Completed' } })).status, 403)
167   })
168
169   test('login rate limit returns safe JSON', async () => {
170     let result
171     for (let i = 0; i < 21; i++) result = await request('/api/auth/login', { method: 'POST
    .   ', body: {} })
172     assert.equal(result.status, 429)
173     assert.match(result.data.error, /Too many sign-in attempts/)
174   })

```

backend/test/database-outage.test.js

Automated tests | 44 source lines | 2,288 bytes

SHA-256: 39117e646e40cd86cfb95be227afa844f054b726f9ea190b58f633c5c87301bd

```

1  const { test } = require('node:test')
2  const assert = require('node:assert/strict')
3  const net = require('node:net')
4  const { once } = require('node:events')
5  const { randomBytes } = require('node:crypto')
6
7  test('database outage returns safe health/login failures without stopping Express', async
  . context => {
8    // Use an unused local port in this isolated test process; never stop the real databas
  . e.
9    const reservation = net.createServer().listen(0, '127.0.0.1')
10   await once(reservation, 'listening')
11   const port = reservation.address().port
12   await new Promise(resolve => reservation.close(resolve))
13   Object.assign(process.env, {
14     NODE_ENV: 'development', SESSION_SECRET: randomBytes(48).toString('hex'),
15     DB_HOST: '127.0.0.1', DB_PORT: String(port), DB_NAME: 'unused',
16     DB_USER: 'unused', DB_PASSWORD: randomBytes(24).toString('hex'),
17   })
18   const logs = []
19   context.mock.method(console, 'error', (...args) => logs.push(args.join(' ')))
20   const app = require('../server')
21   const pool = require('../config/database')
22   const server = app.listen(0, '127.0.0.1')
23   await once(server, 'listening')
24   try {
25     const base = `http://127.0.0.1:${server.address().port}/api`
26     const health = await fetch(base + '/health')
27     assert.equal(health.status, 503)
28     assert.deepEqual(await health.json(), { status: 'error', error: 'Database connecti
  . on failed' })
29     const login = await fetch(base + '/auth/login', { method: 'POST',
30       headers: { 'Content-Type': 'application/json', 'X-Requested-With': 'WorkTracke
  . r' },
31       body: JSON.stringify({ email: 'outage@example.invalid', password: 'test-only-p
  . assword' }) })
32     assert.equal(login.status, 500)
33     assert.deepEqual(await login.json(), { error: 'An unexpected server error occurred
  . ' })
34     assert.equal(login.headers.get('set-cookie'), null)
35     assert.equal((await fetch(base + '/tasks')).status, 401)
36     const output = logs.join('\n')
37     for (const secret of [process.env.DB_PASSWORD, process.env.SESSION_SECRET, 'SELECT
  . ', 'outage@example.invalid']) {
38       assert.ok(!output.includes(secret))
39     }
40   } finally {
41     await new Promise(resolve => server.close(resolve))
42     await pool.end()
43   }
44 })

```

backend/test/environment.test.js

Automated tests | 54 source lines | 2,170 bytes

SHA-256: b80da905320a03978d8ba7947b98cedff7ca94f5b6fa69508cd4b21c0bfd9c6a

```

1  const { test } = require('node:test')
2  const assert = require('node:assert/strict')
3  const validate = require('../config/environment')
4  const createSessionMiddleware = require('../config/session')
5  const { readFileSync } = require('node:fs')
6  const { join } = require('node:path')
7
8  // Use temporary process values only; tests neither read nor change the real .env file.
9  function withEnvironment(overrides, run) {
10     const values = {
11       DB_HOST: 'localhost', DB_PORT: '5432', DB_NAME: 'my_web_app_v2',
12       DB_USER: 'postgres', DB_PASSWORD: 'test-only-not-a-real-password', ...overrides,
13     }
14     const previous = {}
15     for (const name of Object.keys(values)) {
16       previous[name] = process.env[name]
17       process.env[name] = values[name]
18     }
19     try { run() } finally {
20       for (const name of Object.keys(values)) {
21         if (previous[name] === undefined) delete process.env[name]
22         else process.env[name] = previous[name]
23       }
24     }
25   }
26
27   test('valid database environment is accepted', () => {
28     withEnvironment({}, () => assert.doesNotThrow(validate))
29   })
30
31   test('missing settings fail without revealing credential values', () => {
32     for (const name of ['DB_HOST', 'DB_PORT', 'DB_NAME', 'DB_USER', 'DB_PASSWORD']) {
33       withEnvironment({ [name]: '' }, () => assert.throws(validate, {
34         message: `Missing required environment variable: ${name}`,
35       }))
36     }
37   })
38
39   test('invalid ports and the example password are rejected', () => {
40     for (const port of ['0', '65536', '1.5', 'abc', '1e3']) {
41       withEnvironment({ DB_PORT: port }, () => assert.throws(validate, /DB_PORT/))
42     }
43     withEnvironment({ DB_PASSWORD: 'your_local_postgres_password' }, () => {
44       assert.throws(validate, /placeholder/)
45     })
46   })
47
48   test('the documented session-secret placeholder cannot start a session store', () => {
49     const example = readFileSync(join(__dirname, '../.env.example'), 'utf8')
50     const placeholder = example.match(/^SESSION_SECRET=(.+)$/m)[1].trim()
51     withEnvironment({ NODE_ENV: 'test', SESSION_SECRET: placeholder }, () => {
52       assert.throws(createSessionMiddleware, /SESSION_SECRET/)
53     })
54   })

```

backend/test/management.test.js

Automated tests | 146 source lines | 9,590 bytes

SHA-256: 5aff3d39ffdd2fa1c6daad088f3475069a6f9648a389370170702073889eafc0

```

1  const { test, before, after, beforeEach } = require('node:test')
2  const assert = require('node:assert/strict')
3  const bcrypt = require('bcryptjs')
4  process.env.NODE_ENV = 'test'
5  process.env.SESSION_SECRET = require('node:crypto').randomBytes(48).toString('hex')
6  const app = require('../server')
7  const pool = require('../config/database')
8  let server, base, adminCookie, userCookie, query, statements
9  const password = 'management-test-only-password'
10 const user = { id: 2, name: 'Member', email: 'member@example.invalid', role: 'user', active: true, canLogin: true }
11 const task = { id: 7, title: 'Work', description: '', status: 'Open', priority: 'Medium', userId: 2 }
12
13 before(async () => {
14   const hash = await bcrypt.hash(password, 4)
15   const original = pool.query
16   pool.query = async (sql, values) => ({ rows: [{ id: values[0] === 'admin@example.invalid' ? 1 : 2,
17     name: 'Test', email: values[0], role: values[0] === 'admin@example.invalid' ? 'admin' : 'user', active: true, password_hash: hash }] })
18   server = app.listen(0, '127.0.0.1')
19   await new Promise(resolve => server.once('listening', resolve))
20   base = `http://127.0.0.1:${server.address().port}`
21   for (const email of ['admin@example.invalid', 'member@example.invalid']) {
22     const r = await fetch(base + '/api/auth/login', { method: 'POST', headers: { 'Content-Type': 'application/json', 'X-Requested-With': 'WorkTracker' }, body: JSON.stringify({ email, password }) })
23     assert.equal(r.status, 200)
24     const cookie = r.headers.get('set-cookie').split(';')[0]
25     if (email.startsWith('admin')) adminCookie = cookie
26     else userCookie = cookie
27   }
28   pool.query = original
29 })
30 after(async () => { await new Promise(resolve => server.close(resolve)); await pool.end() })
31 beforeEach(context => {
32   statements = []
33   query = async () => { throw Error('Unexpected SQL') }
34   const run = async (sql, values) => {
35     statements.push(sql)
36     if (['BEGIN', 'COMMIT', 'ROLLBACK'].includes(sql) || sql.startsWith('SET LOCAL') || sql.startsWith('LOCK TABLE')) return { rows: [] }
37     if (sql.includes('WHERE id = $1 AND password_hash IS NOT NULL')) return { rows: [{ id: values[0], role: values[0] === 1 ? 'admin' : 'user' }] }
38     if (sql.includes('active = true AND role = \'admin\'')) return { rows: [{ id: 1 }] }

```

backend/test/management.test.js (continued)

8dea970 | Listing segment 2 of 4

```

39     return query(sql, values)
40   }
41   context.mock.method(pool, 'query', run)
42   context.mock.method(pool, 'connect', async () => ({ query: run, release() {} }))
43 })
44 async function request(path, method = 'GET', body, cookie = adminCookie) {
45   const r = await fetch(base + '/api' + path, { method, headers: { Cookie: cookie, 'Content-Type': 'application/json', 'X-Requested-With': 'WorkTracker' }, body: body === undefined ? undefined : JSON.stringify(body) })
46   const data = await r.json()
47   assert.ok(!JSON.stringify(data).includes('password_hash'))
48   return { status: r.status, data }
49 }
50
51 test('user details include a safe workload summary and missing IDs return 404', async () => {
52   query = async sql => ({ rows: sql.includes('FROM tasks') ? [{ total: 2, open: 1, inProgress: 0, completed: 1 }] : [user] })
53   const result = await request('/users/2')
54   assert.equal(result.status, 200); assert.equal(result.data.taskSummary.total, 2)
55   query = async () => ({ rows: [] })
56   assert.equal((await request('/users/99')).status, 404)
57 })
58 test('editing normalizes profile fields without changing passwords', async () => {
59   query = async (sql, values) => {
60     if (sql.startsWith('UPDATE users')) assert.deepEqual(values, ['Member', 'member@example.invalid', 'user', 2])
61     return { rows: [user] }
62   }
63   assert.equal((await request('/users/2', 'PUT', { name: 'Member ', email: 'MEMBER@example.invalid ', role: 'user' })).status, 200)
64   assert.ok(statements.includes('COMMIT'))
65   assert.ok(!statements.some(sql => sql.startsWith('UPDATE') && sql.includes('password')))
66 })
67 test('deactivation revokes stored sessions; reactivation preserves task relationships', async () => {
68   for (const active of [false, true]) {
69     query = async (sql, values) => {
70       if (sql.startsWith('UPDATE users')) assert.deepEqual(values, [active, 2])
71       if (sql.startsWith('DELETE FROM sessions')) assert.deepEqual(values, ['2'])
72       return { rows: [{ ...user, active }] }
73     }
74     assert.equal((await request('/users/2/status', 'PATCH', { active })).status, 200)
75   }
76   assert.ok(statements.some(sql => sql.startsWith('DELETE FROM sessions')))
77   assert.ok(!statements.some(sql => /^(UPDATE|DELETE FROM) tasks/.test(sql)))

```

backend/test/management.test.js (continued)

8dea970 | Listing segment 3 of 4

```

78 })
79 test('current Admin cannot delete, deactivate, or demote themselves', async () => {
80   query = async () => ({ rows: [{ ...user, id: 1, role: 'admin' }] })
81   for (const [path, method, body] of [['/users/1', 'DELETE'], ['/users/1/status', 'PATCH', { active: false }], ['/users/1', 'PUT', { name: user.name, email: user.email, role: 'user' }]]) {
82     assert.equal((await request(path, method, body)).status, 409)
83   }
84   assert.ok(!statements.some(sql => sql.startsWith('UPDATE') || sql.startsWith('DELETE'))
85 )
86 test('last usable Admin and legacy privilege escalation are protected', async () => {
87   query = async sql => ({ rows: sql.includes('id <>') ? [] : [{ ...user, role: 'admin' }] })
88   assert.equal((await request('/users/2', 'DELETE')).status, 409)
89   query = async () => ({ rows: [{ ...user, canLogin: false }] })
90   assert.equal((await request('/users/2', 'PUT', { name: 'Member', email: user.email, role: 'admin' })).status, 409)
91 })
92 test('assigned users cannot be deleted; safe deletion also removes sessions', async () => {
93   query = async () => ({ rows: [user] })
94   assert.equal((await request('/users/2', 'DELETE')).status, 409)
95   assert.ok(statements.includes('ROLLBACK'))
96   query = async sql => ({ rows: sql.includes('FROM tasks') ? [] : [user] })
97   assert.equal((await request('/users/2', 'DELETE')).status, 200)
98   assert.ok(statements.includes('DELETE FROM users WHERE id = $1'))
99 })
100 test('user edits validate every field and keep duplicate emails safe', async () => {
101   for (const change of [{ name: '' }, { name: ' ' }, { name: 'x'.repeat(101) }, { email: 'wrong' }, { email: 'x'.repeat(256) }, { role: 'owner' }, { password: 'forbidden' }]) {
102     assert.equal((await request('/users/2', 'PUT', { name: user.name, email: user.email, role: user.role, ...change })).status, 400)
103   }
104   for (const active of ['false', null, 1]) assert.equal((await request('/users/2/status', 'PATCH', { active })).status, 400)
105   query = async sql => { if (sql.startsWith('UPDATE')) throw { code: '23505' }; return { rows: [user] } }
106   assert.equal((await request('/users/2', 'PUT', { name: user.name, email: user.email, role: user.role })).status, 409)
107 })
108 test('task details enforce ownership in SQL', async () => {
109   query = async (sql, values) => { assert.match(sql, /id = \$1 AND \(\$2:boolean OR use r_id = \$3\)/); assert.deepEqual(values, [7, false, 2]); return { rows: [task] } }
110   assert.equal((await request('/tasks/7', 'GET', undefined, userCookie)).status, 200)
111 })
112 test('task edit changes assignment, priority, status and timestamp atomically', async () => {

```

backend/test/management.test.js (continued)

8dea970 | Listing segment 4 of 4

```

.   > {
113     query = async (sql, values) => {
114       if (sql.startsWith('UPDATE tasks')) {
115         assert.deepEqual(values, ['Changed', 'Details', 'High', 3, 'Completed', 7])
116         assert.match(sql, /updated_at = CURRENT_TIMESTAMP/)
117         return { rows: [{ ...task, title: 'Changed', userId: 3, priority: 'High', stat
.         us: 'Completed' }] }
118       }
119       return { rows: sql.includes('FROM users') ? [{ id: 3, active: true }] : [task] }
120     }
121     const r = await request('/tasks/7', 'PUT', { title: ' Changed ', description: 'Details
.     ', priority: 'High', userId: 3, status: 'Completed' })
122     assert.equal(r.status, 200); assert.equal(r.data.userId, 3)
123   })
124   test('task assignment rejects missing/inactive accounts but preserves existing inactive as
.   signment', async () => {
125     for (const rows of [[], [{ id: 3, active: false }]]) {
126       query = async sql => ({ rows: sql.includes('FROM users') ? rows : [task] })
127       assert.equal((await request('/tasks/7', 'PUT', { ...task, userId: 3 })).status, 40
.       0)
128     }
129     query = async sql => ({ rows: sql.includes('FROM users') ? [{ id: 2, active: false }]
.     : [task] })
130     assert.equal((await request('/tasks/7', 'PUT', task)).status, 200)
131     for (const fields of [{ title: ' ' }, { description: 'x'.repeat(10001) }, { priority:
.     'Urgent' }, { status: 'Done' }, { userId: '2' }]) {
132       assert.equal((await request('/tasks/7', 'PUT', { ...task, ...fields })).status, 40
.       0)
133     }
134   })
135   test('Admin can delete tasks and receives 404 for missing tasks', async () => {
136     query = async (sql, values) => { assert.equal(sql, 'DELETE FROM tasks WHERE id = $1 RE
.     TURNING id'); assert.deepEqual(values, [7]); return { rows: [task] } }
137     assert.equal((await request('/tasks/7', 'DELETE')).status, 200)
138     query = async () => ({ rows: [] })
139     assert.equal((await request('/tasks/7', 'DELETE')).status, 404)
140   })
141   test('all new management writes reject unauthenticated and regular User requests', async (
.   ) => {
142     for (const [path, method] of [['/users/2', 'GET'], ['/users/2', 'PUT'], ['/users/2', '
.     DELETE'], ['/users/2/status', 'PATCH'], ['/tasks/7', 'PUT'], ['/tasks/7', 'DELETE']]) {
143       assert.equal((await request(path, method, method === 'GET' ? undefined : {}, '')).
.     status, 401)
144       assert.equal((await request(path, method, method === 'GET' ? undefined : {}, userC
.     ookie)).status, 403)
145     }
146   })

```

backend/test/profile.test.js

Automated tests | 161 source lines | 10,349 bytes

SHA-256: a00912b45bbc8a7143fd8c441fc06334cff226510e41452836c2e968113a5e1b

```

1  const { test, before, after, beforeEach } = require('node:test')
2  const assert = require('node:assert/strict')
3  const { randomBytes } = require('node:crypto')
4  const bcrypt = require('bcryptjs')
5  const session = require('express-session')
6  process.env.NODE_ENV = 'test'
7  process.env.SESSION_SECRET = randomBytes(48).toString('hex')
8  const app = require('../server')
9  const pool = require('../config/database')
10 const oldPassword = randomBytes(24).toString('hex')
11 const newPassword = randomBytes(24).toString('hex')
12 let server, base, account, cookie, store, statements, sequence = 100
13 const memorySet = session.MemoryStore.prototype.set
14 before(async () => {
15     server = app.listen(0, '127.0.0.1')
16     await new Promise(resolve => server.once('listening', resolve))
17     base = `http://127.0.0.1:${server.address().port}/api`
18 })
19 after(async () => { await new Promise(resolve => server.close(resolve)); await pool.end()
20 . })
21 beforeEach(async context => {
22     account = { id: ++sequence, name: 'Personal Test', email: 'personal@example.invalid',
23     . role: 'user', active: true,
24     password_hash: await bcrypt.hash(oldPassword, 4), createdAt: '2026-09-29', updated
25     . At: '2026-09-29' }
26     statements = []
27     context.mock.method(session.MemoryStore.prototype, 'set', function (...args) { store =
28     . this; return memorySet.apply(this, args) })
29     const query = async (sql, values) => {
30     . statements.push({ sql, values })
31     . if (['BEGIN', 'COMMIT', 'ROLLBACK'].includes(sql) || sql.startsWith('SET LOCAL') |
32     . | sql.startsWith('LOCK TABLE')) return { rows: [] }
33     . if (sql.includes('WHERE lower(email) = $1')) return { rows: values[0] === account.
34     . email ? [{ ...account }] : [] }
35     . if (sql.includes('WHERE id = $1 AND password_hash IS NOT NULL')) {
36     . const { password_hash, ...safe } = account
37     . return { rows: account.active && values[0] === account.id ? [safe] : [] }
38     . }
39     . if (sql.startsWith('SELECT password_hash')) return { rows: account.active ? [{ pas
40     . sword_hash: account.password_hash }] : [] }
41     . if (sql.startsWith('UPDATE users SET name')) {
42     . assert.equal(values[2], account.id)
43     . if (values[1] === 'duplicate@example.invalid') throw { code: '23505' }
44     . account.name = values[0]; account.email = values[1]
45     . const { password_hash, ...safe } = account
46     . return { rows: [safe] }
47     . }
48     . if (sql.startsWith('UPDATE users SET password_hash')) {
49     . assert.equal(values[1], account.id); account.password_hash = values[0]; return
50     . { rows: [] }
51     . }
52     . if (sql.startsWith('DELETE FROM sessions')) {

```

backend/test/profile.test.js (continued)

8dea970 | Listing segment 2 of 4

```

45         assert.deepEqual(values, [String(account.id)])
46         // Adapt the SQL-backed deletion to the isolated in-memory test store.
47         const all = await new Promise((resolve, reject) => store.all((error, rows) =>
48         . error ? reject(error) : resolve(rows)))
49         for (const [sid, value] of Object.entries(all)) if (value.userId === account.i
50         . d) {
51             await new Promise((resolve, reject) => store.destroy(sid, error => error ?
52             . reject(error) : resolve()))
53             }
54             return { rows: [] }
55         }
56         throw Error('Unexpected SQL')
57     }
58     context.mock.method(pool, 'query', query)
59     context.mock.method(pool, 'connect', async () => ({ query, release() } {}))
60     cookie = await login(oldPassword)
61 })
62 async function request(path, method = 'GET', body, authCookie = cookie) {
63     const r = await fetch(base + path, { method, headers: { Cookie: authCookie || '', 'Con
64     . tent-Type': 'application/json', 'X-Requested-With': 'WorkTracker' }, body: body === undefi
65     . ned ? undefined : JSON.stringify(body) })
66     const data = await r.json()
67     const output = JSON.stringify(data)
68     for (const secret of ['password_hash', oldPassword, newPassword, account.password_hash
69     . ]) assert.ok(!output.includes(secret))
70     return { status: r.status, data, cookie: r.headers.get('set-cookie')?.split(';')[0] }
71 }
72 async function login(password) {
73     const r = await request('/auth/login', 'POST', { email: account.email, password }, '')
74     assert.equal(r.status, 200)
75     return r.cookie
76 }
77 const passwordBody = () => ({ currentPassword: oldPassword, newPassword, confirmPassword:
78 . newPassword })
79
80 test('me exposes safe profile dates/status and profile edits use only the session identity
81 . ', async () => {
82     const me = await request('/auth/me')
83     assert.equal(me.data.user.active, true); assert.equal(me.data.user.updatedAt, '2026-09
84     . -29')
85     const r = await request('/profile', 'PUT', { name: ' New Name ', email: ' NEW@example.
86     . invalid ' })
87     assert.equal(r.status, 200); assert.equal(r.data.name, 'New Name'); assert.equal(r.dat
88     . a.email, 'new@example.invalid')
89     assert.equal((await request('/auth/me')).data.user.email, 'new@example.invalid')
90 })
91 test('profile validation rejects blank, malformed, overlong and privileged fields', async
92 . () => {
93     for (const body of [{}, { name: '' }, { name: ' ' }, { name: 'x'.repeat(101) }, { emai
94     . l: 'bad' }, { email: 'x'.repeat(256) }, { name: null }, { email: [] }]) {
95         assert.equal((await request('/profile', 'PUT', body)).status, 400)
96     }
97 }

```

backend/test/profile.test.js (continued)

8dea970 | Listing segment 3 of 4

```

84     for (const key of ['id', 'role', 'active', 'userId', 'password', 'password_hash', 'sta
85     tus']) {
86         assert.equal((await request('/profile', 'PUT', { name: account.name, email: accoun
87         t.email, [key]: 'forged' })).status, 400)
88     }
89     assert.equal(statements.filter(({ sql }) => sql.startsWith('UPDATE')).length, 0)
90 })
91 test('duplicate profile email is safe and does not change account data', async () => {
92     const r = await request('/profile', 'PUT', { name: account.name, email: 'duplicate@exa
93     mple.invalid' })
94     assert.equal(r.status, 409); assert.match(r.data.error, /Email already exists/)
95     assert.equal(account.email, 'personal@example.invalid')
96 })
97 test('password change verifies current password, changes hash, revokes both sessions and p
98     ermits fresh login only', async () => {
99     const otherCookie = await login(oldPassword)
100    const previous = account.password_hash
101    const r = await request('/profile/password', 'PUT', passwordBody())
102    assert.equal(r.status, 200); assert.match(r.data.message, /sign in again/)
103    assert.notEqual(account.password_hash, previous)
104    assert.equal(await bcrypt.compare(newPassword, account.password_hash), true)
105    assert.equal(await bcrypt.compare(oldPassword, account.password_hash), false)
106    for (const old of [cookie, otherCookie]) assert.equal((await request('/profile', 'PUT'
107    , { name: 'No', email: account.email }, old)).status, 401)
108    assert.equal((await request('/auth/login', 'POST', { email: account.email, password: o
109    ldPassword }, '')).status, 401)
110    const fresh = await login(newPassword)
111    assert.equal((await request('/auth/me', 'GET', undefined, fresh)).data.user.id, accoun
112    t.id)
113    assert.ok(statements.some(({ sql }) => sql === 'COMMIT'))
114 })
115 test('wrong current password preserves hash and session', async () => {
116     const previous = account.password_hash
117     const r = await request('/profile/password', 'PUT', { ...passwordBody(), currentPasswo
118     rd: 'incorrect-current-password' })
119     assert.equal(r.status, 400); assert.equal(r.data.error, 'Current password is incorrect
120     ')
121     assert.equal(account.password_hash, previous)
122     assert.equal((await request('/auth/me')).data.user.id, account.id)
123 })
124 test('password policy, confirmation and malformed values are authoritative', async () => {
125     for (const fields of [{ currentPassword: null }, { currentPassword: {} }, { newPasswor
126     d: 'short' }, { newPassword: 'x'.repeat(73) }, { newPassword: '\U0001f600'.repeat(19) }, {
127     newPassword: null }, { confirmPassword: 'mismatch' }, { role: 'admin' }, { newPassword: o
128     ldPassword, confirmPassword: oldPassword }]) {
129         assert.equal((await request('/profile/password', 'PUT', { ...passwordBody(), ...fi
130         elds })).status, 400)
131     }
132     assert.equal(statements.filter(({ sql }) => sql.startsWith('UPDATE')).length, 0)
133 })
134 test('password attempts are limited per authenticated account', async () => {
135     let r

```

backend/test/profile.test.js (continued)

8dea970 | Listing segment 4 of 4

```

123     for (let i = 0; i < 11; i++) r = await request('/profile/password', 'PUT', {})
124     assert.equal(r.status, 429)
125   })
126   test('profile mutations require authentication and inactive accounts remain blocked', async
    . c () => {
127     for (const path of ['/profile', '/profile/password']) assert.equal((await request(path
    . , 'PUT', {}, {})).status, 401)
128     account.active = false
129     assert.equal((await request('/profile', 'PUT', {})).status, 401)
130   })
131   test('regular Users cannot submit Admin fields through status updates', async () => {
132     for (const key of ['title', 'description', 'priority', 'userId', 'role']) {
133       assert.equal((await request('/tasks/7/status', 'PATCH', { status: 'Completed', [ke
    . y]: 'forged' })).status, 403)
134     }
135     assert.equal(statements.filter(({ sql }) => sql.startsWith('UPDATE tasks')).length, 0)
136   })
137   test('login racing a password change destroys the newly saved stale session', async contex
    . t => {
138     const original = pool.query
139     let reads = 0
140     context.mock.method(pool, 'query', async (sql, values) => {
141       const result = await original(sql, values)
142       if (sql.includes('WHERE lower(email) = $1') && ++reads === 2) result.rows[0].passw
    . ord_hash = await bcrypt.hash(newPassword, 4)
143       return result
144     })
145     assert.equal((await request('/auth/login', 'POST', { email: account.email, password: o
    . ldPassword }, {})).status, 401)
146   })
147
148   test('a failed final login check does not leave an authenticated session', async context =
    . > {
149     const original = pool.query
150     context.mock.method(console, 'error', () => {})
151     context.mock.method(pool, 'query', async (sql, values) => {
152       if (sql.includes('FOR SHARE')) throw Object.assign(new Error('Unavailable'), { cod
    . e: 'ECONNREFUSED' })
153       return original(sql, values)
154     })
155     const failed = await request('/auth/login', 'POST', { email: account.email, password:
    . oldPassword }, {})
156     assert.equal(failed.status, 500)
157     const all = await new Promise((resolve, reject) => store.all((error, rows) => error ?
    . reject(error) : resolve(rows)))
158     // Only the pre-existing session from beforeEach should survive the failed attempt.
159     assert.equal(Object.values(all).filter(value => value.userId === account.id).length, 1
    . )
160     assert.equal((await request('/auth/me', 'GET', undefined, failed.cookie || {})).data.u
    . ser, null)
161   })

```

backend/test/startup.test.js

Automated tests | 28 source lines | 1,324 bytes

SHA-256: 2314f00f97749bde5fd2739ce599ba05d8a071949c42d71d02ec957e42b78743

```
1  const { test } = require('node:test')
2  const assert = require('node:assert/strict')
3  const net = require('node:net')
4  const { spawn } = require('node:child_process')
5  const { once } = require('node:events')
6  const { randomBytes } = require('node:crypto')
7  const { join } = require('node:path')
8
9  test('an occupied port fails startup without reporting a running server', async () => {
10     const occupied = net.createServer().listen(0)
11     await once(occupied, 'listening')
12     try {
13         const child = spawn(process.execPath, ['server.js'], {
14             cwd: join(__dirname, '..'), windowsHide: true,
15             env: { ...process.env, NODE_ENV: 'test', PORT: String(occupied.address().port)
16             ,
17             SESSION_SECRET: randomBytes(48).toString('hex'), DB_HOST: 'localhost',
18             DB_PORT: '5432', DB_NAME: 'unused', DB_USER: 'unused', DB_PASSWORD: 'test-
19             only-unused' },
20             timeout: 10000,
21         })
22         let output = ''
23         child.stdout.on('data', chunk => { output += chunk })
24         child.stderr.on('data', chunk => { output += chunk })
25         const [code] = await once(child, 'close')
26         assert.equal(code, 1)
27         assert.doesNotMatch(output, /Server is running/)
28         assert.match(output, /EADDRINUSE/)
29     } finally { await new Promise(resolve => occupied.close(resolve)) }
30 }
```

frontend/test/api.test.js

Automated tests | 116 source lines | 6,258 bytes

SHA-256: b52430a054ad6154d22b01b34382163fa35d930f21f7fade56024a938e7d6536

```
1 import { test } from 'node:test'
2 import assert from 'node:assert/strict'
3 import * as api from '../src/services/api.js'
4
5 const user = { id: 1, name: 'Test User', email: 'test@example.invalid', role: 'user' }
6 const task = { id: 2, title: 'Test Task', description: '', status: 'Open', priority: 'Medium', userId: 1 }
7
8 test('loads valid records and rejects incompatible responses', async context => {
9   const fetchMock = context.mock.method(globalThis, 'fetch', async () => Response.json([
10     user]))
11   assert.deepEqual(await api.getUsers(), [user])
12   fetchMock.mock.mockImplementation(async () => Response.json([task]))
13   assert.deepEqual(await api.getTasks(), [task])
14   for (const data of [{}, [null], [{ ...task, status: 'Unknown' }]]) {
15     fetchMock.mock.mockImplementation(async () => Response.json(data))
16     await assert.rejects(api.getTasks(), /unexpected data/)
17   }
18 })
19
20 test('writes use JSON and the correct REST methods and return saved records', async context => {
21   const calls = []
22   context.mock.method(globalThis, 'fetch', async (url, options) => {
23     calls.push({ url, method: options.method, body: JSON.parse(options.body) })
24     assert.equal(options.headers['Content-Type'], 'application/json')
25     assert.equal(options.headers['X-Requested-With'], 'WorkTracker')
26     assert.equal(options.credentials, 'same-origin')
27     return Response.json(url === '/api/users' ? user : task)
28   })
29   assert.deepEqual(await api.createUser({ name: user.name, email: user.email }), user)
30   assert.deepEqual(await api.createTask({ title: task.title, userId: 1 }), task)
31   await api.updateTaskStatus(2, 'Completed')
32   assert.deepEqual(calls, [
33     { url: '/api/users', method: 'POST', body: { name: user.name, email: user.email } },
34     { url: '/api/tasks', method: 'POST', body: { title: task.title, userId: 1 } },
35     { url: '/api/tasks/2/status', method: 'PATCH', body: { status: 'Completed' } },
36   ])
37 })
38
39 test('validation messages are preserved while server details are hidden', async context => {
40   const fetchMock = context.mock.method(globalThis, 'fetch', async () =>
41     Response.json({ error: 'Email already exists' }, { status: 409 }))
```

frontend/test/api.test.js (continued)

8dea970 | Listing segment 2 of 3

```
41     await assert.rejects(api.createUser(user), /Email already exists/)
42     fetchMock.mock.mockImplementation(async () => Response.json({ error: 'internal details
.   ' }, { status: 500 })))
43     await assert.rejects(api.getUsers(), { message: 'The service is unavailable. Please tr
.   y again shortly.' })
44     fetchMock.mock.mockImplementation(async () => new Response('', { status: 502 })))
45     await assert.rejects(api.getTasks(), /service is unavailable/)
46   })
47
48   test('HTML and malformed JSON produce readable errors', async context => {
49     const fetchMock = context.mock.method(globalThis, 'fetch', async () => new Response('<
.   html>Error</html>'))
50     await assert.rejects(api.getUsers(), /unexpected response/)
51     fetchMock.mock.mockImplementation(async () => new Response('', { headers: { 'Content-
.   Type': 'application/json' } })))
52     await assert.rejects(api.getUsers(), /unexpected response/)
53   })
54
55   test('connection failures produce a useful message without retrying writes', async context
.   => {
56     const fetchMock = context.mock.method(globalThis, 'fetch', async () => { throw new Typ
.   eError('fetch failed') })
57     await assert.rejects(api.createTask(task), /Unable to reach the service/)
58     assert.equal(fetchMock.mock.callCount(), 1)
59   })
60
61   test('canceling a page load cancels the fetch signal', async context => {
62     const controller = new AbortController()
63     context.mock.method(globalThis, 'fetch', async (url, options) => {
64       controller.abort()
65       assert.equal(options.signal.aborted, true)
66       throw new DOMException('Aborted', 'AbortError')
67     })
68     await assert.rejects(api.getUsers(controller.signal), { name: 'AbortError' })
69   })
70
71   test('restores safe authenticated accounts and unauthenticated sessions', async context =>
.   {
72     const fetchMock = context.mock.method(globalThis, 'fetch', async () => Response.json({
.   user }))
73     assert.deepEqual(await api.getSession(), user)
74     fetchMock.mock.mockImplementation(async () => Response.json({ user: null })))
75     assert.equal(await api.getSession(), null)
76     fetchMock.mock.mockImplementation(async () => Response.json({ user: { ...user, role: '
.   superuser' } })))
```

frontend/test/api.test.js (continued)

8dea970 | Listing segment 3 of 3

```
77     await assert.rejects(api.getSession(), /unexpected data/)
78   })
79
80   test('login and logout use session endpoints and safe responses', async context => {
81     const calls = []
82     context.mock.method(globalThis, 'fetch', async (url, options) => {
83       calls.push({ url, method: options.method })
84       return Response.json(url.endsWith('/login') ? { user } : { message: 'Signed out' }
85     .  )
86     })
87     assert.deepEqual(await api.login({ email: user.email, password: 'test-only-password' }
88 .  ), user)
89     assert.deepEqual(await api.logout(), { message: 'Signed out' })
90     assert.deepEqual(calls, [
91       { url: '/api/auth/login', method: 'POST' },
92       { url: '/api/auth/logout', method: 'POST' },
93     ])
94   })
95
96   test('expired protected requests notify App while invalid login preserves its form', async
97 .  context => {
98     const events = []
99     globalThis.window = { dispatchEvent: event => events.push(event.type) }
100     context.after(() => { delete globalThis.window })
101     context.mock.method(globalThis, 'fetch', async () => Response.json({ error: 'Please si
102 .  gn in' }, { status: 401 }))
103     await assert.rejects(api.getTasks(), /Please sign in/)
104     assert.deepEqual(events, ['session-expired'])
105     await assert.rejects(api.login({}), /Please sign in/)
106     assert.deepEqual(events, ['session-expired'])
107   })
108
109   test('a canceled response cannot expire the current session', async context => {
110     const events = []
111     globalThis.window = { dispatchEvent: event => events.push(event.type) }
112     context.after(() => { delete globalThis.window })
113     const controller = new AbortController()
114     context.mock.method(globalThis, 'fetch', async () => ({
115       ok: false, status: 401, headers: new Headers({ 'content-type': 'application/json'
116 .  }),
117     },
118     async json() { controller.abort(); return { error: 'Please sign in' } },
119   )))
120     await assert.rejects(api.getTasks(controller.signal))
121     assert.deepEqual(events, [])
122   })
123 }
```

frontend/test/management.test.js

Automated tests | 39 source lines | 3,110 bytes

SHA-256: 24e4ec8d298b477a2ad17bb22cc7f8cf25f12300a5c9ec561cd9f3bcc243c5f5

```

1  import { test } from 'node:test'
2  import assert from 'node:assert/strict'
3  import { filterUsers, filterTasks } from '../src/services/management.js'
4  import * as api from '../src/services/api.js'
5
6  test('user search, role/status filtering and sorting do not mutate source state', () => {
7      const users = [{ id: 1, name: 'Zoe', email: 'z@example.invalid', role: 'admin', active
      : true }, { id: 2, name: 'Amy', email: 'a@example.invalid', role: 'user', active: false }]
8      assert.deepEqual(filterUsers(users, {}).map(u => u.id), [2, 1])
9      assert.deepEqual(filterUsers(users, { search: ' Z@ ', role: 'admin', active: 'true' })
      .map(u => u.id), [1])
10     assert.equal(filterUsers(users, { active: 'false' }).length, 1)
11     assert.equal(filterUsers(users, { search: 'absent' }).length, 0)
12     assert.deepEqual(users.map(u => u.id), [1, 2])
13 })
14 test('task filters combine assignment, status, priority and title with stable sorting', ()
    => {
15     const tasks = [{ id: 1, title: 'Z task', status: 'Open', priority: 'High', userId: nul
    l }, { id: 2, title: 'A task', status: 'Completed', priority: 'Low', userId: 3 }]
16     assert.deepEqual(filterTasks(tasks, { sort: 'priority' }).map(t => t.id), [1, 2])
17     assert.deepEqual(filterTasks(tasks, { userId: '3', status: 'Completed', priority: 'Low
    ', search: 'A TASK' }).map(t => t.id), [2])
18     assert.deepEqual(filterTasks(tasks, { userId: 'unassigned' }).map(t => t.id), [1])
19     assert.deepEqual(filterTasks(tasks, { sort: 'title' }).map(t => t.id), [2, 1])
20     assert.deepEqual(tasks.map(t => t.id), [1, 2])
21 })
22 test('management API methods send correct paths, methods and JSON payloads', async context
    => {
23     const user = { id: 2, name: 'User', email: 'u@example.invalid', role: 'user', active:
    true }
24     const task = { id: 7, title: 'Work', description: '', status: 'Open', priority: 'High'
    , userId: 2 }
25     const calls = []
26     context.mock.method(globalThis, 'fetch', async (url, options) => {
27         calls.push([url, options.method || 'GET', options.body ? JSON.parse(options.body)
        : undefined])
28         assert.equal(options.credentials, 'same-origin')
29         assert.equal(options.headers['X-Requested-With'], 'WorkTracker')
30         return Response.json(options.method === 'DELETE' ? { message: 'Deleted' } : url.in
        cludes('users') ? user : task)
31     })
32     await api.getUser(2); await api.updateUser(2, user); await api.updateUserStatus(2, fal
    se); await api.deleteUser(2)
33     await api.getTask(7); await api.updateTask(7, task); await api.deleteTask(7)
34     assert.deepEqual(calls, [['/api/users/2', 'GET', undefined], ['/api/users/2', 'PUT', u
        ser], ['/api/users/2/status', 'PATCH', { active: false }], ['/api/users/2', 'DELETE', unde
        fined], ['/api/tasks/7', 'GET', undefined], ['/api/tasks/7', 'PUT', task], ['/api/tasks/7'
        , 'DELETE', undefined]])
35 })
36 test('blocked deletion preserves the backend business-rule message', async context => {
37     context.mock.method(globalThis, 'fetch', async () => Response.json({ error: 'This user
        cannot be deleted while tasks are assigned.' }, { status: 409 }))
38     await assert.rejects(api.deleteUser(2), /tasks are assigned/)
39 })

```

frontend/test/personal.test.js

Automated tests | 54 source lines | 3,882 bytes

SHA-256: 348888c00b9f279a94e0d8eb2753273b0ad9ba79c28d65d682d1d1951095dca5

```

1  import { test } from 'node:test'
2  import assert from 'node:assert/strict'
3  import { filterMyTasks, personalSummary, passwordError } from '../src/services/personal.js'
4  .
5  import * as api from '../src/services/api.js'
6  const tasks = [
7    { id: 1, title: 'First', description: 'Documentation', status: 'Open', priority: 'Low'
8    , userId: 2, createdAt: '2026-09-01' },
9    { id: 2, title: 'Second', description: null, status: 'Completed', priority: 'High', us
10    erId: 2, createdAt: '2026-09-02' },
11    { id: 3, title: 'Third', description: 'Review', status: 'In Progress', priority: 'Medi
12    um', userId: 2, createdAt: '2026-09-03' },
13  ]
14  test('personal task search includes description and combines status/priority filters', ()
15  => {
16    assert.deepEqual(filterMyTasks(tasks, { search: 'DOCUMENT ', status: 'Open', priority
17    : 'Low' }).map(t => t.id), [1])
18    assert.deepEqual(filterMyTasks(tasks, { search: 'second', priority: 'Low' }), [])
19    assert.deepEqual(filterMyTasks([], {}), [])
20  })
21  test('personal task sorting is stable and leaves shared state unchanged', () => {
22    for (const [sort, expected] of [['newest', [3, 2, 1]], ['oldest', [1, 2, 3]], ['priori
23    ty', [2, 3, 1]], ['status', [1, 3, 2]]]) {
24      assert.deepEqual(filterMyTasks(tasks, { sort }).map(t => t.id), expected)
25    }
26    assert.deepEqual(tasks.map(t => t.id), [1, 2, 3])
27  })
28  test('personal dashboard summarizes only supplied own tasks and excludes completed attenti
29  on items', () => {
30    assert.deepEqual(personalSummary(tasks), { total: 3, priorities: { High: 1, Medium: 1,
31    Low: 1 }, attention: [tasks[2], tasks[0]] })
32    assert.equal(personalSummary([]).attention.length, 0)
33  })
34  test('profile update uses the self-service endpoint and returns saved account data', async
35  context => {
36    const user = { id: 2, name: 'Changed', email: 'changed@example.invalid', role: 'user',
37    active: true }

```

frontend/test/personal.test.js (continued)

8dea970 | Listing segment 2 of 2

```

27     context.mock.method(globalThis, 'fetch', async (url, options) => {
28         assert.equal(url, '/api/profile'); assert.equal(options.method, 'PUT')
29         assert.deepEqual(JSON.parse(options.body), { name: user.name, email: user.email })
30         return Response.json(user)
31     })
32     assert.deepEqual(await api.updateProfile({ name: user.name, email: user.email }), user
33     .)
34 })
35 test('successful password change triggers authentication clearing but failures preserve the session', async context => {
36     const events = []
37     globalThis.window = { dispatchEvent: event => events.push(event.type) }
38     context.after(() => { delete globalThis.window })
39     const fields = { currentPassword: 'test-only-current', newPassword: 'test-only-replacement', confirmPassword: 'test-only-replacement' }
40     const mocked = context.mock.method(globalThis, 'fetch', async (url, options) => {
41         assert.equal(url, '/api/profile/password'); assert.equal(options.method, 'PUT')
42         assert.deepEqual(JSON.parse(options.body), fields)
43         return Response.json({ message: 'Your password has been changed. Please sign in again.' })
44     })
45     await api.changePassword(fields)
46     assert.deepEqual(events, ['password-changed'])
47     mocked.mock.mockImplementation(async () => Response.json({ error: 'Current password is incorrect' }, { status: 400 }))
48     await assert.rejects(api.changePassword(fields), /Current password is incorrect/)
49     assert.deepEqual(events, ['password-changed'])
50 })
51 test('password form validation checks UTF-8 byte limits and confirmation', () => {
52     const valid = { currentPassword: 'test-only-current', newPassword: 'test-only-new-secret', confirmPassword: 'test-only-new-secret' }
53     assert.equal(passwordError(valid), '')
54     for (const change of [{ currentPassword: '' }, { newPassword: 'short' }, { newPassword: '\U0001f600'.repeat(19) }, { confirmPassword: 'mismatch' }, { newPassword: valid.currentPassword, confirmPassword: valid.currentPassword }]) assert.notEqual(passwordError({ ...valid, ...change }), '')
55 })

```

.gitignore

Package and environment setup | 24 source lines | 329 bytes

SHA-256: bf254bedbbb923071c9b70321fe5a4c16f92f64b76805e28f8ea8bc404f00d69

```
1  # Dependencies / installed packages
2  node_modules/
3
4  # Environment variables and secrets
5  .env
6  .env.*
7
8  # Allow example environment files
9  !.env.example
10
11 # Build output / generated frontend build
12 dist/
13 build/
14
15 # Logs / runtime / development logs
16 *.log
17
18 # OS-generated files
19 .DS_Store
20 Thumbs.db
21
22 # Editor-specific files
23 .vscode/
24 .idea/
```

backend/.env.example

Package and environment setup | 13 source lines | 456 bytes

SHA-256: ee8bed21d5e632b083dfc5b93d8a077e3f9e08f0cd01e98572a72b01b14e245b

```
1 DB_HOST=localhost
2 DB_PORT=5432
3 DB_NAME=my_web_app_v2
4 DB_USER=postgres
5 DB_PASSWORD=your_local_postgres_password
6 PORT=3000
7
8 # Generate a private random value; do not use this placeholder.
9 SESSION_SECRET=replace_with_a_random_secret_at_least_32_characters
10 # Configure these privately only when bootstrapping the first Admin.
11 BOOTSTRAP_ADMIN_NAME=Local Admin
12 BOOTSTRAP_ADMIN_EMAIL=admin@example.com
13 BOOTSTRAP_ADMIN_PASSWORD=your_private_initial_admin_password
```

backend/package.json

Package and environment setup | 28 source lines | 907 bytes

SHA-256: 672acc312901cf1e5840316d1dc9bc9497e9d75562599f8f1fad0baa982d7870

```
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "start": "node --env-file=.env server.js",
8      "test": "node --test",
9      "db:check": "node --env-file=.env scripts/check-database.js",
10     "db:init": "node --env-file=.env scripts/init-database.js",
11     "db:migrate:accounts": "node --env-file=.env scripts/migrate-accounts.js",
12     "admin:bootstrap": "node --env-file=.env scripts/bootstrap-admin.js",
13     "account:activate": "node --env-file=.env scripts/activate-legacy-user.js",
14     "db:migrate:management": "node --env-file=.env scripts/migrate-management.js"
15   },
16   "keywords": [],
17   "author": "",
18   "license": "ISC",
19   "type": "commonjs",
20   "dependencies": {
21     "bcryptjs": "^3.0.3",
22     "connect-pg-simple": "^10.0.0",
23     "express": "^5.2.1",
24     "express-rate-limit": "^8.7.0",
25     "express-session": "^1.19.0",
26     "pg": "^8.23.0"
27   }
28 }
```

frontend/.env.example

Package and environment setup | 3 source lines | 166 bytes

SHA-256: ba66f15e1c9b5dd1f34e9bfaceb11bc100a618ef8e24fd40cc1e44509d40cb87

```
1 # Optional: change only when Express runs on another local port.  
2 # This is read by Vite's server, not exposed to browser code.  
3 API_PROXY_TARGET=http://127.0.0.1:3000
```

frontend/index.html

Package and environment setup | 13 source lines | 288 bytes

SHA-256: 7c4721b831ff09b21cabac10b96fe3722dbcb96f36f345b433bf4b6c6bd54442

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>My Web App V2</title>
7  </head>
8  <body>
9    <div id="root"></div>
10
11    <script type="module" src="/src/main.jsx"></script>
12  </body>
13 </html>
```

frontend/package.json

Package and environment setup | 24 source lines | 450 bytes

SHA-256: dd7ac10ab9b57a482f228651e15a7cec2fca74115e80bddc256d8c9e1b7bbea4

```
1  {
2    "name": "frontend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview",
10     "test": "node --test"
11   },
12   "keywords": [],
13   "author": "",
14   "license": "ISC",
15   "type": "module",
16   "dependencies": {
17     "react": "^19.3.0",
18     "react-dom": "^19.3.0"
19   },
20   "devDependencies": {
21     "@vitejs/plugin-react": "^6.1.1",
22     "vite": "^8.3.0"
23   }
24 }
```

frontend/vite.config.js

Package and environment setup | 17 source lines | 726 bytes

SHA-256: 45377a04639aebb07d3c7c661f49d33a04d247683006cab6b5cd82281adc194b

```
1 import { defineConfig, loadEnv } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 export default defineConfig(({ mode }) => {
5   // This server-only setting is not prefixed VITE_, so it is not shipped to the browser
6   .
7   const env = loadEnv(mode, process.cwd(), '')
8   const proxy = {
9     '/api': { target: env.API_PROXY_TARGET || 'http://127.0.0.1:3000', changeOrigin: true },
10  }
11  return {
12    // Enables React development tooling and production compilation.
13    plugins: [react()],
14    // Same-origin browser requests reach Express through Vite during local development
15    .
16    t.
17    server: { port: 5173, strictPort: true, proxy },
18    preview: { port: 4173, strictPort: true, proxy },
19  }
20 })
```